

基于场景线性规划的微网滚动购电调度

摘要

微网购电同时受到负载变化、光伏波动和合同调整规则影响，均值预测误差较小并不保证总费用较低。本文依据题目给定的 2025 年实测与预报数据，在十分钟区间上统一功率、电量和信息可用时刻，分别研究固定日型计划、非对称缺口风险下的日前采购、日内预报更新及波动电价下的购电调度。通过场景随机规划与滚动优化，联合确定采购和储能动作，并区分预测决策、实际执行及费用结算。场景仅使用决策时已经完成的历史观测。

针对问题一，核对原始数据维度、缺失与非负性，保留真实峰值及零光伏观测，按区间长度换算电量；建立含供需平衡、充放电效率、功率及储电量边界的线性规划，使用 HiGHS 求解，日购电费为 35126.95 元，相对无储能对照下降 26.90%。针对问题二，将五倍价紧急采购直接计入经验场景目标，以同日历史保持源荷关联，仅在一月选择场景与终端惩罚；二月至十二月费用为 14794949.72 元，比均值预测控制降低 10.10%。针对问题三，在整个预测窗口共享计划和储能动作以避免前视，利用零点、六点、十二点和十八点预报滚动调整尚未交付的合同；固定共同日前承诺及连续储电量后，四次更新相对仅零点更新节约 123506.61 元。针对问题四，将历史电价与源荷联合取样，日前费用为 15623134.83 元，相对均值控制降低 9.65%；四次更新费用为 16455780.60 元，相对共同合同的仅日前方案节约 155342.73 元。逐笔重算同一承诺链，固定与波动电价下逐次结算分别比最终净额多 96160.53 元和 98826.62 元，表明结算路径影响经济评价。各模型均以真实执行后的缺口计算紧急费用，年末恢复相同储电量；次日预览不形成当日承诺。完全信息结果仅作为事后下界，未参与在线决策，确保不同对照的状态与信息边界可核验。

模型以风险采购、共享动作和共同合同形成针对性改进，并保留一月启动账本，避免将二月初储电量直接重置。检验包括预测误差、两项参数的百分之十与百分之二十扰动、替代求解器全年回放及分块重采样。已有 237 项独立检查均通过；新增固定窗口检验显示目标值与动作稳定性应分别评价。四次相对三次更新的微小费用差在不同块长度下均不足以支持稳定优势。该框架适用于具备相应数据与合同规则的并网微网，结论限于当前年份、设备边界和计费解释。预测误差采用带单位指标描述，夜间零光伏不使用以实际值为分母的百分比误差。

关键词：场景随机规划 HiGHS 算法 微网购电 滚动优化 参数扰动检验

1 问题重述

1.1 背景与已知条件

小区微网需协调负载、光伏、储能与外网购电，兼顾供电平衡和费用。题目提供固定日型及 2025 年负载、光伏、预报和电价数据；储能容量 12000 千瓦时、功率 5000 千瓦，充放电效率各 0.9，储电量限制为 1200—10800 千瓦时。^[1]

1.2 研究任务

问题一制定固定日型下的日购电与储能计划；问题二在源荷不确定、紧急购电为五倍价时安排日前采购；问题三利用每日四次光伏预报调整计划并比较更新频次；问题四将相关策略推广至波动电价。各问还须输出指定日期明细、储能状态和紧急购电区间。

2 问题分析

2.1 数据特征与共同难点

数据同时包含十分钟实测、小时级预报和每日四个预报发布时间。相同时间标签不一定表示相同物理区间，发布时间也不同于预报覆盖时刻。若直接按表头文字连接数据，可能把计划与相邻区间的实测比较；若用完整年度数据构造当前预测，会引入决策时尚不可见的观测。因此，应先确定物理区间和信息可用时刻，再讨论预测或优化精度。

储能把相邻时段联结起来：本段充电改变后续可用电量，提前放电可能降低当前费用，却增加以后高价采购。采购不足与过量的代价不对称，且调整合同另有费用。因此，逐段追求预测误差最小或仅按低价充电、高价放电的规则，均不能完整表示本题的经济目标。比较策略时还需保持初始库存、日前合同及评价末状态一致。

2.2 四个问题的建模思路

问题一的日型已知，负载、光伏和分时价均为确定输入。储能递推、功率边界和购电成本可写成线性关系；在允许处置富余电量的题设下，可论证充放电互斥变换，避免为形式上的复杂性引入启发式算法。线性规划的可行性和目标值可由独立矩阵复求核验。

问题二的关键是把不确定净需求转化为经济风险，而不是先确定一个均值预测再被动付罚款。有限历史联合场景能保留负载与光伏的同时变化；样本平均近似将计划采购与各场景缺口代价放在一个目标内，使采购量随缺口损失调整。^[2] 场景只能来自自己完成历史，跨日储能必须从真实运行末状态衔接。

问题三在日前计划之外增加日内信息。滚动优化可以随预报更新调整剩余时段，但已交付区间不可回写，次日预览也不应被计作今日合同。场景经济模型预测控制已有微网应用研究。^[3] 本题需要进一步把不可前视的采购、储能动作和多次合同变更连在一起，才能解释更新频次的费用差。

问题四使电价本身成为未知量。相同缺口在不同价格下造成不同损失，源荷与价格的相关性因此具有经济意义。应在同一历史日期上联合取样，并明确预测价格用于决策、实际交付价格用于结算。题目并未提供售电收益、潮流或电压约束，不宜以其他微网模型的完整配置替代当前数据条件；微网预测控制的应用层次及建模范围可参照相关综述。^[4]

2.3 技术路线与比较设计

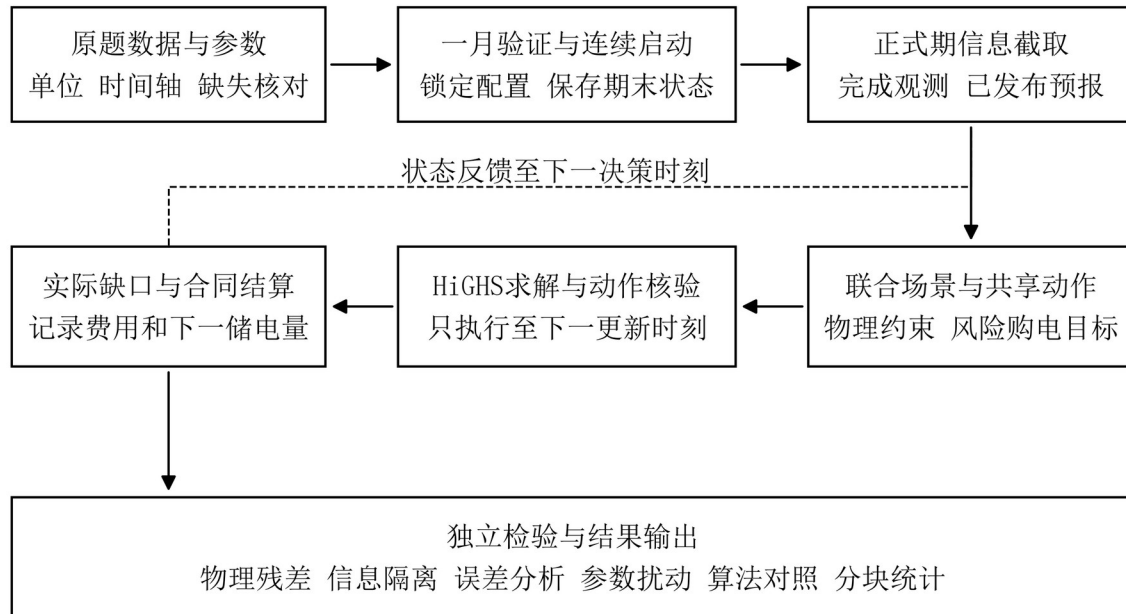


图 1 微网购电调度与检验技术路线

技术路线先完成单位和时间索引核对，再把设备状态与可用信息送入优化器。优化输出仅执行到下一允许更新时刻，随后以实际观测计算缺口和费用，并更新储电量。参数选择使用一月验证区间；正式运行、对照实验和补充诊断分开保存，避免依据正式期表现反向选择参数。

本文的针对性改进在于将五倍缺口代价直接纳入场景规划、用共享动作限制前视，并在频次比较中固定共同日前合同。前两项改变决策结构，后一项保证比较口径可解释。它们均可由对应程序、约束及对照结果核验；本文不将成熟的线性规划求解器本身作为新算法。

3 模型假设

(1) 单母线与成本范围假设。把微网视为统一交流母线，忽略未提供参数的线路损耗、自放电和寿命折旧，不引入售电收入。该简化与题给数据范围一致；它限定费用结论适用于购电和合同调整，不代表储能全寿命成本。

(2) 富余处置与供给假设。计划采购、紧急采购均非负；允许不用光伏及已付费富余电量，外网无另加的容量上限。题目未给相应上限或售电规则，因此以紧急购电闭合供需；紧急购电量不能解释为停电量。

(3) 离散化与信息可用假设。十分钟数据作为对应物理区间的代表功率，小时预报按零阶保持分配到六段；实测在区间结束后可用，预报在标注发布时间可用。这使不同数据尺度具有一致接口；存在通信延迟时须后移信息截止时刻。

(4) 结算解释假设。主口径按交付区间价格计费，取消量退原购电费后保留 50% 违约金，新增量按 1.5 倍价计；同一交付区间的多次调整使用相同价格。题意未完全确定退费与逐次收取方式，故另列不退款和逐次结算的同路径诊断。

(5) 局部经验分布假设。最近已完成的联合历史轨迹可作为当前不确定量的有限代表；只作局部近似，不假设全年严格平稳。采用滚动历史与一月验证降低跨期混用风险，并用月度、尾部及块长度检验识别这种近似的局限。

4 符号说明

表 1 主要符号及单位

符号	含义	单位
L_t, V_t	负载与光伏区间电量	kWh
B_t, A_t	0 点计划量与当前最终承诺	kWh
C_t, D_t	充电与放电母线电量	kWh
E_t	区间开始时储电量	kWh
Q_t, U_t	紧急采购与未用富余	kWh
p_t	交付区间电价	元/kWh
$H\tau, s, N\tau$	窗口集合、场景索引与场景数量	—
G_t	窗口内共享计划采购量	kWh
Δt	离散区间长度 (1/6)	h
$\eta c, \eta d$	充电效率与放电效率 (均为 0.9)	—
λ, ε	终端储电量惩罚与数值正则系数	元/kWh
$F\tau, \tau$	决策信息集与发布时间	—, h
$k, \Delta A_t^k$	调整次数索引与承诺变化量	—, kWh
J, f_t	总费用与调整费用函数	元
r_t^E	储电量递推残差	kWh

符号中的电量均以交流母线侧千瓦时计，储电量为绝对能量，不以百分比表示。上标 s 表示场景，上标 k 表示同一交付时段的第 k 次合同变化；时间索引用物理区间，预报发布时间另用 τ 表示，避免把预测提前量误当成执行时刻。

5 模型建立与求解

5.1 数据核对与统一时间尺度

附件 2—4 覆盖 2025 年 365 天，正式结果表覆盖 2 月 1 日至 12 月 31 日 334 天，每天 144 段，共 48,096 段。1 月 1 日 6000 kWh 是题给初值，不能不经运行过程移到 2 月。本文将一月用于有明示规则的设备启动和离线参数验证；正式比较自二月开始，所有方案读取同一启动末状态。

表 2 原始输入的完整性与范围检查

输入变量	数值数量	缺失/负值	范围
负载功率/kW	52560	0 / 0	1995.7176—7978.8849
光伏实测功率/kW	52560	0 / 0	0.0000—10216.2000
实时电价/(元/kWh)	52560	0 / 0	0.0076—1.7936
光伏预报功率/kW	35040	0 / 0	0.0000—9995.8875

输入检查按矩阵尺寸、日期唯一性、时间轴一致性、有限数值和预报覆盖执行。三组实测矩阵各含 52560 个值，1460 条预报各含 24 个提前小时值。检查未发现空白、非有限数值或负值；真实高峰与零光伏时段保留，未进行平滑、缩尾或插值补造。字段非负与完整并不能证明测量无误，故保留原文件及其校验值供追溯。

功率与电量必须在建模前区分。十分钟内负载或光伏的区间电量为代表功率乘以六分之一小时，额定功率 5000 千瓦对应每段最多 833.3333 千瓦时；电价保持元每千瓦时。储能约束中的效率分别作用于充入和放出过程，往返效率为 0.81，不能把两次效率折算为单次 0.9。

令 t 表示区间起点， $\Delta t=1/6$ h。源数据 00:10 解释为[00:00,00:10)的代表功率，依次至 24:00。该右端点代表值解释是统一采用的离散化约定。原模板的 0:10–0:20 等文字与列序存在偏移，因此保存位置键、源时间戳、物理区间、模板标签四个字段；计算与论文按物理键，官方表按原位置保存，并另附映射页。六个指定区间的起始位置分别为 61、73、85、97、109、121。

预报“提前 1 小时”对应发布后第一个整点，本文将该值作为前一个小时六个十分钟区间的代表值；例如 6 点发布的第 1 小时值用于 6:00–7:00，18 点的第 7 小时值用于次日 0:00–1:00。此零阶保持离散化不使用未来实际插值。凌晨、跨日和全天终点采用同一映射。

信息集包括决策时刻以前已完成的观测、截至该时刻发布的预报及已形成的合同。允许观测在物理区间结束后立即可得；若存在实际通信延迟，应在可用时刻中加入延迟量。

$F_{\tau} = \sigma(\text{观测结束时刻} \leq \tau; \text{预报发布时间} \leq \tau; \text{既有采购承诺})$ (1)

计划采购和储能控制必须可由 F_{τ} 决定。当前或未来的实际负载、出力与电价只进入事后执行评价，不能作为当前优化输入。完全信息下界单独命名为事后对照。

对预报提前量的映射属于离散化约定，而非对未来观测的修正。例如 6 点发布的第一小时预报覆盖 6 点至 7 点，其中每一段使用相同代表功率；18 点发布的第七小时值覆盖次日 0 点至 1 点。用于估计这类误差的历史轨迹，必须在当前发布时刻前完整结束，不能只凭历史日期早于今天就认为已经可知。

5.2 问题一的确定性储能购电模型

5.2.1 决策变量与物理约束

执行层由实际负载和光伏重新计算紧急缺口及未用能量。储能状态使用实际执行的充放电动作递推，前一日末值等于后一日初值。

$$A_t + V_t + D_t + Q_t = L_t + C_t + U_t; \quad Q_t, U_t \geq 0 \quad (2)$$

$$E_{t+1} = E_t + 0.9 C_t - \frac{D_t}{0.9} \quad (3)$$

$$1200 \leq E_t \leq 10800; \quad 0 \leq C_t, D_t \leq \frac{5000}{6} \quad (4)$$

供需平衡同时容纳紧急缺口与未用富余；问题一信息完全给定，故紧急变量固定为零。储电量在区间开始与区间结束之间递推，相邻日连续衔接。功率限制约束的是本段流量，容量边界约束的是累计状态，两者不能互相替代。

物理设备不允许同时充放电。线性松弛求解后令 $z=0.9C-D/0.9$ ，将动作替换为 $C'=\max(z,0)/0.9$ 、 $D'=0.9\max(-z,0)$ 。该变换保持储电量变化、降低循环损耗和所需供电，在允许免费处置富余、没有最小功率约束的本模型下不会破坏供需可行性。程序逐段验证转换后的互斥、状态、功率及情景供电约束，而不仅检查求解器返回成功。

上述互斥处理具有支配关系。若 $z \geq 0$ ，则 $C'=C-D/(\eta c \eta d)$ 、 $D'=0$ ，减少的母线需求为 $D(1/(\eta c \eta d)-1) \geq 0$ ；若 $z < 0$ ，则 $C'=0$ 、 $D'=D-\eta c \eta d C$ ，减少的母线需求为 $C(1-\eta c \eta d) \geq 0$ 。两种情形均有 $C' \leq C$ 、 $D' \leq D$ ，故额定功率上界保留，储电量轨迹不变，采购需求不减。由于本题允许不用富余且不设最小充放功率，此变换给出不劣的可行互斥解。若另加禁止弃电、负电价收益或最小运行时间等条件，需要重新论证，不能直接沿用该结论。

5.2.2 目标函数与求解步骤

问题一以 $Q=0$ 、 $A=B$ 建立确定性线性规划（LP），最小化 ΣpB ，并施加 $E_0=E_{144}=6000$ 。另按原附件独立构造约束矩阵重新求解，核对最优目标。存在并列最优时，不要求逐段动作唯一。无储能对照固定 $C=D=0$ ，只购入 $\max(L-V,0)$ 。

$$\min J_1 = \sum_{t=1}^{144} p_t B_t; \quad E_0=E_{144}=6000 \quad (5)$$

目标只计计划购电成本，约束使用本节供需平衡、储电量递推及上下界。首先从附件1读取144段价格、负载与光伏；其次建立稀疏线性约束矩阵，并将首末储电量固定为6000千瓦时；随后调用HiGHS求解，执行互斥变换并重新检查约束；最后按题面指定物理区间抽取计划及四小时充放电量。求解器报告成功后仍需检查矩阵与数据映射是否正确。

表 3 确定性调度与无储能对照

方案	全天购电量/kWh	全天费用/元
储能 LP	59,482.6990	35,126.95
无储能	61,789.9354	48,052.05

储能方案日费用为 35126.95 元，无储能对照为 48052.05 元，差额 12925.10 元，按表中费用计算约下降 26.90%。储能通过移动购电时点和消纳可用光伏改变费用；充放电损耗使“低价与高价之差”不足以单独判断套利，应结合 0.81 往返效率及容量约束。该确定性日型结论不代表同一固定计划可直接用于全年。

5.2.3 独立核验与结果表达

问题一另从原附件重建约束，而不是把模型自身保存的目标值作为证明。对于并列最优，费用与可行性一致是主要验收条件，逐段动作不必完全相同。结果工作簿保留全部 144 段，论文附录给出题面要求的区间值；显示舍入不参与后续求解或费用累计。

5.3 问题二的日前场景随机规划

5.3.1 场景构造与损失适配

场景仅来自决策时已完整观测的最近 20 条 24 小时轨迹；不足 20 条时使用全部可用轨迹，并记录实际样本数。负载、光伏误差和价格保持同一历史日期配对，以避免独立拼接破坏相关结构。以下两种场景模型在一月内比较，不在正式期根据结果挑选。

原始历史候选直接使用历史联合负载、光伏和价格轨迹。因果残差候选先以决策日前最近 4 个同星期日均值为中心；不存在同星期历史时取最近最多 4 个已完成日。每一历史误差也必须相对当时仅用更早日期生成的预测计算，不能用当前的拟合曲线回填过去。这遵循逐次向前验证的时间切分原则^[5]。

$$\hat{X}_t^s = \max(0, \hat{X}_{t|\tau} + (X_{h^*} - \hat{X}_{h^*|\tau_h})) \quad (6)$$

式中 X 可为负载、光伏或波动电价；固定价格使用给定曲线。负值预测截断为零，这是预测分布的非负约束，不改变原始观测。没有先验基线的首日不进入残差样本。跨日预测对次日目标使用对应星期属性，但统计截止仍是当前决策日之前。

先用历史可见信息形成预测中心，再叠加与该历史时点匹配的预测误差，可把日型变化与剩余不确定性分开表达。这里的非负截断施加于预测场景，原始观测仍原样保存。场景权重相等，反映有限经验分布；这不意味着未来每条轨迹确有相同发生概率。

紧急成本在优化中与计划费同时权衡。无储能单期下，若净需求分布连续且价格固定正值，最小化 $pq + 5pE[(N-q)^+]$ 给出 $F_N(q) = 0.8$ 。该推导解释损失不对称，不能把 80% 分位数机械替代含储能和跨时段耦合的式（9）。本文使用完整 LP 进行正式调度。更具体地，单期目标对采购 q 的导数为 $p - 5p \cdot \Pr(N > q)$ ，令其为零得到 0.8 分位点；离散经验分布取对应分位区间即可。若价格和缺口相关，期望应对联合分布计算，不能先把价格与需求独立平均。

5.3.2 共享动作与完整目标

在给定 $F\tau$ 的窗口内，对所有场景共享采购 G 、充电 C 、放电 D 和储电量；只有场景紧急量 Q^s 允许不同。未来储能动作也不选择“最有利的场景分支”作为承诺，这比只共享第一个执行块更保守，但消除了场景信息先于观测到达的隐性假设。窗口解只执行至下一次允许更新；新信息到达后重新优化。

$$G_t + D_t - C_t + Q_t^s \geq L_t^s - V_t^s; \quad Q_t^s \geq 0, \quad \forall s, t \in H_\tau \quad (7)$$

$$G_t^s = G_t, \quad C_t^s = C_t, \quad D_t^s = D_t, \quad E_t^s = E_t, \quad \forall s \quad (8)$$

$$\min \frac{1}{N_\tau} \sum_s \sum_t [p_t^s G_t + 5 p_t^s Q_t^s] + \lambda |E_{\text{末}} - 6000| + \varepsilon \sum_t (C_t + D_t) \quad (9)$$

式（9）用于未承诺的日前采购；已承诺的当前日区间改用第 5.4 节调整费用。 λ 为场景平均价格中位数乘惩罚倍数； $\varepsilon=10^{-7}$ 元/kWh 用于削弱无意义循环和并列解差异。它是明确的数值正则，实际费用表不计该项；不能据微小目标差宣称经济优势。真实评价终点强制 $E_{\text{末}}=6000$ ，中途分段保存状态不触发终端重置。

供需不等式中的松弛量对应可不用的富余能量，缺口则由非负紧急采购补足；两者不会产生售电收益。每个场景下的负载和光伏不同，但计划购电与电池动作在整个窗口内相同，只有紧急采购随场景变化。这样得到的动作可在当前信息下承诺，代价是相对于允许未来反馈分支的完整多阶段策略更保守。

将绝对终端偏差写成两个非负变量之差，将上调和下调写成非负分量后，目标与约束保持线性。20 个场景、144 个区间时，现有实现含 3746 个变量；稀疏矩阵只保存非零系数。已有成熟对偶单纯形与内点法实现可用于复求，因此无需为本题另设粒子群种群或遗传算法迭代上限。^[6]

5.3.3 一月选择与对照结果

一月 8—31 日用于比较两个场景候选和惩罚倍数 0.5、1、1.5，四类模型合计实际费用最小者被锁定；验证子实验使用相同 6000 首尾储电量。相同目标并列时按预先列出的顺序取首项。正式期不重新选候选或惩罚倍数。选择结果为逐日前推因果残差场景、倍数 1.0。

表 4 一月验证期候选策略费用

场景候选	惩罚倍数	一月验证合计费用/元
因果残差	0.5	6,296,205.83
因果残差	1.0	6,291,638.88
因果残差	1.5	6,291,638.88
原始历史	0.5	6,578,596.29
原始历史	1.0	6,575,237.21
原始历史	1.5	6,575,237.21

两种场景模型各比较三个惩罚倍数，共六个候选。因果残差模型在倍数 1.0 与 1.5 下的合计费用相同，按预先固定顺序选择 1.0；不能把两个并列值写成统计上显著的优劣。选定后将解析配置单独保存，正式期读取该结果，不读取尚未选择时的默认配置。

正式比较包括均值预测确定性采购和同场景无储能控制。前者将场景折叠为一个均值场景，后者固定充放电为零；采用共同日期、起始状态和最终状态。完全信息下界一次性使用正式期实际曲线求解，仅为事后最优信息对照。在线策略与其差额包含预测不足、有限场景、窗口和算法选择，不能全部解释为纯信息价值。

表 5 日前采购与可行基线

模式	方案	正式期总费用/元	紧急购电量/kWh
Q2	联合场景风险采购	14,794,949.72	304,382.5519
Q2	均值预测	16,457,210.18	1,084,623.2193
Q2	无储能	18,581,927.00	243,596.7610
Q2	完全信息事后下界	12,230,383.65	0.0000
Q4-2	联合场景风险采购	15,623,134.83	301,847.5698
Q4-2	均值预测	17,291,874.90	1,084,623.2193
Q4-2	无储能	19,297,857.40	244,173.5421
Q4-2	完全信息事后下界	12,783,021.74	0.0000

问题二联合场景费用为 14794949.72 元，相对均值预测控制减少 1662260.45 元，降幅约 10.10%；紧急购电由 1084623.2193 千瓦时降至 304382.5519 千瓦时。无储能方案的紧急电量更低，但总费用更高，说明紧急电量最少并不等于总费用最少。表内波动电价扩展采用相同比较逻辑，其建模解释见第 5.5 节。

完全信息下界允许提前知道正式期实际源荷，并在同一边界条件下求解。它只能作为事后参考，不能作为可实施方案参加排名；在线模型与它的费用差同时包含信息不足、场景近似、窗口截断和控制保守性，不宜全部称为预测误差成本。

5.4 问题三的滚动调整与承诺链

5.4.1 新预报如何进入模型

问题三/四-3 的光伏中心使用当次正式发布预报，场景误差为同一历史发布时间对应的整段实际小时均值减原预报；历史轨迹结束时刻不得晚于当前决策时刻。两个候选都保留这种发布时间对齐。场景是由真实历史误差构成的预测样本，并非声称未来实际会等于某一历史日。

日内更新针对尚未交付的时段。6 点获得新预报后，之前 36 段动作已经执行，不能用新解修改；新的充放电必须从 6 点真实储电量出发。18 点的 24 小时窗口包含次日预览，但只有本日剩余 36 段进入本日承诺账本。这个边界决定哪些量可被执行、哪些量只帮助近似未来价值。

先用只在 0 点求解的参考策略在线形成每一天的 B 及其参考状态。该参考过程本身只使用当日允许信息，可与实际调度并行运行。S0、S1、S2、S3 分别允许在 {0}、{0,6}、{0,6,12}、{0,6,12,18} 求解，但四组读取完全相同的 B。其他策略 0 点保持 B 锁定，只安排自身当前储电量下的储能；随后在允许时刻调整尚未交付的承诺。实际状态因过去更新而产生的差异属于处理结果，不另注入免费库存。

每次求解使用未来 24 小时；当前日剩余区间形成正式承诺，次日部分仅用于储能前瞻，不进入当前日调整账本。年末窗口按评价终点截断。已经执行的时段不得重写。S3 作为题目完整四时点机制的正式结果表口径；费用最低的受控方案另如实报告，不将“机制最完整”当作最优证明。

5.4.2 调整费用的分步推导

令 $x=A-B$ ，下调取消部分退原价后保留 50% 违约费，上调新增量按 1.5 倍价计。主费用与逐次费用分别为：

$$f_t(x) = p_t x + 0.5 p_t |x| \quad (10)$$

当变化量非负时，线性项与绝对值项相加得到 1.5 倍价；当变化量为负时，净费用变化为负的 0.5 倍价，即退回原价后仍承担取消量 50% 的违约费。按最终合同相对日前计划计费，得到主费用；若每次变更分别收取费用，则需累计每次承诺差额。

$$J_{\text{净}} = \sum_t [p_t B_t + f_t(A_t^* - B_t) + 5 p_t Q_t] \quad (11)$$

$$J_{\text{逐次}} = \sum_t [p_t B_t + \sum_k f_t(A_t^k - A_t^{k-1}) + 5 p_t Q_t] \quad (12)$$

两式严格使用同一正式承诺链和同一执行轨迹。每条承诺记录发布时间、决策时储电量、交付物理键、旧值、新值和可用历史截止。以价格 1 元、计划 100 kWh 说明：100→80 的主费用为 90 元，100→120 为 130 元；100→80→100 的净变化为 0，逐次费用比净额多 20 元。

在某交付时段各次调整均采用相同非负价格的本文口径下，由三角不等式：

$$J_{\text{逐次}} - J_{\text{净}} = 0.5 \sum_t p_t [\sum_k |\Delta A_t^k| - |\sum_k \Delta A_t^k|] \geq 0 \quad (13)$$

仅一次有效调整时可相等，因此审计不要求两种费用必须不同。若采用每次下单瞬时价格而非交付区间价，应重新定义合同价格和决策目标，上式不能直接套用。本文将上述差异作为计费假设的适用边界。

不退还取消部分原价的对照在相同路径上额外加 $\Sigma p(B-A \text{ 末})^+$ ；它反映计费解释变化，未重新优化策略，故不能称为另一制度下的最优策略收益。

同一时段先减后增会增加总变动幅度，而最终净变化可能为零。因此，逐次费用差来自合同路径，不能通过只比较最终购电量恢复。本文保留旧承诺、新承诺及发布时间，并以同一执行轨迹重算两类费用，以分离策略差异和计费解释差异。

5.4.3 指定日期轨迹与频次比较

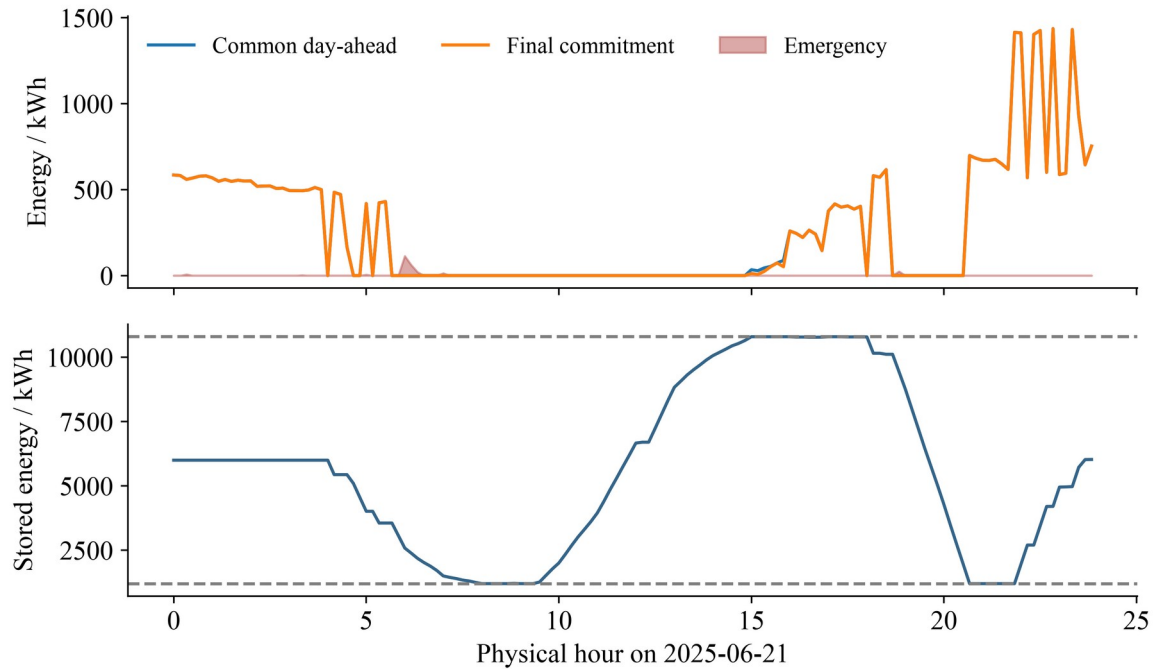


图 2 六月二十一日问题三的调度轨迹与储电量

图 2 展示六月二十一日的采购、紧急购电与储电量轨迹。上图电量对应十分钟区间，下图储电量是累计状态；两者峰值时刻无需相同。结合附录充放电明细，可检查充电是否增加下一状态、放电是否按效率扣除，再结合价格和负载分析经济原因。

Q3 在本样本共同日前合同下，主费用最低为 S2，费用 15,632,278.97 元；S3 相对 S0 减少 123,506.61 元。该差额来自本算法在更新信息下选择的后续采购和储能，不能推广成信息本身必然有害或必然有益。

如果扩大的信息集允许完全忽略新增预报，且优化覆盖相同状态合同下的全部可行策略，则较丰富信息的最优期望费用不会更高：旧策略仍属于新策略集合。这是可行策略集合包含关系的结果，而不是某次实现路径上的费用保证。本文采用有限窗口、有限经验场景和数值求解，因此既不保证每次更新降低实际费用，也不以 S2 略低于 S3 否定信息的一般价值。是否引入更新应依据受控费用、缺口和实现负担共同判断。

5.5 问题四的波动电价扩展

5.5.1 联合价格场景与决策结算分离

问题四沿用相同设备约束，改变价格的信息条件。固定价模型可直接使用题给分时曲线；波动价模型在决策时仅利用已完成历史生成价格场景，执行后才用附件 4 的实际交付价格结算。价格预报不能由当天完整实际价格回填，否则会把事后最优信息当作可实施输入。

在共同采购量下，计划费的场景期望可写为平均价格乘采购量；紧急量随场景不同，紧急费用必须保留每个场景的价格与缺口乘积。高价格是否与低光伏、高负载同时发生，会影响最优风险准备。把价格与净需求先分别平均，会丢掉这种相关结构，故场景按历史日期配对。

$$E[p_t Q_t] = E[p_t] E[Q_t] + Cov(p_t, Q_t) \quad (14)$$

目标中未承诺时段使用计划采购项，已承诺的本日区间使用调整费用项，不能两项重复计费。终端惩罚按场景平均价格的中位数缩放，正则项只用于数值选择；两者与真实结算费用分开记录。模型不额外假设未来实时价格已知，也未添加题外的负价套利机制。

5.5.2 四类正式结果及费用构成

表 6 正式期费用构成

模式	计划费/元	调整净费/元	紧急费/元	总费用/元
Q2	13,628,630.29	0.00	1,166,319.43	14,794,949.72
Q4-2	14,398,125.99	0.00	1,225,008.84	15,623,134.83
Q3	13,787,650.62	-94,795.02	1,939,629.88	15,632,485.48
Q4-3	14,529,578.19	-76,763.21	2,002,965.61	16,455,780.60

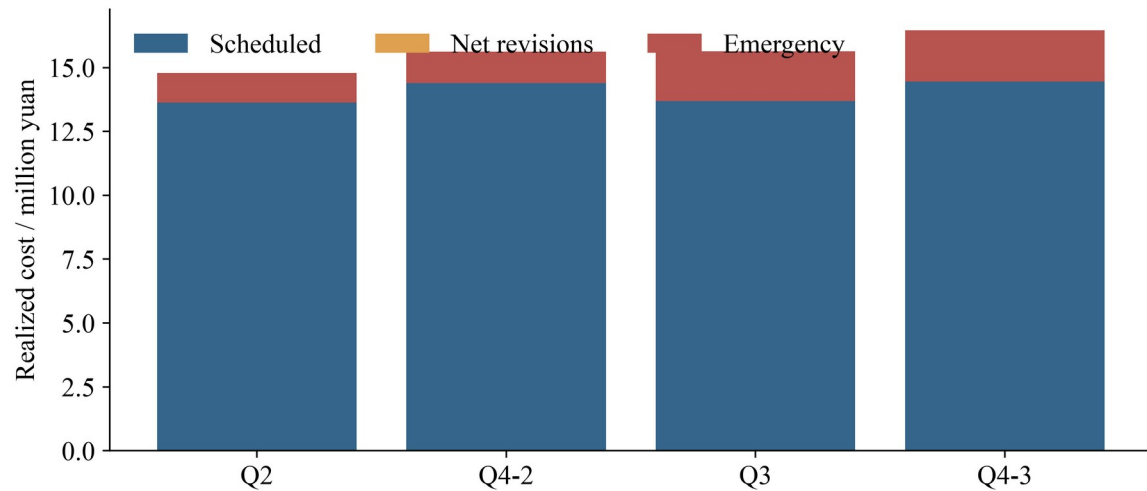


图 3 四类正式结果的计划、调整与紧急费用

问题四日前策略费用为 15623134.83 元，相对均值预测控制减少 1668740.06 元，约下降 9.65%。含四次日内更新的正式方案费用为 16455780.60 元。两类模型的信息结构和参考日前合同不同，因此不能把后者高于前者简单解释为光伏预报有害；更新机制的效果应在各自共同日前合同下比较。

表 7 更新频次的费用与紧急购电量

模式	时点策略	主费用/元	逐次费用/元	紧急购电量/kWh
Q3	S0	15,755,992.09	15,755,992.09	480,127.6923
Q3	S1	15,719,285.75	15,719,285.75	458,877.2938
Q3	S2	15,632,278.97	15,728,445.92	461,452.2838
Q3	S3	15,632,485.48	15,728,646.00	461,512.7475
Q4-3	S0	16,611,123.32	16,611,123.32	491,220.5871
Q4-3	S1	16,551,767.48	16,551,767.48	458,919.9717
Q4-3	S2	16,455,768.85	16,554,595.47	461,095.1851
Q4-3	S3	16,455,780.60	16,554,607.22	461,100.4099

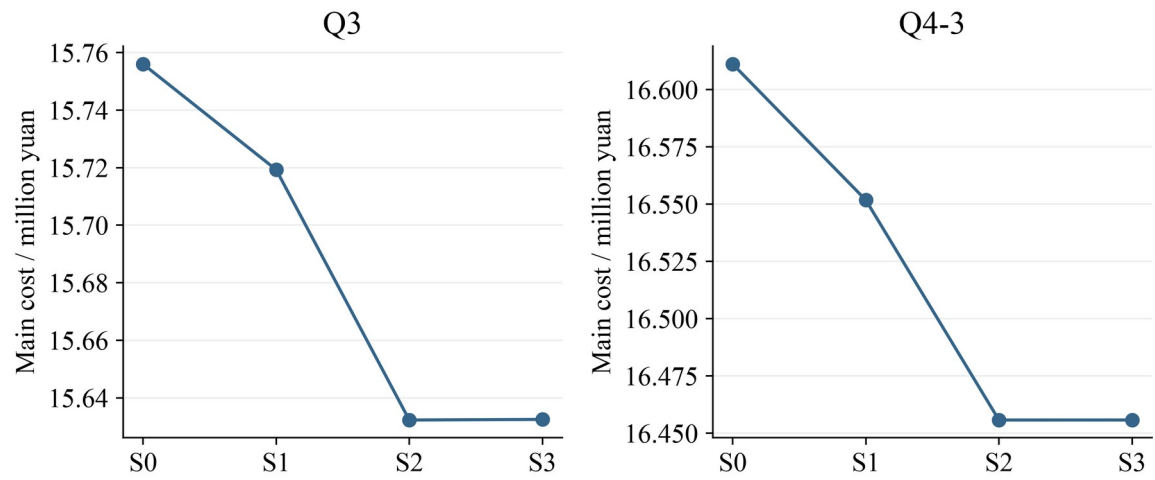


图 4 共同日前合同下的更新频次比较

Q4-3 在本样本共同日前合同下，主费用最低为 S2，费用 16,455,768.85 元；S3 相对 S0 减少 155,342.73 元。该差额来自本算法在更新信息下选择的后续采购和储能，不能推广成信息本身必然有害或必然有益。

从共同合同的结果看，固定和波动电价下 S3 相对 S0 分别节约 123506.61 元和 155342.73 元，约为参考费用的 0.78% 和 0.94%。S2 与 S3 的差额远小于这一尺度，后续需检验日际波动与求解器并列解对排序的影响。主结果表保留题目四次更新机制，不用事后表现最好的频次替换规定口径。

5.5.3 结算口径的同路径对照

表 8 同一承诺链的不同结算解释

模式	最终净额/元	同链逐次费用/元	不退原价费用/元
Q2	14,794,949.72	14,794,949.72	14,794,949.72
Q3	15,632,485.48	15,728,646.00	16,097,957.53
Q4-2	15,623,134.83	15,623,134.83	15,623,134.83
Q4-3	16,455,780.60	16,554,607.22	16,936,032.61

固定价滚动方案逐次结算比最终净额多 96160.53 元，波动价方案多 98826.62 元；这不是重新优化得到的新策略收益。无日内调整的日前模型没有承诺变化，故两种口径金额相同。取消部分不退款的结果同样只评价既有路径；若该制度成为正式规则，必须重新优化采购与储能动作。

5.6 算法实现与连续状态衔接

5.6.1 求解配置与终止条件

表 9 模型求解与实验配置

项目	设置	用途
求解环境	Python 3.12.14; SciPy 1.16.1	线性规划
常用数据依赖	NumPy 2.3.5; pandas 2.2.3	矩阵与记录
区间/窗口	10 分钟 / 24 小时	离散与滚动
场景数/终端倍数	20 / 1.0	一月选择后锁定
原始/对偶可行容差	均为 10^{-8}	可行性检查
内点法最优容差	10^{-9}	替代求解
吞吐量正则系数	10^{-7} 元/kWh	数值选择
统计重采样	每组 2000 次	区间估计

HiGHS 采用确定的线性规划可行性容差，无需外层随机迭代。模型的时间推进以允许更新时点为循环边界，求解返回失败时停止该实验并报告原因，不能把失败时段删除后仍累计为全年结果。每次成功求解后检查供需、储电量递推、功率和互斥，再输出至下一更新时点的动作。

算法 1 基于共同日前合同的滚动调度

输入：已选配置、当前储电量、允许更新时点、可用历史及发布预报。

- 1 用仅零点更新的参考策略形成各日共同日前合同 B 。
- 2 按时间顺序读取下一允许更新时间 τ 。
- 3 截取 τ 以前完整历史与截至 τ 已发布的预报，生成联合场景。
- 4 保持已执行区间不变；零点锁定共同 B ，后续允许调整剩余承诺。
- 5 求解共享采购与储能动作的线性规划，执行互斥变换与残差检查。
- 6 保存本日剩余区间的承诺变化；次日预览不写入正式合同。
- 7 仅执行至下一更新时点，以随后实际观测计算缺口、费用及状态。
- 8 未到评价终点则返回步骤 2；年末施加明确的末状态约束。

输出：逐时段采购、充放电、储电量、紧急缺口、费用与完整承诺链。

伪代码中的实际观测只出现在区间执行后的结算环节。下一窗口读取上次实际执行结束的储电量，而不是上次整段预测的终端值。由于全年正式期为 334 天， S_0 至 S_3 分别对应每日一至四次优化；问题三和问题四采用相同时间推进规则，从而使频次差异具有明确含义。

5.6.2 首日无历史与一月启动

缺少首日前史时，本文不从首日完整实际曲线构造 0 点预测。一月采用共同设备启动策略：储能保持待机，储电量按零充放电逐段递推；首日计划采购为零，实际负载减光伏的正缺口由紧急合同供给，负缺口弃置。随后各日按截至前一日已完成的经验净负载与价格进行单期采购。价格变化时以历史价格为权的经验 80%分位数求解采购量，一月仍不操作储能。所有计划、实际紧急量和费用逐段记录。

该启动合同提供缺少首日前史时的可实施基线，其费用单独列示，未进行一月策略最优性论证。由于每段 $C=D=0$ ，一月末 6000 kWh 由题给一月初值连续递推得到。训练候选回放属于独立的离线参数选择实验，与用于衔接正式期初始状态的启动策略分别保存。

表 10 一月启动与二月状态衔接

模式	一月启动费用/元	一月末储电量/kWh	二月初储电量/kWh
Q2	2,637,166.81	6,000.0000	6,000.0000
Q4-2	2,939,367.99	6,000.0000	6,000.0000
Q3	2,637,166.81	6,000.0000	6,000.0000
Q4-3	2,939,367.99	6,000.0000	6,000.0000

训练期候选回放与设备启动是两类计算：前者比较模型候选，后者确定正式期可承接的状态。即使二者都使用一月数据，也不能把离线候选的末状态冒充实际启动末状态。一月启动费用单列，二月至十二月主表费用不包含该部分；全年合计需要在同一模式下相加。

分段复验比较连续运行与在中途保存储电量后继续运行，不能因传入了临时 `end_date` 就额外强制储电量恢复。年末终端和训练子实验终端采用显式评价端点，普通中途截断不改变控制规则。

6 模型检验

6.1 物理残差与信息隔离

证据按“原始附件及哈希—物理时间映射—可用历史与发布预报—源码与配置—逐时段执行与承诺链—工作簿及论文数值”保存。独立审计直接从原始 Excel 读取实际源荷及价格，重新计算状态和费用；另建问题—线性规划，不以求解器状态代替数学核验。原结果表读取冻结记录；本次新增的数据检查和参数扰动另存输入范围、实例明细与汇总，并保留正文生成源文件。

本文独立检查共 237 项，失败 0 项。检查覆盖逐时段原数据匹配、物理区间、设备边界、真实缺口及费用、正式承诺连续性、共同日前合同和一月衔接。测试还覆盖未来观测隔离、分段状态不重置、不同充放电效率和年末边界。该结果是所列检查范围内的证据，不是模型最优性或外推有效性的证明。

未来信息扰动检验保持决策时已知数据不变，只改其后的实际负载、光伏和实时价，要求场景及当前购电/充放电决策不变；同时验证更晚发布的预报不会影响更早决策。对于前一日同一发布时间的 24 小时残差，其完整观测恰截至本日同一时刻，可在立即观测假设下使用；不是仅比较日期大小就认定因果性。

$$r_t^E = E_{t+1} - E_t - \eta_c C_t + \frac{D_t}{\eta_d} \quad (15)$$

物理残差由原始负载、光伏和已执行动作独立重算，供需式两侧统一为千瓦时；储电量残差使用前后状态与分别的充放电效率。信息隔离检验固定已知历史，只扰动未来观测；若当前决策随之改变，即使物理约束仍可行，也不满足在线使用条件。两类检查检验的是不同性质，不能互相替代。

源码还比较连续回放与中途保存状态后续跑的结果，并测试不同充放电效率与年末恢复约束。它们用于排除无意重置状态、混用效率以及窗口跨年处理错误。验证结论限定在测试覆盖范围，无法证明未建模线路约束、通信中断或未来极端天气下仍满足全部工程要求。

6.2 预测误差与月度差异

预测误差根据历史回放各执行时段采用的场景均值计算。问题三包含多个预报发布时间，其误差与问题二的差异同时受信息可用时点和模型影响。平均绝对误差 $MAE = \text{mean}(|\text{预测} - \text{实测}|)$ ，均方根误差 $RMSE = \sqrt{\text{mean}((\text{预测} - \text{实测})^2)}$ ，偏差为 $\text{mean}(\text{预测} - \text{实测})$ ；各指标与被预测变量使用相同单位。夜间光伏及接近零的观测不适合采用平均绝对百分比误差（MAPE）。

表 11 执行时刻预测误差

模式	变量	MAE	RMSE	偏差
Q2	负载/kWh	27.0520	36.3298	-0.4195
Q2	光伏/kWh	30.7103	61.3999	1.3741
Q4-2	负载/kWh	27.0520	36.3298	-0.4195
Q4-2	光伏/kWh	30.7103	61.3999	1.3741
Q4-2	电价/(元/kWh)	0.0423	0.0582	-0.0011
Q3	负载/kWh	27.0520	36.3298	-0.4195
Q3	光伏/kWh	35.0775	59.2895	0.3130
Q4-3	负载/kWh	27.0520	36.3298	-0.4195
Q4-3	光伏/kWh	35.0775	59.2895	0.3130
Q4-3	电价/(元/kWh)	0.0423	0.0582	-0.0011

日内预报对应光伏的均方根误差由 61.3999 降至 59.2895 千瓦时，约降低 3.44%，但平均绝对误差由 30.7103 升至 35.0775 千瓦时。两类指标反映不同误差形态，不能只选更有利的一项宣布全面改善；比较也同时受发布时刻不同影响，不是固定信息集上的独立预测算法竞赛。

负载预测的误差水平在四模式中相同，价格误差也只在波动电价模式出现。光伏夜间有大量零值，直接用实际值作分母会使百分比误差失真，所以采用带单位的绝对误差和均方根误差。费用评价还与误差发生时的价格及储能可用量有关，低平均误差不能保证低尾部缺口成本。

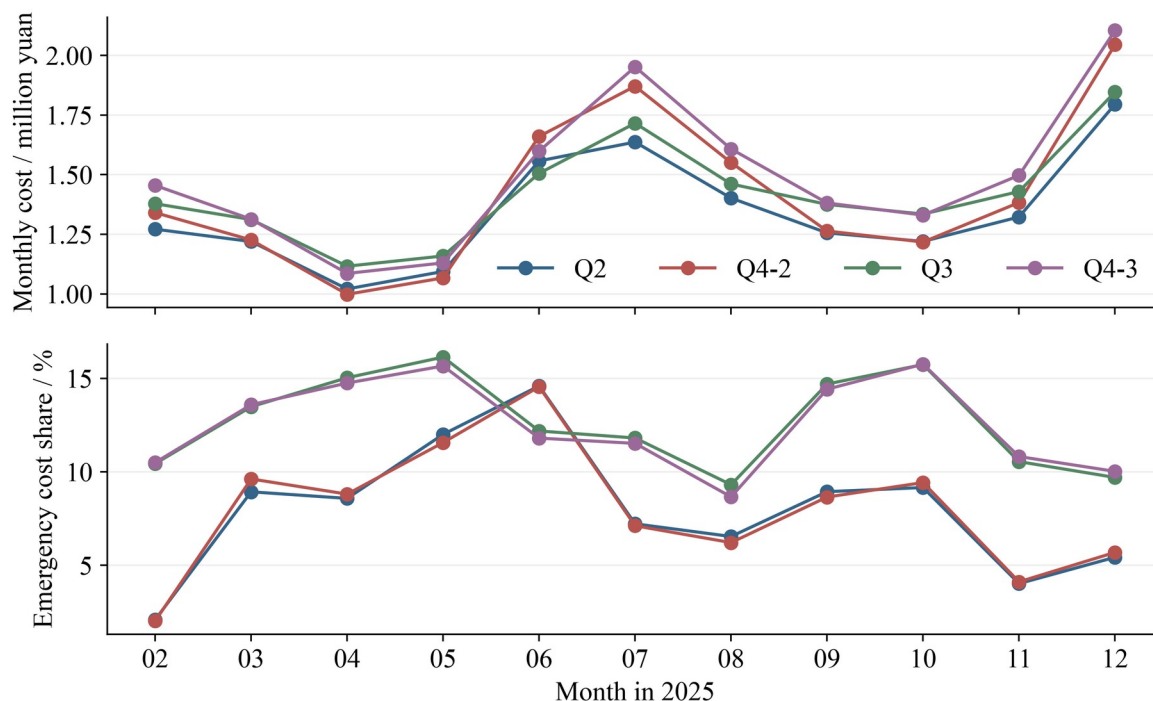


图 5 正式期月度费用与紧急采购占比

Q2 月总费最高为 2025-12，1,795,177.79 元；紧急费用占比最高为 2025-06，14.59%。

Q4-2 月总费最高为 2025-12，2,045,204.52 元；紧急费用占比最高为 2025-06，14.55%。

Q3 月总费最高为 2025-12，1,846,391.96 元；紧急费用占比最高为 2025-05，16.14%。

Q4-3 月总费最高为 2025-12，2,104,869.85 元；紧急费用占比最高为 2025-10，15.76%。

月度最高费用与紧急费最高占比出现于不同月份，说明采购总量、价格和预报缺口共同决定支出。年度汇总适合评价总经济性，月度分解用于定位季节性薄弱环节；后续若按季节重新校准，应仅使用当时已获得的数据，并另设验证区间，不能追溯修正已经发布的正式结果。

6.3 核心参数的梯度扰动

针对终端惩罚倍数和储能额定功率，分别在基准值的 80%、90%、100%、110%、120%上求解。样本固定为四个指定日期、两种电价、每天四个决策时点，共 32 个窗口；两参数单独变化，其余场景、初始状态、日前合同及算法保持一致。共 288 次实际求解，基准解在两组中复用，得到 320 条诊断记录。

表 12 两项参数梯度扰动的固定窗口结果

参数	扰动/%	目标变化最大绝对值/%	采购变化最大 L1/kWh	储电量最大变化/kWh
终端倍数	-20	1.83	5333.33	4800.00
终端倍数	-10	0.91	5090.88	4737.16
终端倍数	+10	0.83	4578.71	4800.00
终端倍数	+20	0.93	4578.71	4800.00
额定功率	-20	15.23	13247.96	1863.27
额定功率	-10	3.58	5696.33	914.61
额定功率	+10	0.49	4474.91	900.00
额定功率	+20	0.73	8162.11	1350.00

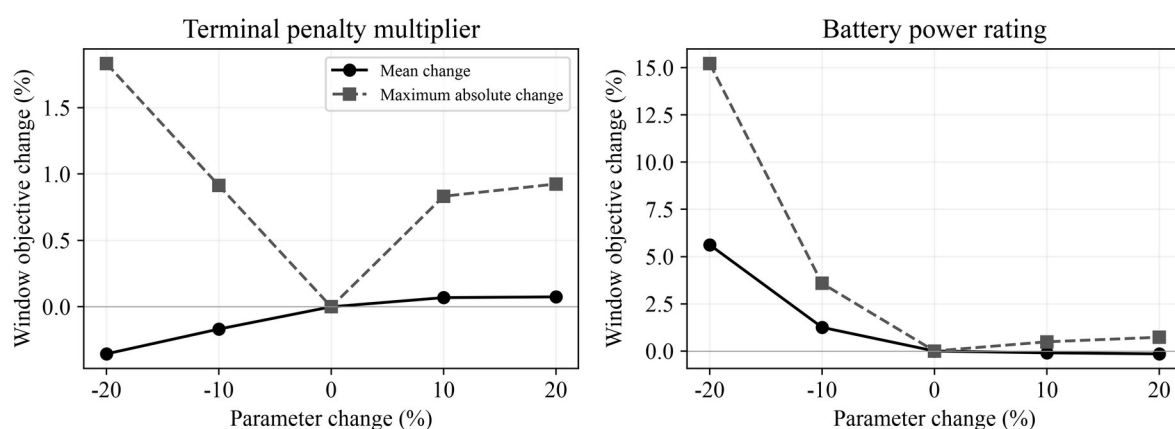


图 6 固定窗口对终端惩罚与功率参数的响应

终端惩罚倍数上下扰动 20% 时，32 个窗口的目标变化最大绝对值为 1.83%，但采购轨迹累计绝对变化最大达 5333.33 千瓦时，储电量最大变化达 4800 千瓦时。额定功率降低 20% 时，目标变化最大达 15.23%，超过 5%；提高 20% 时最大变化约 0.73%。这说明当前模型对功率下降更敏感，不能概括为所有参数扰动下均稳定。

相对变化使用补回日前合同常数项后的窗口目标作基值，防止求解器省略常数后出现接近零分母。终端惩罚改变时，目标本身的权重也改变，故这里衡量的是参数响应，不是同一费用函数上的策略优劣。功率扰动属于设备规格的假想变动，正式结果仍使用题给 5000 千瓦。

这些实验显示，目标值平缓并不意味着动作唯一。线性目标下存在并列或近似并列的动作，微小参数变化可能使求解器在不同轨迹间切换；需要同时报告采购量累计绝对变化和储电量最大变化。检验仅覆盖 32 个固定窗口，尚不能推断全年重新优化后的费用变化幅度。

6.4 替代求解器与全年回放

以四个指定日期、两种电价、每天四次决策构成 32 个固定 LP 实例，分别用 HiGHS 自动、对偶单纯形和内点法复求。HiGHS 求解器及其对偶单纯形实现的来源见文献^[6]。相同初始状态、场景、B 和窗口下比较目标及动作，检验可行性和并列解敏感性。该检验考察固定窗口的数值一致性，全年策略费用采用下述连续回放另行比较。

共完成 96 次固定实例复求。相对 HiGHS 自动法的目标最大绝对差为 $7.27596\text{e-}12$ 元，窗口采购量最大 L1 差为 292.46 kWh；最大储电量递推残差为 $1.023\text{e-}12$ kWh。目标接近而动作不一致时应视为并列解敏感性，其全年影响进一步通过下述连续回放检验。

在补充回放前固定协议，以相同一月末储电量、同一组 S0 参考日前承诺和一月已选定参数，分别对两种电价的 S2、S3 运行 HiGHS 内点法。每组连续回放 334 天，不在每日重置状态；保留每段动作并从原始附件重算费用。四组结果全部报告，不据本实验更换正式方案。

表 13 全年替代求解器回放费用

模式	求解方法	S2 费用/元	S3 费用/元	S3 减 S2/元
Q3	highs	15,632,278.97	15,632,485.48	206.51
Q3	highs-ipm	15,615,324.64	15,618,759.04	3,434.40
Q4-3	highs	16,455,768.85	16,455,780.60	11.75
Q4-3	highs-ipm	16,456,829.52	16,457,060.93	231.40

Q3 中，默认方法的 S3 减 S2 为 206.51 元，内点法为 3,434.40 元。两种方法的排序方向一致，但差距大小随数值方法变化。

Q4-3 中，默认方法的 S3 减 S2 为 11.75 元，内点法为 231.40 元。两种方法的排序方向一致，但差距大小随数值方法变化。

固定窗口检验回答“相同问题是否得到相同最优值”，全年回放回答“并列动作经状态递推后是否造成不同后续费用”。前者目标差约为数值容差，后者可能出现可见经济差异，这并不矛盾。正式方案和算法在结果比较前固定，不能根据本次事后诊断挑出费用较低的一条替换原结果。

6.5 时间相关与块长度稳健性

逐日配对差参考块自助法处理局部时间相关的思路^[7]，使用 7 天移动块、2000 次重采样估计区间，重采样对象是历史回放得到的逐日配对费用差。该区间描述这一个月份样本中的时序变动，并不排除有限样本偏差或分布漂移。

表 14 配对费用差及区间估计

模式	比较	配对差额/元	95%区间下端/元	95%区间上端/元
Q3	S1-S0	-36,706.34	-94,800.29	24,743.50
Q3	S2-S0	-123,713.12	-184,599.51	-64,757.47
Q3	S3-S0	-123,506.61	-183,617.04	-62,973.55
Q4-3	S1-S0	-59,355.84	-120,183.32	9,131.22
Q4-3	S2-S0	-155,354.47	-212,721.58	-91,199.75
Q4-3	S3-S0	-155,342.73	-216,836.95	-92,212.38

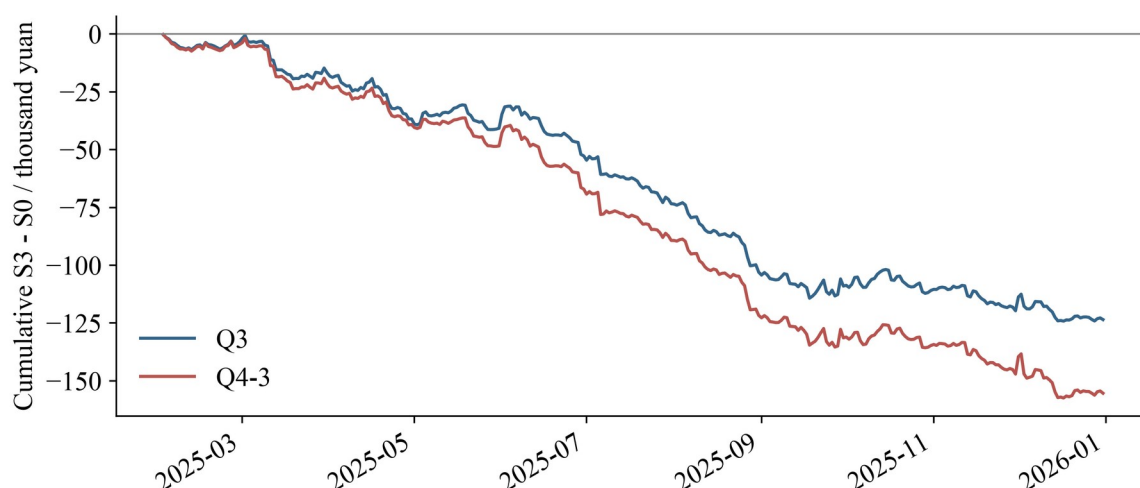


图 7 两种电价下 S3 相对 S0 累计费用差

累计差曲线反映全年不同阶段对总差额的贡献；持续低于零支持该年份样本的累计节约，局部回升表明部分日期更新后支出增加。不能把最终总额的符号解读成每天均有改善，也不能把按日重采样得到的区间当成另一年份的保证。

在同一 334 天费用差上，预先列出 3、7、14、28 天四种移动块长度，每种重采样 2000 次，使用固定随机种子 20260913。表中列 S3 减 S2，正数表示 S3 较贵；完整 S3 减 S0 区间也随支撑材料交付。年份存在季节变化，块重采样不保证严格平稳条件，区间用于判断样本内结论对重采样尺度的敏感性，不解释为下一年收益保证。

表 15 分块重采样对块长度的敏感性

模式	块长/天	实际差额/元	95%下端/元	95%上端/元
Q3	3	206.51	-306.79	942.80
Q3	7	206.51	-303.02	921.92
Q3	14	206.51	-241.45	956.14
Q3	28	206.51	-221.26	944.08
Q4-3	3	11.75	-96.55	108.69
Q4-3	7	11.75	-71.06	85.97
Q4-3	14	11.75	-70.47	81.74
Q4-3	28	11.75	-60.92	83.44

四种块长度下，S3 相对 S0 的费用差区间均低于零，S3 相对 S2 的区间均跨零。日内更新具有本样本经济价值，但尚不足以认定增加 18 点更新严格改善经济性。将更新计算或交易操作成本加入后，还需重新计算其盈亏阈值。

块自助法保留一段相邻日期，较独立抽样更能反映局部时间相关；但只观察一个年份，仍无法排除季节漂移和少数高影响日。S3 与 S2 的差额在不同块长下都不能与零稳定区分，因此结论应聚焦“相对仅日前更新的样本改善”，而不夸大最后一次更新的边际收益。

6.6 缺口暴露与尾部风险

紧急购电补足实际缺口，以下指标衡量额外采购暴露，不等同于停电概率。每日费用最高的 17 天定义为本样本上尾约 5%，其均值仅作描述性尾部指标；本文没有把目标函数改为 CVaR。

表 16 紧急购电与日费用尾部风险

模式	紧急时段占比	最高日费/元	上尾 17 日均费/元	紧急费占比
Q2	19.28%	85,284.12	71,024.39	7.88%
Q4-2	19.18%	96,029.77	83,271.29	7.84%
Q3	24.01%	78,340.53	71,191.53	12.41%
Q4-3	24.03%	93,916.64	84,263.76	12.17%

问题二与问题四日前方案紧急费用占比分别为 7.88% 和 7.84%，滚动方案为 12.41% 和 12.17%。这类跨模型差异同时受到参考合同与储能动作影响，需要与同口径总费用一起解读。尾部 17 日均费度量样本中高支出日期的集中程度，没有被直接放入当前优化目标。

风险评价仍有明确边界：有限历史场景不保证覆盖未来极端情况；负载/光伏/价格联合关系可能随季节改变；全部未来动作共享限制了场景补救自由度，可能增加保守性；有限窗口终端惩罚只是长期价值的近似。本文不引入附件未给出的外网限额或售电收益，也不把紧急购电量当作最终停电量。

7 模型改进与推广

7.1 已实现改进与对应证据

相对把历史场景折叠为均值的采购，当前模型显式计入五倍缺口代价，使计划与风险准备共同决定；对应的两种电价费用降幅为 10.10% 和 9.65%。相对仅在零点更新共同合同策略，允许日内更新具有本年份样本的节约证据。两组改进解决不同问题，不能相加为一个总提升率。

同时，保留联合历史日期、共享动作、连续储电量与真实承诺链，使数据可用性和计费含义能够被独立检查。这些设计提高了结果解释与复算的完整性，但不等于证明模型达到了所有可行在线策略中的全年最优。新参数扰动和原替代求解器检验进一步说明，经济目标和动作稳定性需要分开评估。

7.2 针对当前局限的改进方向

第一，改进终端价值近似。当前用储电量距 6000 千瓦时的绝对偏差代表窗口外价值，可能忽略次日时段价格和季节差异。可用仅含过去数据的多日回放估计分段线性终端价值，并用新的时间向前验证区间比较；没有完成该实验前，不能宣称能降低全年费用。

第二，改进场景代表性。在保持历史可见性与联合相关结构的前提下，可按季节和预测误差特征选择或压缩场景。已有研究在具体微网应用中将场景生成、削减与预测控制结合。^[8] 本题可借鉴其计算思路，但场景权重和数量应由本题的时间验证确定，不能移植他文的数值收益。

第三，扩展风险约束。若使用者需要约束供电违约概率，应先定义风险事件和可接受水平，再考虑数据驱动机会约束等方法；小样本下概率估计误差也须处理。^[9] 当前紧急采购能从外网补足，因此“发生紧急采购”与“未供电”是不同事件，不能直接把紧急时段占比设为停电概率约束。

7.3 可推广场景与新增条件

该框架可迁移至同样具备光伏、储能和外网合同的园区或商业建筑，前提是重新提供负荷、出力、可用预报、设备效率和结算规则。其可复用部分是信息时间索引、储能递推、场景供需约束和承诺记录；原参数、购电金额和频次排序不能随场景直接迁移。

若推广到含电动汽车、可调负荷或多微网系统，应增加接入状态、舒适度或任务期限，以及网络交换能力；若进入孤网，则外网紧急采购不能继续作为无限补救变量，需要建立失负荷或备用机组模型。这些变化会改变可行域和目标，不属于仅替换输入表格即可完成的推广。

8 模型优缺点评价与结论

8.1 优点

(1) 物理与经济口径统一。144 段设备约束、跨日储电量和合同费用在同一时间尺度计算；问题一独立复求与 237 项既有检查提供可核验依据，避免只以求解器状态评价可行性。

(2) 改进收益有对应基线。日前联合场景相对均值采购的费用减少约 166.23 万元和 166.87 万元；滚动频次使用共同日前合同，使信息更新比较不混入不同初始库存或不同参考采购。

(3) 检验结果同时保留有利与不利证据。论文给出预测误差、梯度扰动、96 次固定实例复求、四条全年替代回放及四种块长度统计；不把费用接近、动作不同或末次更新收益不显著的问题隐藏在总费用之后。

8.2 不足

(1) 单年份数据与有限历史场景难以覆盖未知极端情况，月度差异说明经验分布并非全年固定。未把当前置信区间推广为未来年份的收益保证。

(2) 共享整个窗口的储能动作有利于避免前视，但比多阶段可反馈策略更保守；终端价值仅为绝对偏差近似，参数响应存在明显动作切换。

(3) 线路、自放电、退化与外网容量未获得题目参数，当前成本未覆盖这些工程因素；结算解释改变时需要重新定义目标。固定窗口功率扰动不能替代设备选型的全年效益评估。

(4) 部分线性规划存在并列最优轨迹。不同算法可能把小动作差异传递为后续费用差；本样本 S2 与 S3 差距很小，尚无充分证据认定增加 18 点更新稳定改善费用。

8.3 结论

问题一通过线性规划求得题设边界内的确定性日计划；问题二的非对称紧急成本需要进入采购目标；问题三的更新价值应在共同日前合同和连续状态下评价；问题四必须保留价格与缺口的联合关系，并区分预测价格与实际结算价格。现有结果支持在当前数据与计费假设下使用风险采购和日内更新，同时保留有限场景、终端近似及频次排序的适用边界。

AI 工具使用声明

本轮使用 OpenAI Codex 辅助审查既有程序、实现修订、组织实验、复算结果和生成文档。保留原始输入，不编造观测数据或计算结果；预测场景与统计重采样均明确标记为数学估计。此前 AI 会话的具体型号和完整操作记录无法由现存项目全部核验，不猜填。参赛队仍需对题意假设、模型取舍、代码、数值和最终提交内容进行人工审阅。详细范围和记录索引见 AI 工具使用详情.pdf。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题: 微网与外部电网电力调控策略及附件[Z]. 2026.
- [2] SHAPIRO A, DENTCHEVA D, RUSZCZYNSKI A. Lectures on stochastic programming: modeling and theory[M]. 3rd ed. Philadelphia: SIAM, 2021. DOI:10.1137/1.9781611976595.
- [3] ALARCON M A, ALARCON R G, GONZALEZ A H, et al. A scenario-based economic-stochastic model predictive control for the management of microgrids[J]. Sustainable Energy, Grids and Networks, 2023, 36: 101205. DOI:10.1016/j.segan.2023.101205.
- [4] JOSHAL K S, GUPTA N. Microgrids with model predictive control: a critical review[J]. Energies, 2023, 16(13): 4851. DOI:10.3390/en16134851.
- [5] HYNDMAN R J, ATHANASOPOULOS G. 预测: 方法与实践[M/OL]. 3 版. Melbourne: OTexts, 2021[2026-09-13]. <https://otexts.com/fpp3cn/>.
- [6] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119-142. DOI:10.1007/s12532-017-0130-5.
- [7] KUNSCH H R. The jackknife and the bootstrap for general stationary observations[J]. The Annals of Statistics, 1989, 17(3): 1217-1241. DOI:10.1214/aos/1176347265.
- [8] LIN Y, LI L, ZHANG J, et al. A scenario-based stochastic model predictive control approach for microgrid operation at an Australian cotton farm under uncertainties[J]. International Journal of Electrical Power & Energy Systems, 2024, 159: 110025. DOI:10.1016/j.ijepes.2024.110025.
- [9] BABIC L, LAURICELLA M, CEUSTERS G, et al. Data-driven non-parametric chance-constrained model predictive control for microgrids energy management using small data batches[J]. Frontiers in Control Engineering, 2023, 4: 1237759. DOI:10.3389/fcteg.2023.1237759.

附录 1 数据与题面指定计算结果

表格按物理区间抽取。问题三及问题四-3 均为 S3 完整机制口径；表中同时列 0 点计划和最终承诺，不将两种量混用。所有电量单位为 kWh，费用单位为元。全精度参与计算，四位小数只在展示时舍入；紧急表按每天连续正缺口区间聚合。原始 Excel、完整冻结明细和中间计算表随配套支撑材料交付，文件索引见附录 2；完整数据不以抽样展示表替代。

Q1 2025-01-01 指定结果

表 17 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	480.4125	480.4125
14:00-14:10	0.0000	0.0000
16:00-16:10	445.4317	445.4317
18:00-18:10	531.8940	531.8940
20:00-20:10	0.0000	0.0000

表 18 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
59,482.6990	59,482.6990	35,126.95

表 19 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,500.0000	0.0000
4:00-8:00	833.3333	6,365.8412
8:00-12:00	4,787.9643	1,702.9970
12:00-16:00	5,286.0352	91.1014
16:00-20:00	0.0000	5,780.1319
20:00-24:00	5,333.3333	2,859.8681

表 20 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

Q2 2025-03-20 指定结果

表 21 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	662.0628	662.0628
14:00-14:10	0.0000	0.0000
16:00-16:10	521.3045	521.3045
18:00-18:10	701.0510	701.0510
20:00-20:10	0.0000	0.0000

表 22 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
71,611.5551	71,611.5551	45,330.57

表 23 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,500.0000	0.0000
4:00-8:00	833.3333	4,986.8567
8:00-12:00	5,898.7854	3,653.1433
12:00-16:00	5,481.1266	577.7287
16:00-20:00	0.0000	5,666.2340
20:00-24:00	5,333.3333	2,973.7660

表 24 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 25 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	19.0054
1:10-1:30	42.3965
2:30-2:40	0.8868
3:00-3:40	106.3099
3:50-4:00	0.7579
4:50-5:20	40.5585
5:50-6:00	9.6804
8:20-8:30	0.0281
15:30-16:50	136.6221
18:40-19:10	79.7381
20:00-20:20	20.2454
21:20-21:30	9.9067
23:30-23:50	43.2042

Q2 2025-06-21 指定结果

表 26 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	0.0000	0.0000
14:00-14:10	0.0000	0.0000
16:00-16:10	270.9544	270.9544
18:00-18:10	0.0000	0.0000
20:00-20:10	0.0000	0.0000

表 27 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
44,317.3693	44,317.3693	25,430.97

表 28 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	925.2727	0.0000
4:00-8:00	833.3333	2,546.3943
8:00-12:00	4,007.4024	32.7666
12:00-16:00	2,796.9521	36.8372
16:00-20:00	0.0000	5,869.8662
20:00-24:00	5,333.3333	2,770.1338

表 29 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 30 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	7.1975
3:20-3:30	2.6717
6:00-6:10	27.2964

Q2 2025-09-23 指定结果

表 31 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	389.9034	389.9034
14:00-14:10	0.0000	0.0000
16:00-16:10	495.7207	495.7207
18:00-18:10	756.8145	756.8145
20:00-20:10	0.0000	0.0000

表 32 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
64,683.9027	64,683.9027	54,413.55

表 33 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,500.0000	0.0000
4:00-8:00	833.3333	6,370.3856
8:00-12:00	5,660.4829	2,269.6144
12:00-16:00	5,234.9501	185.3007
16:00-20:00	0.0000	5,560.6287
20:00-24:00	5,333.3333	3,079.3713

表 34 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 35 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:30-1:20	171.6364
1:30-2:40	179.8680
3:10-3:20	7.2253
3:40-4:00	38.1843
4:20-4:30	6.3537
4:50-5:20	64.3631
5:40-5:50	17.4609
6:00-6:30	39.9485
6:50-7:10	46.4634
9:20-11:10	687.3832
12:10-12:20	34.5086
14:20-15:30	393.8956
15:50-17:50	639.8084
18:00-18:50	216.4812
19:00-19:30	100.8129
19:50-22:00	586.8745
22:10-22:40	92.9401
23:00-23:40	95.4675

Q2 2025-12-21 指定结果

表 36 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	1,060.2059	1,060.2059
14:00-14:10	0.0000	0.0000
16:00-16:10	904.1864	904.1864
18:00-18:10	731.0644	731.0644
20:00-20:10	0.0000	0.0000

表 37 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
103,683.5072	103,683.5072	67,672.11

表 38 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,500.0000	0.0000
4:00-8:00	1,666.6667	1,508.3333
8:00-12:00	5,833.3333	7,806.6667
12:00-16:00	8,333.3333	2,835.0000
16:00-20:00	0.0000	5,614.8196
20:00-24:00	5,333.3333	3,025.1804

表 39 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 40 连续紧急购电区间

紧急购电物理区间	紧急购电量
3:00-3:10	8.2484
3:30-3:50	25.2708
11:00-11:10	1.2873
17:20-17:30	6.0454
18:40-19:00	6.6983
19:40-20:00	59.0221

Q3 2025-03-20 指定结果

表 41 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	850.5195	572.9210
14:00-14:10	0.0000	0.0000
16:00-16:10	691.5551	670.3114
18:00-18:10	701.0163	701.0163
20:00-20:10	0.0000	0.0000

表 42 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
73,153.5596	68,035.7056	53,295.77

表 43 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,486.8644	0.0000
4:00-8:00	833.3333	6,288.2413
8:00-12:00	5,443.1711	2,351.7587
12:00-16:00	6,101.9723	711.5661
16:00-20:00	0.0000	5,666.2340
20:00-24:00	5,333.3333	2,973.7660

表 44 日初与日末储能状态

0 点储电量	24 点储电量
6,011.8220	6,000.0000

表 45 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	12.6675
1:10-1:30	41.4065
2:30-2:40	0.8868
3:00-3:40	106.3099
3:50-4:00	0.7579
4:50-5:20	40.5518
5:50-6:40	283.9429
7:00-7:30	148.6680
8:00-8:40	389.2008
9:00-9:20	231.9733
10:00-10:30	75.2595
11:00-11:50	276.4465
12:30-12:50	185.5360
13:10-14:00	102.8504
14:50-15:00	91.4396
15:40-16:00	185.6210
16:30-17:00	215.6824
17:50-18:00	1.6308
18:40-19:10	79.8939
20:00-20:20	20.2454
21:20-21:30	9.9067
23:30-23:50	43.2042

Q3 2025-06-21 指定结果

表 46 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	0.0000	0.0000
14:00-14:10	0.0000	0.0000
16:00-16:10	259.5906	259.5906
18:00-18:10	0.0000	0.0000
20:00-20:10	0.0000	0.0000

表 47 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
39,027.9639	38,924.9185	23,844.92

表 48 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	0.0000	0.0000
4:00-8:00	0.0000	4,320.0000
8:00-12:00	6,083.2092	9.2682
12:00-16:00	4,594.8997	0.0000
16:00-20:00	27.5462	5,893.4659
20:00-24:00	5,366.1146	2,768.8465

表 49 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,029.5032

表 50 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	7.1975
3:20-3:30	2.6720
5:00-5:10	5.6007
6:00-6:30	195.8855
7:00-7:10	13.9667
18:50-19:00	24.3090

Q3 2025-09-23 指定结果

表 51 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	556.9921	435.2244
14:00-14:10	0.0000	0.0000
16:00-16:10	697.7418	697.7418
18:00-18:10	755.3255	755.3255
20:00-20:10	0.0000	0.0000

表 52 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
71,018.2496	67,255.4981	56,892.92

表 53 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,469.4905	0.0000
4:00-8:00	832.9685	5,845.3760
8:00-12:00	6,055.9581	2,821.5471
12:00-16:00	5,546.0894	757.6585
16:00-20:00	16.6471	5,574.1128
20:00-24:00	5,357.0539	3,079.3713

表 54 日初与日末储能状态

0 点储电量	24 点储电量
6,057.7015	6,021.3485

表 55 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:30-1:20	164.5246
1:30-2:40	179.8680
3:10-3:20	7.2253
3:40-4:00	38.1843
4:20-4:30	6.3539
4:50-5:10	54.8209
5:40-5:50	17.4609
6:00-6:30	133.6420
7:00-7:30	248.2242
8:00-8:30	163.7542
10:00-10:10	34.2439
11:00-11:10	21.9809
11:40-11:50	1.5358
12:50-13:00	12.1870
13:30-14:00	174.9619
14:40-15:00	156.3889
15:40-16:00	219.8262
16:30-17:00	178.4692
17:20-18:50	511.2547
19:00-19:30	100.8152
19:50-22:00	586.8756
22:10-22:40	92.9401
23:00-23:50	99.7459

Q3 2025-12-21 指定结果

表 56 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	1,006.9192	1,006.9192
14:00-14:10	0.0000	0.0000
16:00-16:10	965.4095	965.4095
18:00-18:10	731.0633	731.0633
20:00-20:10	0.0000	0.0000

表 57 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
101,836.4072	99,908.8762	68,839.39

表 58 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	4,475.2658	0.0000
4:00-8:00	1,647.9174	1,600.4611
8:00-12:00	5,833.3333	7,465.6904
12:00-16:00	8,213.1269	2,975.9722
16:00-20:00	0.4947	5,614.8196
20:00-24:00	5,362.9476	3,025.1804

表 59 日初与日末储能状态

0 点储电量	24 点储电量
6,027.0131	6,026.6528

表 60 连续紧急购电区间

紧急购电物理区间	紧急购电量
3:00-3:10	8.2484
3:30-3:50	25.2708
7:00-7:20	85.2606
8:00-8:10	36.6391
9:00-9:20	57.0566
11:00-12:00	687.2493
12:50-13:00	3.0959
13:50-14:00	20.9081
14:50-15:00	11.7177
15:50-16:00	55.8293
16:40-17:00	91.7416
17:20-17:30	6.2231
18:40-19:00	6.6997
19:40-20:00	59.0221

Q4-2 2025-03-20 指定结果

表 61 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	662.0628	662.0628
14:00-14:10	0.0000	0.0000
16:00-16:10	521.3045	521.3045
18:00-18:10	701.0510	701.0510
20:00-20:10	0.0000	0.0000

表 62 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
71,751.2395	71,751.2395	46,667.50

表 63 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	3,333.3333	0.0000
4:00-8:00	2,833.3333	5,283.2858
8:00-12:00	5,898.7854	3,704.8972
12:00-16:00	5,101.5338	597.0756
16:00-20:00	0.0000	5,666.2340
20:00-24:00	5,333.3333	2,973.7660

表 64 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 65 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	19.0054
1:10-1:30	42.3965
2:30-2:40	0.8868
3:00-3:40	109.9422
3:50-4:00	0.7579
4:50-5:20	42.8569
5:50-6:00	9.6804
15:30-16:50	136.6221
18:40-19:10	79.7381
20:00-20:20	20.2454
21:20-21:30	9.9067
23:30-23:50	43.2042

Q4-2 2025-06-21 指定结果

表 66 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	0.0000	0.0000
14:00-14:10	0.0000	0.0000
16:00-16:10	270.9544	270.9544
18:00-18:10	496.1851	496.1851
20:00-20:10	0.0000	0.0000

表 67 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
44,903.6591	44,903.6591	22,307.40

表 68 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	0.0000	0.0000
4:00-8:00	2,247.6359	2,559.5579
8:00-12:00	4,303.0422	40.1968
12:00-16:00	2,062.8868	57.2328
16:00-20:00	0.0000	5,869.8662
20:00-24:00	6,263.5118	3,523.5784

表 69 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 70 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	7.1975
3:20-3:30	2.6717
6:00-6:10	27.2964

Q4-2 2025-09-23 指定结果

表 71 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	389.9034	389.9034
14:00-14:10	0.0000	0.0000
16:00-16:10	495.7207	495.7207
18:00-18:10	0.0000	0.0000
20:00-20:10	0.0000	0.0000

表 72 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
64,704.0238	64,704.0238	56,947.39

表 73 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	5,333.3333	0.0000
4:00-8:00	0.0000	6,352.9335
8:00-12:00	5,691.2229	2,287.0665
12:00-16:00	5,038.1602	50.8003
16:00-20:00	0.0000	6,312.6995
20:00-24:00	5,333.3333	2,327.3005

表 74 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 75 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:30-1:20	171.8013
1:30-2:40	179.8680
3:10-3:20	7.2253
3:40-4:00	38.1843
4:20-4:30	6.3537
4:50-5:20	70.1346
5:40-5:50	17.4609
6:00-6:30	39.9485
6:50-7:10	46.4634
9:20-11:10	658.6726
12:10-12:20	34.5086
14:20-15:30	411.5260
15:50-17:50	639.8084
18:00-18:50	216.4812
19:00-19:30	100.8129
19:50-22:00	586.8746
22:10-22:40	92.9401
23:00-23:40	97.1327

Q4-2 2025-12-21 指定结果

表 76 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	1,060.2059	1,060.2059
14:00-14:10	0.0000	0.0000
16:00-16:10	904.1864	904.1864
18:00-18:10	731.0644	731.0644
20:00-20:10	0.0000	0.0000

表 77 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
103,577.5086	103,577.5086	79,771.49

表 78 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	5,333.3333	0.0000
4:00-8:00	0.0000	833.3333
8:00-12:00	5,833.3333	7,806.6667
12:00-16:00	8,696.6951	3,129.3230
16:00-20:00	0.0000	5,614.8196
20:00-24:00	5,333.3333	3,025.1804

表 79 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 80 连续紧急购电区间

紧急购电物理区间	紧急购电量
3:00-3:10	8.2484
3:30-3:50	25.2708
11:00-11:10	1.2873
17:20-17:30	6.0454
18:40-19:00	6.6983
19:40-20:00	59.0221

Q4-3 2025-03-20 指定结果

表 81 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	850.5195	570.8778
14:00-14:10	309.1426	0.0000
16:00-16:10	691.5551	687.3114
18:00-18:10	701.0163	701.0163
20:00-20:10	0.0000	0.0000

表 82 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
72,867.5694	68,055.5090	54,809.53

表 83 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	3,333.3333	0.0000
4:00-8:00	2,000.0000	6,278.9662
8:00-12:00	5,443.1711	2,361.0338
12:00-16:00	6,101.9723	711.5661
16:00-20:00	0.0000	5,666.2340
20:00-24:00	5,328.5294	2,976.6885

表 84 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	5,992.4292

表 85 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	19.0054
1:10-1:30	42.3965
2:30-2:40	0.8868
3:00-3:40	109.9422
3:50-4:00	0.7579
4:50-5:20	42.9154
5:50-6:40	278.5855
7:00-7:30	148.6680
8:00-8:40	382.4816
9:00-9:20	231.9733
10:00-10:30	75.2595
11:00-11:50	274.2590
12:30-12:50	181.9861
13:10-14:00	102.8504
14:50-15:00	91.4396
15:30-16:00	167.7821
16:30-17:00	192.2386
17:50-18:00	12.0391
18:40-19:10	79.8939
20:00-20:20	20.2454
21:20-21:30	9.9067
23:30-23:50	43.2042

Q4-3 2025-06-21 指定结果

表 86 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	0.0000	0.0000
14:00-14:10	0.0000	0.0000
16:00-16:10	259.5906	259.5906
18:00-18:10	572.3498	572.3498
20:00-20:10	0.0000	0.0000

表 87 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
39,488.2755	39,426.6223	20,827.64

表 88 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	0.0000	3,540.3852
4:00-8:00	1,669.7950	2,099.7834
8:00-12:00	6,083.2092	9.2682
12:00-16:00	4,594.8997	0.0000
16:00-20:00	27.5462	5,892.1757
20:00-24:00	6,263.5118	3,521.6217

表 89 日初与日末储能状态

0 点储电量	24 点储电量
5,964.0385	6,002.1773

表 90 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:20-0:30	7.1975
3:20-3:30	2.6720
5:00-5:10	5.6007
6:00-6:30	195.8855
7:00-7:10	13.9667
18:50-19:00	24.3090

Q4-3 2025-09-23 指定结果

表 91 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	66.3309	0.0000
12:00-12:10	556.9921	435.2244
14:00-14:10	211.4517	157.5829
16:00-16:10	697.7418	697.7418
18:00-18:10	0.0000	0.0000
20:00-20:10	0.0000	0.0000

表 92 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
70,773.8684	67,327.8786	59,667.96

表 93 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	5,333.3333	0.0000
4:00-8:00	0.0000	5,987.9969
8:00-12:00	6,042.8171	2,652.0031
12:00-16:00	5,200.6404	467.2005
16:00-20:00	14.2894	6,324.2739
20:00-24:00	5,333.3333	2,327.3005

表 94 日初与日末储能状态

0 点储电量	24 点储电量
6,000.0000	6,000.0000

表 95 连续紧急购电区间

紧急购电物理区间	紧急购电量
0:30-1:20	171.8013
1:30-2:40	179.8680
3:10-3:20	7.2253
3:40-4:00	38.1843
4:20-4:30	6.3539
4:50-5:20	70.1352
5:40-5:50	17.4609
6:00-6:30	133.6420
7:00-7:30	236.0701
8:00-8:30	163.7542
10:00-10:10	34.2439
11:00-11:20	26.2146
11:40-11:50	2.4173
13:30-14:00	174.1958
14:40-15:00	192.0331
15:40-16:00	193.0308
16:30-17:00	187.2556
17:20-18:50	511.2547
19:00-19:30	100.8152
19:50-22:00	586.8756
22:10-22:40	92.9401
23:00-23:40	97.1327

Q4-3 2025-12-21 指定结果

表 96 指定物理区间购电量

物理区间	0 点计划量	最终承诺量
10:00-10:10	0.0000	0.0000
12:00-12:10	1,006.9192	1,006.9192
14:00-14:10	0.0000	0.0000
16:00-16:10	965.4095	965.4095
18:00-18:10	731.0633	731.0633
20:00-20:10	0.0000	0.0000

表 97 全天购电及费用汇总

全天计划量	全天最终量	全天总购电费
101,586.3615	99,749.3345	81,948.38

表 98 指定物理区间储能动作

物理区间	充电量	放电量
0:00-4:00	5,314.1330	0.7858
4:00-8:00	0.0000	938.9771
8:00-12:00	5,833.3333	7,462.1538
12:00-16:00	8,211.2025	2,975.3438
16:00-20:00	0.4947	5,614.8196
20:00-24:00	5,343.6832	3,025.1804

表 99 日初与日末储能状态

0 点储电量	24 点储电量
6,018.1534	6,009.3148

表 100 连续紧急购电区间

紧急购电物理区间	紧急购电量
3:00-3:10	8.2484
3:30-3:50	25.2708
7:00-7:20	85.2606
8:00-8:10	36.6391
9:00-9:20	57.0566
11:00-12:00	767.0957
12:50-13:00	3.0959
13:50-14:00	20.9081
14:50-15:00	11.7177
15:50-16:00	55.8293
16:40-17:00	91.7416
17:20-17:30	6.2231
18:40-19:00	6.6997
19:40-20:00	59.0221

附录 2 运行环境与完整源代码

数值流水线入口为支撑材料/run_pipeline.sh；本次论文排版另用论文修订源文件/build.py，旧流水线的文档步骤仍生成旧版。原始附件路径在 config/final.yaml 中配置，也可用 CUMCM_SOURCE_ROOT 覆盖。数据校验、模型训练、正式期运行与独立审查依序执行，任一步失败即停止。记录环境为 Python 3.12.14、NumPy 2.3.5、SciPy 1.16.1、pandas 2.2.3 及 openpyxl 3.1.5，版本明细见 evidence/environment_versions.json。配套包中“原始附件”保存输入 Excel；frozen/columns.tar.xz 保存完整冻结明细，用 scripts/unpack_frozen.py 恢复。年度与训练费用分开，三种结算口径分栏；中间统计保存在 evidence 及 research 目录，原文件与完整精度值均保留。

表 101 数据与复算证据索引

证据	位置
正式明细	frozen/q*_dispatch.csv（解包后）
真实承诺链	frozen/decision_versions.csv（解包后）
训练与解析配置	evidence/january_parameter_validation.csv 及 frozen/resolved_config.json
独立复核	audit/independent_validation.json
源码与输入校验	evidence/source_provenance.json
可执行入口	run_pipeline.sh
数学约束测试	tests/test_closed_loop.py

支撑材料按功能列示如下。原模型与审查源码在附录 2.1 给出，新增参数扰动程序见附录 2.2；排版源文件另包交付。完整文件校验清单见包内 SHA256SUMS.json。

表 102 支撑材料目录与用途

目录或文件	内容
src/model	全部模型与数据加载程序
src/audit	独立逐时段及工作簿审核程序
scripts	运行、补充实验、图表、文稿、打包程序
tests	数学约束和未来观测扰动测试
config/final.yaml 与 requirements.txt	配置及依赖
frozen 与 workbooks	原正式结果及五份指定工作簿
research	补充全年回放、尾部、月度、误差与统计诊断
evidence 与 audit	来源、源码附录清单及复验记录
charts 与 logs	图像及运行日志
AI 工具使用详情.pdf	真实使用范围及仍需人工核验事项

2.1 完整源程序与运行配置

以下逐文件收录原工程 32 个程序与配置文件。第三方依赖按 requirements.txt 安装；工作簿导出还需 Node 与 @oai/artifact-tool。程序以支撑材料内原始文件运行，版面换行不改变源码；独立数值复验仅需 Python。本次论文生成源文件与运行方法见论文修订源文件目录。

config/final.yaml

SHA256 49e195de542a215f9483bbae47b362703bd7eeb51177439936b3cecd42c06570

```
{
  "schema_version": "3.1-closed-loop",
  "source_root": "input",
  "output_root": "outputs/c_final_v1",
  "study": {
    "year": 2025,
    "training_start": "2025-01-01",
    "training_end": "2025-01-31",
    "official_start": "2025-02-01",
    "official_end": "2025-12-31",
    "initial_soc_kwh": 6000.0,
    "carry_soc_from_january": true,
    "terminal_soc_kwh": 6000.0,
    "startup_policy": "January_idle_battery_empirical_procurement"
  },
  "storage": {
    "capacity_kwh": 12000.0,
    "soc_min_kwh": 1200.0,
    "soc_max_kwh": 10800.0,
    "power_kw": 5000.0,
    "charge_efficiency": 0.9,
    "discharge_efficiency": 0.9,
    "interval_minutes": 10
  },
  "forecast": {
    "scenario_count": 20,
    "scenario_history": "latest_completed_24h_trajectories",
    "pv_hour_mapping": "right_endpoint_zero_order_hold",
    "scenario_model_candidates": [
      "raw_history",
      "causal_residual"
    ],
    "scenario_model": "raw_history"
  },
  "rolling": {
    "horizon_hours": 24,
    "settlement": "final_relative_to_common_00_plan",
    "sequential_sensitivity": "same_formal_commitment_chain",
    "soft_penalty_multipliers": [
      0.5,
```

```

    1.0,
    1.5
  ],
  "selected_soft_penalty_multiplier": 0.5,
  "selection_method": "minimum combined January 8-31 holdout cost across four modes;
equal 6000 endpoints",
  "terminal_rule": "soft_absolute_deviation; forced only at explicit evaluation endpoint"
},
"cost": {
  "down_adjustment_multiplier": 0.5,
  "up_adjustment_multiplier": 1.5,
  "emergency_multiplier": 5.0,
  "throughput_tiebreak_yuan_per_kwh": 1e-07
},
"audit": {
  "numeric_tolerance": 1e-07,
  "feasibility_tolerance": 1e-08,
  "display_decimals": 4,
  "emergency_display_omit_threshold_kwh": 5e-05
},
"experiments": {
  "official_arm": "joint_empirical",
  "official_scenario": "S3",
  "schedules": {
    "S0": [
      0
    ],
    "S1": [
      0,
      6
    ],
    "S2": [
      0,
      6,
      12
    ],
    "S3": [
      0,
      6,
      12,
      18
    ]
  }
},
"controls": [
  "mean_forecast",
  "no_storage"
],
"solver_methods": [
  "highs",
  "highs-ds",

```

```

    "highs-ipm"
  ],
  "representative_dates": [
    "2025-03-20",
    "2025-06-21",
    "2025-09-23",
    "2025-12-21"
  ]
}
}

```

requirements.txt

```

SHA256 6566f256706346dc54cc9e02a45794ab938bc296a5878ca670f886052e1702a3
numpy==2.3.5
scipy==1.16.1
pandas==2.2.3
openpyxl==3.1.5
python-docx==1.2.0
pypdf==6.10.0
Pillow==12.3.0
matplotlib==3.11.2

```

run_pipeline.sh

```

SHA256 af6e9596d23433b106aeb6a2856e0bed0e0a027152d85c84d0f5acb544eadd90
#!/usr/bin/env bash
set -euo pipefail
PROJECT_DIR="$(cd "$(dirname "$0")" && pwd)"
cd "$PROJECT_DIR"
RUNTIME_BASE="${HOME}/.cache/codex-runtimes/codex-primary-runtime/dependencies"
PY_BIN="${CODEX_PYTHON_BIN:-${RUNTIME_BASE}/python/bin/python3}"
NODE_BIN="${CODEX_NODE_BIN:-${RUNTIME_BASE}/node/bin/node}"
export PYTHONPATH="$PROJECT_DIR/work/python_deps:$PROJECT_DIR${PYTHONPATH:+:$PYTHONPATH}"
FINAL="outputs/c_final_v1"
mkdir -p "$FINAL/logs" "$FINAL/qa" work/fontconfig
if [[ ! -e node_modules ]]; then ln -s "$RUNTIME_BASE/node/node_modules"
node_modules; fi
if [[ "${1:-}" != "--skip-model" ]]; then
  "$PY_BIN" -m unittest discover -s tests -v 2>&1 | tee
"$FINAL/logs/constraint_and_causality_tests.log"
  "$PY_BIN" -u scripts/run_model.py --scope all 2>&1 | tee
"$FINAL/logs/repaired_model.log"
fi
"$PY_BIN" scripts/analyze_results.py 2>&1 | tee "$FINAL/logs/analysis.log"
"$PY_BIN" -u scripts/strengthen_evidence.py 2>&1 | tee
"$FINAL/logs/strengthen_evidence.log"
"$PY_BIN" src/audit/audit_final.py 2>&1 | tee "$FINAL/logs/independent_model_audit.log"
"$NODE_BIN" scripts/build_workbooks.mjs 2>&1 | tee "$FINAL/logs/workbook_build.log"

```

```

"$PY_BIN" src/audit/audit_final.py --workbooks 2>&1 | tee
"$FINAL/logs/independent_workbook_audit.log"
"$PY_BIN" scripts/make_charts.py 2>&1 | tee "$FINAL/logs/charts.log"
"$PY_BIN" scripts/build_documents.py 2>&1 | tee "$FINAL/logs/documents.log"
if [[ -d /Library/Fonts ]]; then
    cat > work/fontconfig/fonts.conf <<'XML'
    <?xml version="1.0"?><!DOCTYPE fontconfig SYSTEM
    "fonts.dtd"><fontconfig><dir>/Library/Fonts</dir><dir>/System/Library/Fonts</
    dir><cachedir>/tmp/v3-font-cache</cachedir></fontconfig>
XML
    export FONTCONFIG_FILE="$PROJECT_DIR/work/fontconfig/fonts.conf"
fi
DOC_SKILL="${HOME}/.codex/plugins/cache/openai-primary-runtime/documents/
26.905.11957/skills/documents"
"$PY_BIN" "$DOC_SKILL/render_docx.py" "$FINAL/paper/C 题论文.docx" --output_dir
"$FINAL/qa/paper_render" --emit_pdf 2>&1 | tee "$FINAL/logs/paper_render.log"
"$PY_BIN" "$DOC_SKILL/render_docx.py" "$FINAL/paper/AI 工具使用详情.docx" --
output_dir "$FINAL/qa/ai_render" --emit_pdf 2>&1 | tee "$FINAL/logs/ai_render.log"
cp "$FINAL/qa/paper_render/C 题论文.pdf" "$FINAL/paper/C 题论文.pdf"
cp "$FINAL/qa/ai_render/AI 工具使用详情.pdf" "$FINAL/paper/AI 工具使用详情.pdf"
"$PY_BIN" scripts/final_quality.py 2>&1 | tee "$FINAL/logs/final_quality.log"
"$PY_BIN" scripts/write_improvement_report.py
"$PY_BIN" scripts/package_final.py 2>&1 | tee work/package.log
echo "Current-run artifacts are in $FINAL; synchronize only after visual review."

```

scripts/analyze_results.py

```

SHA256 40e3144113b35b4c7ad68969b78af1beac6d619ec616af748b7e5b15feaeefc1
from pathlib import Path
import sys,json,copy
from datetime import date
import pandas as pd,numpy as np
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
from src.model.config import load_config
from src.model.data import load_data
from src.model.risk import cases,solve
from src.model.experiments import write_csv

def main():

cfg=load_config('config/final.yaml');F=Path(cfg['_output_root'])/'frozen';E=F.parent/'evidenc
e';cfg.update(json.loads((F/'resolved_config.json').read_text()));data=load_data(cfg);out=[];rn
g=np.random.default_rng(20260912)
    for mode in ['q3','q4_3']:
        base=pd.read_csv(F/f'{mode}_S0_daily.csv')
        for sc in ['S1','S2','S3']:
            d=pd.read_csv(F/f'{mode}_{sc}
            _daily.csv');delta=d.final_relative_cost_yuan.to_numpy()-
            base.final_relative_cost_yuan.to_numpy();samples=[]

```

```

    for _ in range(2000):

starts=rng.integers(0,len(delta)-6,size=(len(delta)+6)//7);indices=np.concatenate([np.arange(x,
x+7) for x in starts][:len(delta)];samples.append(float(delta[indices].sum()))

out.append(dict(mode=mode,comparison=sc+'_minus_S0',difference_yuan=float(delta.sum()),
,paired_days=len(delta),block_days=7,bootstrap_replicates=2000,ci_low=float(np.quantile(sa
mples,.025)),ci_high=float(np.quantile(samples,.975))))
    write_csv(F/'comparison_stats.csv',out)
    sensitivity=[];solver=[]
    for mode in ['q2','q3','q4_2','q4_3']:
        d=pd.read_csv(F/f'{mode}
_dispatch.csv');p=d.settlement_price.to_numpy();down=np.maximum(d.plan_00_grid_kwh-
d.final_adjusted_grid_kwh,0)

sensitivity.append(dict(mode=mode,main_cost=float(d.final_relative_cost_yuan.sum()),same
_chain_sequential_cost=float(d.sequential_cost_yuan.sum()),no_refund_same_path_cost=floa
t(d.final_relative_cost_yuan.sum()+np.dot(p,down)),note='same actions; no reoptimization;
delivery-time prices'))
        if mode in ['q2','q4_2']:continue
        for daystr in cfg['experiments']['representative_dates']:
            day=date.fromisoformat(daystr);dd=d[d.date==daystr]
            for issue in [0,6,12,18]:
                pos=issue*6;length=min(144,len(data.dates)*144-
(data.date_index[day]*144+pos));baseline=np.full(length,np.nan);baseline[:144-
pos]=dd.plan_00_grid_kwh.to_numpy()[pos:];lock=np.full(length,np.nan)
                if issue==0:lock[:144]=baseline[:144]
                cs=cases(data,cfg,day,issue,length,mode);initial=float(dd.soc_start_kwh.iloc[pos]);
sols=[]
                for method in ['highs','highs-ds','highs-ipm']:

r=solve(cs,initial,baseline,cfg,lock=lock,method=method);sols.append(r);solver.append(dict(
mode=mode,date=daystr,issue=issue,method=method,objective=r.objective,objective_differe
nce_vs_highs=r.objective-sols[0].objective,grid_l1_difference=float(abs(r.grid-
sols[0].grid).sum()),soc_max_difference=float(abs(r.soc_end-
sols[0].soc_end).max()),balance_residual=r.balance_residual,soc_residual=r.soc_residual))

write_csv(F/'settlement_sensitivity.csv',sensitivity);write_csv(F/'solver_instance_sensitivity.c
sv',solver)
    # Explicit diagnosis remains a replay, never a trained test-period parameter.
    reserve=[]
    for mode in ['q2','q4_2']:
        d=pd.read_csv(F/f'{mode}_dispatch.csv')
        for delta in [0,10,20,50]:
            q=np.maximum(d.actual_load_kwh+d.charge_kwh-d.actual_pv_kwh-
d.discharge_kwh-d.plan_00_grid_kwh-
delta,0);cost=float(np.sum(d.settlement_price*(d.plan_00_grid_kwh+delta)
+5*d.settlement_price*q));reserve.append(dict(mode=mode,delta_kwh=delta,total_cost=cost,
difference_vs_current=cost-
d.final_relative_cost_yuan.sum(),role='ex_post_diagnostic_not_parameter_selection'))

```

```

write_csv(F/'reserve_diagnostic.csv',reserve)
# Evidence source hashes include actual input bytes and active source code.
sources=[]
for p in (Path(cfg['_source_root'])/'C 题').rglob('*'):
    if
p.is_file():sources.append(dict(kind='original_input',path=str(p.relative_to(Path(cfg['_source_
root']))))),sha256=__import__('hashlib').sha256(p.read_bytes()).hexdigest())
    for folder in ['src','scripts','tests','paper','config']:
        for p in (ROOT/folder).rglob('*'):
            if p.is_file() and '__pycache__' not in
p.parts:sources.append(dict(kind='active_source',path=str(p.relative_to(ROOT)),sha256=__im
port__('hashlib').sha256(p.read_bytes()).hexdigest()))

(E/'source_provenance.json').write_text(json.dumps(sources,ensure_ascii=False,indent=2));pr
int('Analysis and exact-instance solver diagnostics complete',flush=True)

```

```

if __name__=='__main__':main()

```

scripts/build_documents.py

```

SHA256 cedb94b106d5a9eacd55d7753cb41a32b2222d08e9ff2284dc2efff480e70be9
#!/usr/bin/env python3
"""Inject frozen results into the paper and create the final DOCX sources."""
from __future__ import annotations

import csv
import json
import re
import shutil
from pathlib import Path

import numpy as np
import pandas as pd
from docx import Document
from docx.enum.section import WD_SECTION
from docx.enum.table import WD_ALIGN_VERTICAL, WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK,
WD_LINE_SPACING
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
FROZEN = FINAL / "frozen"
PAPER = FINAL / "paper"
CHARTS = FINAL / "charts"
WORK = ROOT / "work" / "paper_build"

```

```

# Arial Unicode MS is available as a TrueType font to the bundled headless
# converter and contains the complete Simplified Chinese glyph set. macOS TTC
# family names such as Songti SC/Heiti SC are visible to desktop applications
# but are not resolved reliably by this conversion runtime.
BODY_FONT = "Arial Unicode MS"
HEADING_FONT = "Arial Unicode MS"

def clean_inline(value: str) -> str:
    return value.replace('*', '').replace(' ', '')

def set_cell_shading(cell, fill: str) -> None:
    tc_pr = cell._tc.get_or_add_tcPr(); shd = tc_pr.find(qn("w:shd"))
    if shd is None: shd = OxmlElement("w:shd"); tc_pr.append(shd)
    shd.set(qn("w:fill"), fill)

def set_cell_borders(cell, color: str = "D9D9D9") -> None:
    tc_pr = cell._tc.get_or_add_tcPr(); borders = tc_pr.first_child_found_in("w:tcBorders")
    if borders is None: borders = OxmlElement("w:tcBorders"); tc_pr.append(borders)
    for edge in ("top", "left", "bottom", "right", "insideH", "insideV"):
        node = borders.find(qn(f"w:{edge}"))
        if node is None: node = OxmlElement(f"w:{edge}"); borders.append(node)
        node.set(qn("w:val"), "single"); node.set(qn("w:sz"), "4"); node.set(qn("w:color"), color)

def repeat_header(row) -> None:
    tr_pr = row._tr.get_or_add_trPr(); tbl_header = OxmlElement("w:tblHeader");
    tbl_header.set(qn("w:val"), "true"); tr_pr.append(tbl_header)

def add_page_number(paragraph) -> None:
    paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
    run = paragraph.add_run(); begin = OxmlElement("w:fldChar");
    begin.set(qn("w:fldCharType"), "begin")
    instr = OxmlElement("w:instrText"); instr.set(qn("xml:space"), "preserve"); instr.text = "
PAGE "
    separate = OxmlElement("w:fldChar"); separate.set(qn("w:fldCharType"), "separate")
    end = OxmlElement("w:fldChar"); end.set(qn("w:fldCharType"), "end")
    run._r.extend([begin, instr, separate, end])

def configure(doc: Document) -> None:
    doc.core_properties.author = "
    doc.core_properties.last_modified_by = "
    section = doc.sections[0]; section.page_width = Cm(21); section.page_height = Cm(29.7)

```

```

    section.top_margin = Cm(2.5); section.bottom_margin = Cm(2.5); section.left_margin =
Cm(2.5); section.right_margin = Cm(2.5)
    styles = doc.styles
    normal = styles["Normal"]; normal.font.name = BODY_FONT; normal.font.size = Pt(10.5)
    normal._element.rPr.rFonts.set(qn("w: eastAsia"), BODY_FONT)
    normal.paragraph_format.line_spacing = 1.25; normal.paragraph_format.space_after =
Pt(3)
    for style_name, size, font in (("Title", 18, HEADING_FONT), ("Heading 1", 14,
HEADING_FONT), ("Heading 2", 12, HEADING_FONT), ("Heading 3", 11,
HEADING_FONT)):
        st = styles[style_name]; st.font.name = font; st.font.size = Pt(size); st.font.color.rgb =
RGBColor(0, 0, 0); st.font.bold = True
        st._element.rPr.rFonts.set(qn("w: eastAsia"), font); st.paragraph_format.keep_with_next
= True
        styles["Title"].paragraph_format.space_after = Pt(12)
    for st in styles:
        if st.type == 1:
            pr=st._element.find(qn('w:pPr'))
            if pr is not None:
                for border in list(pr.findall(qn('w:pBdr'))):pr.remove(border)
            table_style=styles.add_style('ResultTableText',1)
            table_style.font.name=BODY_FONT;table_style.font.size=Pt(8.5)
            table_style._element.get_or_add_rPr().rFonts.set(qn('w: eastAsia'),BODY_FONT)
            table_style.paragraph_format.line_spacing=1.0

table_style.paragraph_format.space_before=Pt(0);table_style.paragraph_format.space_after=P
t(0)
    add_page_number(section.footer.paragraphs[0])

def add_table(doc: Document, lines: list[str]) -> None:
    data = [[clean_inline(x.strip()) for x in line.strip().split("|")] for line in lines if not
re.match(r"^\s*|\?\s*?:?-\+", line)]
    if not data: return
    cols = max(map(len, data)); table = doc.add_table(rows=len(data), cols=cols);
table.alignment = WD_TABLE_ALIGNMENT.CENTER
    table.autofit = False
    for i, row in enumerate(data):
        for j in range(cols):
            cell = table.cell(i, j); cell.text = row[j] if j < len(row) else ""; cell.vertical_alignment =
WD_ALIGN_VERTICAL.CENTER; set_cell_borders(cell)
            for p in cell.paragraphs:
                p.style=doc.styles['ResultTableText']
                p.paragraph_format.space_after = Pt(0); p.alignment =
WD_ALIGN_PARAGRAPH.CENTER if j == 0 or i == 0 else
WD_ALIGN_PARAGRAPH.LEFT
                for run in p.runs:
                    run.font.name = BODY_FONT; run.font.size = Pt(8.5);
run._element.rPr.rFonts.set(qn("w: eastAsia"), BODY_FONT)
                    if i == 0: run.font.bold = True; run.font.color.rgb = RGBColor(255,255,255)

```

```

        if i == 0: set_cell_shading(cell, "1F4E78")
        elif i % 2 == 0: set_cell_shading(cell, "F4F7FA")
    for row in table.rows:
        prevent_split = OxmlElement('w:cantSplit')
        row._tr.get_or_add_trPr().append(prevent_split)
        for j, cell in enumerate(row.cells):
            cell.width = Cm(16 / cols)
    if len(data) <= 12:
        for row in table.rows[:-1]:
            for cell in row.cells:
                for p in cell.paragraphs: p.paragraph_format.keep_with_next = True
    repeat_header(table.rows[0])
    spacer=doc.add_paragraph();spacer.paragraph_format.line_spacing=Pt(3)
    spacer.paragraph_format.space_before=Pt(0);spacer.paragraph_format.space_after=Pt(4)

def markdown_to_docx(text: str, output: Path) -> None:
    doc = Document(); configure(doc)
    lines = text.splitlines(); i = 0; in_code = False; figure_no=0; table_no=0; current_heading=""
    while i < len(lines):
        line = lines[i]
        if line.startswith("` ` ") or line.startswith("` `"):
            in_code = not in_code; i += 1; continue
        if in_code:
            p = doc.add_paragraph()
            p.paragraph_format.space_after = Pt(0)
            p.paragraph_format.line_spacing = 1.05
            run = p.add_run(line if line else " ")
            run.font.name = "Consolas"; run.font.size = Pt(7.5)
            run._element.rPr.rFonts.set(qn("w: eastAsia"), BODY_FONT)
            i += 1; continue
        if line.startswith("|") and i + 1 < len(lines) and lines[i + 1].startswith("|"):
            table_no+=1
            caption=re.sub(r'^\d+(?:\.\d+)*\s+', "", current_heading)
            columns=[x.strip() for x in line.strip('|').split('|')]
            names={'场景候选': '一月验证期候选策略费用', '方案': '确定性调度与无储能对照', '物理区间': '指定物理区间购电量' if '0 点计划量' in columns else '指定物理区间储能动作', '0 点 SOC': '日初与日末储能状态', '全天计划量': '全天购电及费用汇总', '紧急购电物理区间': '连续紧急购电区间'}
            caption=names.get(columns[0],caption)
            if columns[0]=='模式':
                caption={'方案': '日前采购与可行基线', '最终净额': '同一承诺链的不同结算解释', '一月启动费用': '一月启动与二月状态衔接', '计划费': '正式期费用构成', '时点策略': '更新频次的费用与紧急购电量', '比较': '配对费用差及区间估计', '紧急时段占比': '紧急购电与日费用尾部风险', '变量': '执行时刻预测误差', '求解方法': '全年替代求解器回放费用', '块长/天': '分块重采样对块长度的敏感性'}.get(columns[1],caption)
            cap=doc.add_paragraph(f'表 {table_no} {caption}')

```

```

cap.alignment=WD_ALIGN_PARAGRAPH.CENTER;cap.paragraph_format.keep_with_next=True
    for run in cap.runs:run.font.size=Pt(9)
    block = []
    while i < len(lines) and lines[i].startswith("|"):
        block.append(lines[i]); i += 1
    add_table(doc, block); continue
fig = re.fullmatch(r"\[FIGURE:([^\|]+)\(.(+)\)\]", line.strip())
if fig:
    figure_no+=1
    p = doc.add_paragraph(); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.keep_with_next = False
    p.add_run().add_picture(str(CHARTS / fig.group(1)), width=Cm(15.2))
    p.add_run().add_break()
    cap=p;cap.add_run(f"图 {figure_no} {fig.group(2)}"); cap.alignment =
WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.keep_together = True
    for run in cap.runs: run.font.size = Pt(9); run.font.name = BODY_FONT;
run._element.rPr.rFonts.set(qn("w: eastAsia"), BODY_FONT)
    i += 1; continue
if line.startswith("# "):
    p = doc.add_paragraph(clean_inline(line[2:]), style="Title"); p.alignment =
WD_ALIGN_PARAGRAPH.CENTER
elif line.startswith("## "):
    heading = clean_inline(line[3:])
    current_heading=heading
    p=doc.add_paragraph(heading, style="Heading 1")
    if heading.startswith('1 ') or heading=='附录':
        p.paragraph_format.page_break_before=True
elif line.startswith("### "):
    current_heading=clean_inline(line[4:])
    p=doc.add_paragraph(clean_inline(line[4:]), style="Heading 2")
    if re.match(r'#### 附录[B-D]',line):p.paragraph_format.page_break_before=True
elif line.startswith("#### "):
    heading=clean_inline(line[5:])
    current_heading=heading
    p=doc.add_paragraph(heading, style="Heading 3")
elif re.match(r"^\d+\.\s+", line):
    doc.add_paragraph(clean_inline(re.sub(r"^\d+\.\s+", "", line)), style="List Number")
elif line.startswith("- "):
    doc.add_paragraph(clean_inline(line[2:]), style="List Bullet")
elif line.startswith("$") and line.endswith("$"):
    from native_equations import equation
    p = doc.add_paragraph(); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    number=int(re.search(r'((\d+))',line).group(1));p._p.append(equation(number))
elif not line.strip():
    pass
else:

```

```

p = doc.add_paragraph(); p.paragraph_format.first_line_indent = Pt(21)
p.paragraph_format.keep_together = False
run = p.add_run(clean_inline(line))
if re.match(r'^\[d+\] ',line):
    p.paragraph_format.first_line_indent=Pt(0)
    p.paragraph_format.line_spacing=1.05
    p.paragraph_format.space_after=Pt(3)
    p.paragraph_format.keep_together=True
    run.font.size=Pt(9)
i += 1
output.parent.mkdir(parents=True, exist_ok=True); doc.save(output)

```

```

def main():
    from paper_content import generate
    text,ai=generate()
    from research_content import enrich
    text,ai=enrich(text,ai)
    PAPER.mkdir(parents=True,exist_ok=True);WORK.mkdir(parents=True,exist_ok=True)
    (FINAL/"evidence/paper_source_frozen.md").write_text(text,encoding='utf-8')
    (FINAL/"evidence/AI 工具使用详情_冻结稿.md").write_text(ai,encoding='utf-8')
    markdown_to_docx(text,PAPER/"C 题论文.docx")
    from research_content import append_sources
    append_sources(PAPER/"C 题论文.docx")
    markdown_to_docx(ai,PAPER/"AI 工具使用详情.docx")
    print('Created current-run DOCX sources',flush=True)

if __name__=='__main__':main()

```

scripts/build_workbooks.mjs

```

SHA256 c5927746622d280f2e7c89dd7669a9965879636a55f8f3340d23ae1151803d88
import fs from "node:fs/promises";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { FileBlob, SpreadsheetFile, Workbook } from "@oai/artifact-tool";

```

```

const projectRoot = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const finalRoot = path.join(projectRoot, "outputs", "c_final_v1");
const frozenRoot = path.join(finalRoot, "frozen");
const workbookRoot = path.join(finalRoot, "workbooks");
const qaRoot = path.join(finalRoot, "qa", "workbook_renders");

```

```

function parseCsv(text) {
    const rows = [];
    let row = [], value = "", quoted = false;
    for (let i = 0; i < text.length; i++) {
        const ch = text[i];
    }
}

```

```

    if (quoted) {
      if (ch === '"' && text[i + 1] === '"') { value += '"'; i++; }
      else if (ch === '"') quoted = false;
      else value += ch;
    } else if (ch === '"') quoted = true;
    else if (ch === ',') { row.push(value); value = ''; }
    else if (ch === '\n') { row.push(value.replace(/r$/, '')); rows.push(row); row = []; value =
    ''; }
    else value += ch;
  }
  if (value.length || row.length) { row.push(value.replace(/r$/, '')); rows.push(row); }
  if (rows.length && rows[0][0]?.charCodeAt(0) === 0xfeff) rows[0][0] = rows[0][0].slice(1);
  const headers = rows.shift();
  return rows.filter(r => r.some(v => v !== '')).map(r => Object.fromEntries(headers.map((h,
  i) => [h, r[i] ?? ''])));
}

```

```

async function readCsv(name) {
  return parseCsv(await fs.readFile(path.join(frozenRoot, name), "utf8"));
}

```

```

function n(v) {
  const x = Number(v);
  if (!Number.isFinite(x)) throw new Error(`Expected numeric value, got $
  {JSON.stringify(v)}`);
  return x;
}

```

```

function roundHalfUp(v, places = 4) {
  // Excel-facing values must follow the audit contract's decimal ROUND_HALF_UP
  // rule. Number.toFixed() rounds the already-binary IEEE-754 value and gives
  // the wrong result at exact decimal half units such as 2238.56265.
  const source = typeof v === "string" ? v.trim() : String(n(v));
  const match = source.match(/^([+-]?)(\d+)(?:\.(\d*))?(?:[eE]([+-]?\d+))?$ /);
  if (!match) throw new Error(`Expected decimal value, got ${JSON.stringify(v)}`);
  const negative = match[1] === "-";
  const fraction = match[3] ?? "";
  const exponent = Number(match[4] ?? 0);
  let magnitude = BigInt(`${match[2]}${fraction}` || "0");
  const scale = fraction.length - exponent;
  if (scale <= places) {
    magnitude *= 10n ** BigInt(places - scale);
  } else {
    const divisor = 10n ** BigInt(scale - places);
    const remainder = magnitude % divisor;
    magnitude /= divisor;
    if (remainder * 2n >= divisor) magnitude += 1n;
  }
  const signed = negative ? -magnitude : magnitude;
  return Number(signed) / 10 ** places;
}

```

```
}
```

```
function rounded4(v) { return roundHalfUp(v, 4); }
```

```
function decimalParts(v) {  
  const source = typeof v === "string" ? v.trim() : String(n(v));  
  const match = source.match(/^[+-]?(\d+)(?:\.(d*))?(?:[eE]([+-]?\d+))?$$/);  
  if (!match) throw new Error(`Expected decimal value, got ${JSON.stringify(v)}`);  
  const fraction = match[3] ?? "";  
  const exponent = Number(match[4] ?? 0);  
  let magnitude = BigInt(`${match[2]}${fraction}` || "0");  
  let scale = fraction.length - exponent;  
  if (scale < 0) {  
    magnitude *= 10n ** BigInt(-scale);  
    scale = 0;  
  }  
  if (match[1] === "-") magnitude = -magnitude;  
  return { magnitude, scale };  
}
```

```
function decimalSum(values) {  
  const parts = values.map(decimalParts);  
  const scale = Math.max(0, ...parts.map(p => p.scale));  
  const total = parts.reduce(  
    (acc, p) => acc + p.magnitude * 10n ** BigInt(scale - p.scale),  
    0n,  
  );  
  const negative = total < 0n;  
  const digits = (negative ? -total : total).toString().padStart(scale + 1, "0");  
  if (scale === 0) return `${negative ? "-" : ""}${digits}`;  
  return `${negative ? "-" : ""}${digits.slice(0, -scale)}${digits.slice(-scale)}`;  
}  
function excelDate(s) { return new Date(`${s}T00:00:00Z`); }
```

```
function officialLabel(position) {  
  const start = position * 10;  
  const end = (position + 1) * 10;  
  const fmt = (mins) => {  
    if (mins >= 1440) return `${Math.floor((mins - 1440) / 60)}:${String((mins - 1440) % 60).padStart(2, "0")}${1}`;  
    return `${Math.floor(mins / 60)}:${String(mins % 60).padStart(2, "0")}`;  
  };  
  return `${fmt(start)}-${fmt(end)}`;  
}
```

```
function physicalLabel(startPosition, endPositionExclusive) {  
  const fmt = (mins) => `${Math.floor(mins / 60)}:${String(mins % 60).padStart(2, "0")}`;  
  return `${fmt((startPosition - 1) * 10)}-${fmt(endPositionExclusive * 10)}`;  
}
```

```

function groupByDate(rows) {
  const out = new Map();
  for (const row of rows) {
    if (!out.has(row.date)) out.set(row.date, []);
    out.get(row.date).push(row);
  }
  for (const rs of out.values()) rs.sort((a, b) => n(a.position) - n(b.position));
  return out;
}

function validateDispatch(rows, name, expectedDays) {
  if (rows.length !== expectedDays * 144) throw new Error(` ${name}: expected $
{expectedDays * 144} rows, got ${rows.length}`);
  const byDate = groupByDate(rows);
  if (byDate.size !== expectedDays) throw new Error(` ${name}: expected $ {expectedDays}
dates, got $ {byDate.size}`);
  const referenceLabels = [...byDate.values()][0].map(r => r.official_label);
  if (referenceLabels.some(v => !v) || new Set(referenceLabels).size !== 144) {
    throw new Error(` ${name}: official labels must contain 144 nonblank unique values`);
  }
  for (const [date, rs] of byDate) {
    if (rs.length !== 144) throw new Error(` ${name} ${date}: expected 144 rows, got $
{rs.length}`);
    for (let i = 0; i < 144; i++) {
      if (n(rs[i].position) !== i + 1) throw new Error(` ${name} ${date}: bad position at $ {i + 1}
`);
      if (rs[i].official_label !== referenceLabels[i]) throw new Error(` ${name} ${date}: official
label mismatch at $ {i + 1}`);
    }
  }
  return byDate;
}

function styleSheet(sheet, usedRange, numericRanges = []) {
  // Keep the template's cell boundaries visible. Several adjacent fields
  // (date/time and charge/discharge/time) otherwise read as one value in the
  // rendered workbook even though the underlying cells are correct.
  sheet.showGridLines = true;
  usedRange.format.font = { name: "Arial", size: 10, color: "#1111827" };
  usedRange.format.verticalAlignment = "center";
  const header = usedRange.getRow(0);
  header.format.fill = "#1F4E78";
  header.format.font = { name: "Arial", size: 10, bold: true, color: "#FFFFFF" };
  header.format.horizontalAlignment = "center";
  header.format.wrapText = true;
  header.format.rowHeight = 30;
  for (const range of numericRanges) range.format.numberFormat = "0.0000";
}

```

```

function addPlanSheet(wb, name, byDate, valueField, costField) {
  const sheet = wb.worksheets.add(name);
  const labels = [...byDate.values()][0].map(r => r.official_label);
  const matrix = [["日期\\时间", ...labels, "全天购电量", "全天购电费"]];
  for (const [date, rows] of byDate) {
    matrix.push([
      excelDate(date),
      ...rows.map(r => rounded4(r[valueField])),
      rounded4(decimalSum(rows.map(r => r[valueField]))),
      rounded4(rows.reduce((s, r) => s + n(r[costField]), 0)),
    ]);
  }
  const used = sheet.getRangeByIndexes(0, 0, matrix.length, matrix[0].length);
  used.values = matrix;
  styleSheet(sheet, used, [sheet.getRangeByIndexes(1, 1, matrix.length - 1, matrix[0].length - 1)]);
  sheet.getRangeByIndexes(1, 0, matrix.length - 1, 1).format.numberFormat = "yyyy-mm-dd";
  sheet.freezePanes.freezeRows(1);
  sheet.freezePanes.freezeColumns(1);
  sheet.getRange(`A1:A${matrix.length}`).format.columnWidth = 12;
  sheet.getRange(`B1:EO${matrix.length}`).format.columnWidth = 10;
  sheet.getRange(`EP1:EQ${matrix.length}`).format.columnWidth = 15;
  return sheet;
}

```

```

function addQ1Sheets(wb, rows) {
  const plan = wb.worksheets.add("计划购电量");
  // result1's official template uses a different final label spelling from
  // the annual templates; preserve that literal template label.
  const planMatrix = [["时间段", "购电量"], ...rows.map((r, i) => [
    officialLabel(i + 1),
    rounded4(r.plan_00_grid_kwh),
  ])];
  const pUsed = plan.getRangeByIndexes(0, 0, planMatrix.length, 2);
  pUsed.values = planMatrix;
  styleSheet(plan, pUsed, [plan.getRange(`B2:B${planMatrix.length}`)]);
  plan.freezePanes.freezeRows(1);
  plan.getRange(`A1:A${planMatrix.length}`).format.columnWidth = 19;
  plan.getRange(`B1:B${planMatrix.length}`).format.columnWidth = 14;

```

```

const charge = wb.worksheets.add("充放电电量");
const cm = [["时间段", "充电量", "放电电量", "时刻", "储电量"]];
for (let block = 0; block < 6; block++) {
  const rs = rows.slice(block * 24, (block + 1) * 24);
  cm.push([
    `${block * 4}:00-${(block + 1) * 4}:00`,
    rounded4(decimalSum(rs.map(r => r.charge_kwh))),
  ]);
}

```

```

        rounded4(decimalSum(rs.map(r => r.discharge_kwh))),
        block === 0 ? "0:00" : block === 1 ? "24:00" : null,
        block === 0 ? rounded4(rows[0].soc_start_kwh) : block === 1 ?
rounded4(rows[143].soc_end_kwh) : null,
    ]);
}
const cUsed = charge.getRangeByIndexes(0, 0, cm.length, 5);
cUsed.values = cm;
styleSheet(charge, cUsed, [charge.getRange("B2:C7"), charge.getRange("E2:E3")]);
charge.getRange(`A1:E${cm.length}`).format.columnWidth = 18;
}

function addChargeSheet(wb, byDate) {
    const sheet = wb.worksheets.add("充放电量");
    const matrix = [["日期", "时间段", "充电量", "放电", "时刻", "储电量"]];
    for (const [date, rows] of byDate) {
        for (let block = 0; block < 6; block++) {
            const rs = rows.slice(block * 24, (block + 1) * 24);
            matrix.push([
                block === 0 ? excelDate(date) : null,
                `${block * 4}:00-${(block + 1) * 4}:00`,
                rounded4(decimalSum(rs.map(r => r.charge_kwh))),
                rounded4(decimalSum(rs.map(r => r.discharge_kwh))),
                block === 0 ? "0:00" : block === 1 ? "24:00" : null,
                block === 0 ? rounded4(rows[0].soc_start_kwh) : block === 1 ?
rounded4(rows[143].soc_end_kwh) : null,
            ]);
        }
    }
    const used = sheet.getRangeByIndexes(0, 0, matrix.length, 6);
    used.values = matrix;
    styleSheet(sheet, used, [sheet.getRangeByIndexes(1, 2, matrix.length - 1, 2),
sheet.getRangeByIndexes(1, 5, matrix.length - 1, 1)]);
    sheet.getRangeByIndexes(1, 0, matrix.length - 1, 1).format.numberFormat = "yyyy-mm-dd";
    sheet.freezePanes.freezeRows(1);
    sheet.getRange(`A1:A${matrix.length}`).format.columnWidth = 13;
    sheet.getRange(`B1:B${matrix.length}`).format.columnWidth = 16;
    sheet.getRange(`C1:F${matrix.length}`).format.columnWidth = 14;
}

function addEmergencySheet(wb, byDate, threshold) {
    const sheet = wb.worksheets.add("紧急购电量");
    const matrix = [["日期", "购电时间段", "购电量"]];
    for (const [date, rows] of byDate) {
        const groups = [];
        let current = null;
        for (const r of rows) {
            const pos = n(r.position), q = n(r.emergency_kwh);
            // The configured half-unit threshold implements ROUND_HALF_UP at four decimals.

```

```

    const shown = Math.abs(q) >= threshold;
    if (!shown) { if (current) { groups.push(current); current = null; } continue; }
    if (!current) current = { start: pos, end: pos, amounts: [r.emergency_kwh] };
    else if (pos === current.end + 1) { current.end = pos;
current.amounts.push(r.emergency_kwh); }
    else { groups.push(current); current = { start: pos, end: pos, amounts:
[r.emergency_kwh] }; }
    }
    if (current) groups.push(current);
    groups.forEach((g, i) => matrix.push([i === 0 ? excelDate(date) : null,
physicalLabel(g.start, g.end), rounded4(decimalSum(g.amounts))]);
    }
    const used = sheet.getRangeByIndexes(0, 0, matrix.length, 3);
    used.values = matrix;
    styleSheet(sheet, used, matrix.length > 1 ? [sheet.getRangeByIndexes(1, 2, matrix.length - 1,
1)] : []);
    if (matrix.length > 1) sheet.getRangeByIndexes(1, 0, matrix.length - 1,
1).format.numberFormat = "yyyy-mm-dd";
    sheet.freezePanes.freezeRows(1);
    sheet.getRange(`A1:A${matrix.length}`).format.columnWidth = 13;
    sheet.getRange(`B1:B${matrix.length}`).format.columnWidth = 18;
    sheet.getRange(`C1:C${matrix.length}`).format.columnWidth = 14;
    }

function addEvidenceSheets(wb, byDate, mode) {
    const wanted = new Set(["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]);
    const matrix = [["日期", "物理时间段", "0 点计划量", "最终购电量"]];
    for (const [day, rows] of byDate) {
        if (mode !== "q1" && !wanted.has(day)) continue;
        for (const hour of [10, 12, 14, 16, 18, 20]) {
            const row = rows.find(r => n(r.position) === hour * 6 + 1);
            matrix.push([excelDate(day), row.physical_interval, rounded4(row.plan_00_grid_kwh),
rounded4(row.final_adjusted_grid_kwh)]);
        }
    }
    const sheet = wb.worksheets.add("指定时段");
    const used = sheet.getRangeByIndexes(0,0,matrix.length,4); used.values=matrix;
    styleSheet(sheet, used, [sheet.getRangeByIndexes(1,2,matrix.length-1,2)]);
    sheet.getRange(`A2:A${matrix.length}`).format.numberFormat="yyyy-mm-dd";
    sheet.getRange(`A1:D${matrix.length}`).format.columnWidth=19;
    const mapping=wb.worksheets.add("时段映射");
    const data=[["位置", "源时间戳", "物理区间", "原模板标签"], ...[...byDate.values()]
[0].map(r=>[n(r.position),r.source_timestamp,r.physical_interval,r.official_label])];
    const
range=mapping.getRangeByIndexes(0,0,data.length,4);range.values=data;styleSheet(mapping
,range);
    mapping.getRange("A1:D145").format.columnWidth=21;
    }

```

```

async function exportAndVerify(wb, filename, sheetNames) {
  wb.recalculate();
  const file = path.join(workbookRoot, filename);
  const blob = await SpreadsheetFile.exportXlsx(wb);
  await blob.save(file);
  const reopened = await SpreadsheetFile.importXlsx(await FileBlob.load(file));
  const errors = await reopened.inspect({
    kind: "match",
    searchTerm: "#REF!|#DIV/0!|#VALUE!|#NAME\\?|#N/A|#NUM!|#NULL!|#SPILL!|#CALC!",
    options: { useRegex: true, maxResults: 100 },
    maxChars: 6000,
    summary: `${filename} formula error scan`,
  });
  const errorLines = (errors.ndjson || "").split("\n").filter(line => /"address"s*:.test(line));
  if (errorLines.length) throw new Error(`${filename}: formula error detected: $
{errorLines.join("\n")}`);
  // Some artifact-tool versions spill inspect output beside large workbooks.
  // It is a transient diagnostic, not part of the official submission.
  await fs.rm(`${file}.inspect.ndjson`, { force: true });
  const renderDir = path.join(qaRoot, filename.replace(/\.xlsx$/, ""));
  await fs.mkdir(renderDir, { recursive: true });
  for (const sheetName of sheetNames) {
    const sheet = reopened.worksheets.getItem(sheetName);
    const used = sheet.getUsedRange(true);
    const rowCount = used?.rowCount ?? 1;
    const colCount = used?.columnCount ?? 1;
    const ranges = [];
    if (colCount > 20) {
      ranges.push(`A1:P${Math.min(rowCount, 12)}`);
      ranges.push(`EI1:EQ${Math.min(rowCount, 12)}`);
      if (rowCount > 20) ranges.push(`A${Math.max(1, rowCount - 5)}:P${rowCount}`);
    } else {
      ranges.push(`A1:${String.fromCharCode(64 + Math.min(colCount, 26))}${Math.min(rowCount, 40)}`);
      if (rowCount > 45) ranges.push(`A${rowCount - 5}:${String.fromCharCode(64 + Math.min(colCount, 26))}${rowCount}`);
    }
    for (let i = 0; i < ranges.length; i++) {
      const png = await reopened.render({ sheetName, range: ranges[i], scale: 1.5, format:
"png" });
      const safe = sheetName.replace(/[\^p{L}\p{N}_-]/gu, "_");
      await fs.writeFile(path.join(renderDir, `${safe}_${i + 1}.png`), new Uint8Array(await
png.arrayBuffer()));
    }
  }
  return file;
}

```

```

async function main() {

```

```

await fs.mkdir(workbookRoot, { recursive: true });
await fs.mkdir(qaRoot, { recursive: true });
const cfg = JSON.parse(await fs.readFile(path.join(projectRoot, "config", "final.yaml"),
"utf8"));
const threshold = Number(cfg.audit.emergency_display_omit_threshold_kwh);

const specs = [
  ["q1_dispatch.csv", "result1.xlsx", "q1"],
  ["q2_dispatch.csv", "result2.xlsx", "q2"],
  ["q3_dispatch.csv", "result3.xlsx", "q3"],
  ["q4_2_dispatch.csv", "result4-2.xlsx", "q4_2"],
  ["q4_3_dispatch.csv", "result4-3.xlsx", "q4_3"],
];
const created = [];
for (const [csvName, xlsxName, mode] of specs) {
  const rows = await readCsv(csvName);
  const expectedDays = mode === "q1" ? 1 : 334;
  const byDate = validateDispatch(rows, mode, expectedDays);
  const wb = Workbook.create();
  let sheetNames;
  if (mode === "q1") {
    addQ1Sheets(wb, rows);
    sheetNames = ["计划购电量", "充放电量"];
  }
  else {
    const initialCostField = (mode === "q2" || mode === "q4_2") ?
"final_relative_cost_yuan" : "plan_cost_yuan";
    addPlanSheet(wb, "计划购电量", byDate, "plan_00_grid_kwh", initialCostField);
    if (mode === "q3" || mode === "q4_3") addPlanSheet(wb, "调整购电量", byDate,
"final_adjusted_grid_kwh", "final_relative_cost_yuan");
    addChargeSheet(wb, byDate);
    addEmergencySheet(wb, byDate, threshold);
    sheetNames = (mode === "q3" || mode === "q4_3")
      ? ["计划购电量", "调整购电量", "充放电量", "紧急购电量"]
      : ["计划购电量", "充放电量", "紧急购电量"];
  }
  addEvidenceSheets(wb, byDate, mode);
  sheetNames.push("指定时段", "时段映射");
  created.push(await exportAndVerify(wb, xlsxName, sheetNames));
}
await fs.writeFile(
  path.join(finalRoot, "qa", "workbook_build.json"),
  JSON.stringify({ created: created.map(file => path.relative(finalRoot, file)), threshold,
verified: true }, null, 2),
);
}

await main();

```

scripts/competition_checks.py

```
SHA256 6842e8294af118e3f923ae4c038c16356057f98d8234297301df1881d296bcba
"""Check verifiable submission properties without inventing human signoff."""
from pathlib import Path
import json,re,hashlib
from research_content import source_files
ROOT=Path(__file__).resolve().parents[1]
FINAL=ROOT/'outputs/c_final_v1'

def inspect(doc,pdf):
    checks=[]
    def add(name,ok,actual):checks.append({'check':name,'status':'PASS' if ok else
'FAIL','actual':actual})
    first="".join(pdf.pages[0].extract_text().split())
    add('abstract_dedicated_first_page','关键词' in first and '题目[1]中的微网' not in first,
{'first_page_has_keywords': '关键词' in first})
    body=next(p for p in doc.paragraphs if p.text=='1 问题与信息边界')

    add('body_explicit_new_page',bool(body.paragraph_format.page_break_before),str(body.para
graph_format.page_break_before))
    appendix=[i for i,p in enumerate(pdf.pages) if re.search(r'(?m)^\s*附录\s*$',p.extract_text())]
    add('main_body_not_over_30_pages',bool(appendix) and appendix[0]-1<=30,
{'appendix_start_pdf_page':appendix[0]+1 if appendix else None,'body_pages':appendix[0]-1
if appendix else None})

    add('all_margins_at_least_2_5cm',all(min(s.top_margin.cm,s.bottom_margin.cm,s.left_margi
n.cm,s.right_margin.cm)>=2.499 for s in doc.sections),len(doc.sections))
    expected={str(p.relative_to(ROOT)):p for p in
source_files()};seen={};active=None;started=False
    for p in doc.paragraphs:
        if p.text=='附录 C 完整源程序与运行配置':started=True
        if not started:continue
        if p.style.name=='Heading 3' and p.text in expected:
            active=p.text;seen[active]=[]
        elif active and not p.text.startswith('SHA256 '):seen[active].append(p.text)
    bad=[]
    for name,p in expected.items():
        want=[x if x else '' for x in p.read_text().splitlines()]
        if seen.get(name)!=want:bad.append(name)
    add('complete_source_appendix_exact_line_match',not bad and len(seen)==len(expected),
{'expected_files':len(expected),'actual_files':len(seen),'mismatch':bad})
    add('blank_author_metadata',not doc.core_properties.author and not
doc.core_properties.last_modified_by,{'author':doc.core_properties.author})
    research=json.loads((FINAL/'research/annual_independent_checks.json').read_text())
    add('four_full_year_independent_replays',len(research)==4 and all(x['status']=='PASS' and
x['rows']==48096 for x in research),research)
    before=[]
    for p in doc.paragraphs:
```

```

        if p.text=='附录':break
        before.append(p.text)
    text='\n'.join(before)
    add('seven_references_and_body_citations',all(f'[{n}]' in text.split('参考文献')[0] for n in
range(1,8)) and all(f'[{n}]' in text.split('参考文献')[-1] for n in range(1,8)),7)
    return checks

```

scripts/final_quality.py

```

SHA256 f014445eb34ddd51f5d113b2d48370fb14b9fcff4ce22dc35aadfe232be5b39
""""Validate generated paper tables against dispatch positions, plus PDF and assets.""""
from pathlib import Path
import json,re,hashlib
from datetime import datetime
import numpy as np,pandas as pd
from docx import Document
from pypdf import PdfReader
from docx.oxml.ns import qn
ROOT=Path(__file__).resolve().parents[1];FINAL=ROOT/'outputs/
c_final_v1';F=FINAL/'frozen'

def main():
    checks=[]
    def check(k,ok,value):checks.append(dict(check=k,status='PASS' if bool(ok) else
'FAIL',actual=value))

    audit=json.loads((FINAL/'audit/independent_validation.json').read_text());check('independent
_model_and_workbooks',audit['status']=='PASS',audit['status'])
    doc=Document(FINAL/'paper/C 题论文.docx');pdf=PdfReader(FINAL/'paper/C 题论
文.pdf');ai=PdfReader(FINAL/'paper/AI 工具使用详情.pdf')
    text='\n'.join(p.text for p in doc.paragraphs)+'\n'+'\n'.join(c.text for t in doc.tables for r in
t.rows for c in r.cells);pdftext='\n'.join(p.extract_text() for p in pdf.pages)
    # Literal template syntax inside the printed source appendix is source code,
    # not an unexpanded value in the paper. Check the rendered paper separately.
    body_paragraphs=[]
    for p in doc.paragraphs:
        if p.text=='附录 C 完整源程序与运行配置':break
        body_paragraphs.append(p.text)
    source_page=next((i for i,p in enumerate(pdf.pages) if '完整源程序与' in p.extract_text()
[:120]),len(pdf.pages))
    display_text='\n'.join(body_paragraphs)+'\n'+'\n'.join(c.text for t in doc.tables for r in t.rows
for c in r.cells)+'\n'+'.join(p.extract_text() for p in pdf.pages[:source_page])
    check('no_unresolved_placeholders',not re.search(r'\{\{^[^}\}]+\}\}
\}',display_text),len(re.findall(r'\{\{^[^}\}]+\}\}',display_text)))
    check('A4_pages',all(abs(float(p.mediabox.width)-595.276)<2 and
abs(float(p.mediabox.height)-841.89)<2 for p in pdf.pages),len(pdf.pages))
    check('PDF_under_20MB',(FINAL/'paper/C 题论文.pdf').stat().st_size<20*1024**2,
(FINAL/'paper/C 题论文.pdf').stat().st_size)

```

```

# Walk actual OOXML order; attach each appendix table to its section date.
active=None;seen={};errors=[];table_i=0;para_i=0
frames={mode:pd.read_csv(F/f{mode}_dispatch.csv') for mode in
['q1','q2','q3','q4_2','q4_3']}
def num(v):return float(v.replace(',').strip())
for el in doc.element.body:
    if el.tag==qn('w:p'):
        t="".join(el.itertext()) # itertext may repeat run text; regex still identifies section.
        m=re.search(r'(Q1|Q2|Q3|Q4_2|Q4_3) (2025-\d\d-\d\d) 指定结果',t)
        if m:active=(m.group(1).lower(),m.group(2));seen.setdefault(active,set())
    elif el.tag==qn('w:tbl'):
        table=doc.tables[table_i];table_i+=1
        if active is None:continue
        data=[[c.text for c in r.cells] for r in
table.rows];head=data[0];mode,day=active;z=frames[mode];z=z[z.date==day]
        if head==['物理区间','0点计划量','最终承诺量']:
            seen[active].add('ten_minute')
            if len(data)!=7:errors.append((active,'ten_minute_count'))
            for row,h in zip(data[1:],[10,12,14,16,18,20]):
                r=z[z.position==h*6+1].iloc[0]
                if row[0]!=r.physical_interval or abs(num(row[1])-r.plan_00_grid_kwh)>.000051
or abs(num(row[2])-r.final_adjusted_grid_kwh)>.000051:errors.append((active,row))
            elif head==['物理区间','充电量','放电量']:
                seen[active].add('four_hour')
                for b,row in enumerate(data[1:]):
                    zz=z[z.position>b*24&(z.position<=(b+1)*24)]
                    if row[0]!=f'{b*4}:00-{(b+1)*4}:00' or abs(num(row[1])-
zz.charge_kwh.sum())>.000051 or abs(num(row[2])-
zz.discharge_kwh.sum())>.000051:errors.append((active,row))
            elif head==['0点 SOC','24点 SOC']:
                seen[active].add('soc')
                if abs(num(data[1][0])-z.soc_start_kwh.iloc[0])>.000051 or abs(num(data[1][1])-
z.soc_end_kwh.iloc[-1])>.000051:errors.append((active,'SOC'))
            elif head==['全天计划量','全天最终量','全天总购电费']:
                seen[active].add('daily')
                if abs(num(data[1][0])-z.plan_00_grid_kwh.sum())>.000051 or abs(num(data[1]
[1])-z.final_adjusted_grid_kwh.sum())>.000051 or abs(num(data[1][2])-
z.final_relative_cost_yuan.sum())>.0051:errors.append((active,'daily'))
            elif head==['紧急购电物理区间','紧急购电量']:
                seen[active].add('emergency');expected=[];start=None;total=0.;last=None
                for r in z.sort_values('position').itertuples():
                    if r.emergency_kwh>=.00005:
                        if start is None:start=(int(r.position)-1)*10
                        last=int(r.position)*10;total+=r.emergency_kwh
                    elif start is not None:
                        expected.append((f'{start//60}:{start%60:02d}-{last//60}:
{last%60:02d}',total));start=None;total=0.
                        if start is not None:expected.append((f'{start//60}:{start%60:02d}-{last//60}:
{last%60:02d}',total))

```

```

        if not expected:expected=[('无正缺口',0.))
        if len(data)-1!=len(expected):errors.append((active,'emergency_count'))
        for row,(span,value) in zip(data[1:],expected):
            if row[0]!=span or abs(num(row[1])-
value)>.000051:errors.append((active,'emergency',row))
            required={(mode,day) for mode in ['q2','q3','q4_2','q4_3'] for day in ['2025-03-20','2025-
06-21','2025-09-23','2025-12-21']};required.add(('q1','2025-01-01'))
            missing=[k for k in required if k not in seen or not ({'ten_minute','four_hour','soc','daily'}|
({'emergency'}) if k[0]!='q1' else set()))<=seen[k]]
            check('all_17_specified_date_groups',not missing,
{'groups':len(seen),'missing':missing});check('paper_numbers_independently_reconciled',not
errors,errors)
            missing_pdf_values=[]
            compact_pdf=re.sub(r'\s+',",",pdftext)
            for table in doc.tables:
                for row in table.rows:
                    for cell in row.cells:
                        value=cell.text.strip()
                        if re.fullmatch(r'-?[d,]+\.\d+',value) and value not in
compact_pdf:missing_pdf_values.append(value)
                        check('all_table_numeric_strings_survive_PDF',not
missing_pdf_values,missing_pdf_values)
                        # Confirm key corrected Q1 numbers survived the PDF conversion.
                        from decimal import Decimal,ROUND_HALF_UP
                        q1value=float(frames['q1'].loc[frames['q1'].position==73,'plan_00_grid_kwh'].iloc[0])
                        for value in
[format(Decimal(str(q1value)).quantize(Decimal('.0001'),rounding=ROUND_HALF_UP),'.4f
)]:
                            check('PDF_corrected_physical_value_'+value,value in pdftext,value)

check('five_workbooks',len(list((FINAL/'workbooks').glob('result*.xlsx')))==5,len(list((FINA
L/'workbooks').glob('result*.xlsx'))))
    check('AI_details_nonempty',len(ai.pages)>0 and len(".join(p.extract_text() for p in
ai.pages))>200,len(ai.pages))

check('charts_present',len(list((FINAL/'charts').glob('*.*png')))>=4,len(list((FINAL/'charts').gl
ob('*.*png'))))
    (FINAL/'qa/paper_text.txt').write_text(pdftext)
    from competition_checks import inspect
    checks.extend(inspect(doc,pdf))
    result=dict(status='PASS' if all(c['status']=='PASS' for c in checks) else
'FAIL',generated_at=datetime.now().astimezone().isoformat(),scope='current_run_numeric_d
ocument_and_export_checks',checks=checks,metrics=dict(pdf_pages=len(pdf.pages),docx_ta
bles=len(doc.tables),specified_date_groups=len(seen)),limitations=['Historical scenarios do
not guarantee unseen tail-risk coverage.','Refund and delivery-time pricing are declared
interpretations, not additional official rules.','Solver diagnostics cover 32 exact LP instances,
not full-year alternative-solver policy rollouts.','Team submission approval and authorship
signoff are not asserted.'])
    result['limitations']=['Finite historical scenarios and one-year replays do not guarantee
future tail-risk coverage.','Refund and delivery-time pricing remain declared

```

interpretations.', 'Annual alternative-solver checks cover S2 and S3 with a fixed common reference contract; no test-period policy selection.', 'Team authorship and manual review are pending; this PASS is not a submission eligibility certificate.']

```
result['submission_readiness']='PENDING_TEAM_MANUAL_REVIEW_AND_RULES_CONFIRMATION'
```

```
artifacts=list((FINAL/'paper').glob('*.*docx'))+list((FINAL/'paper').glob('*.*pdf'))
+list((FINAL/'workbooks').glob('*.*xlsx'))
```

```
result['artifact_hashes']=[dict(path=str(p.relative_to(FINAL)),sha256=hashlib.sha256(p.read_bytes()).hexdigest()) for p in artifacts]
```

```
(FINAL/'final_validation_v3.json').write_text(json.dumps(result,ensure_ascii=False,indent=2))
print(json.dumps(result,ensure_ascii=False,indent=2));return result
```

```
if __name__=='__main__':raise SystemExit(0 if main()['status']=='PASS' else 1)
```

scripts/freeze_codec.py

SHA256 e02914b278ec089c0d24c9612830fc4ca3504ce315fbb169b8e1dee00399fa45

""""Lossless column deduplication for the competition's support-file size limit.

All original CSV strings are preserved. The restored bytes must match SHA256; this is packaging only and never rounds or changes model observations/results.

""""

```
from pathlib import Path
import csv,hashlib,json,lzma,io,shutil,tarfile,tempfile
```

```
def sha(data):return hashlib.sha256(data).hexdigest()
```

```
def pack(source,target):
```

```
source=Path(source);target=Path(target);blobs=target/'columns';blobs.mkdir(parents=True,exist_ok=True)
```

```
manifest={'format':'lossless-csv-columns-v1','files':[]}
```

```
duplicates={'q3_decision_versions.csv','q4_3_decision_versions.csv','q3_S3_decisions.csv','q4_3_S3_decisions.csv','q3_S3_dispatch.csv','q4_3_S3_dispatch.csv'}
```

```
for p in sorted(source.iterdir()):
```

```
    if not p.is_file() or p.name in duplicates:continue
```

```
    if p.suffix!='.csv':shutil.copy2(p,target/p.name);continue
```

```
    raw=p.read_bytes();bom=raw.startswith(b'\xef\xbb\xbf');newline='\r\n' if b'\r\n' in raw
```

```
else '\n'
```

```
    reader=csv.reader(io.StringIO(raw.decode('utf-8-
```

```
sig'),newline="));header=next(reader);cols=[[ ] for _ in header];count=0
```

```
    for row in reader:
```

```
        if len(row)!=len(header):raise ValueError(('Nonrectangular CSV',p.name))
```

```
        count+=1
```

```
        for col,val in zip(cols,row):
```

```
            if '\n' in val or '\r' in val:raise ValueError(('Multiline value',p.name))
```

```

        col.append(val)
    ids=[]
    for col in cols:
        data='\n'.join(col).encode('utf-8');key=sha(data);out=blobs/(key+'.xz')
        if not out.exists():out.write_bytes(lzma.compress(data,preset=6))
        ids.append(key)

record=dict(name=p.name,sha256=sha(raw),bytes=len(raw),bom=bom,newline=newline,header=header,rows=count,columns=ids)
    # Verify the CSV dialect before recording a supposedly lossless manifest.
    if sha(reconstruct(record,cols))!=record['sha256']:raise ValueError(('CSV byte roundtrip failed',p.name))
    manifest['files'].append(record);print('Packed losslessly',p.name,flush=True)
    (target/'csv_manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2))
    return manifest

def reconstruct(record,cols):

    out=io.StringIO(newline="");writer=csv.writer(out,lineterminator=record['newline']);writer.writerow(record['header']);writer.writerows(zip(*cols))
    return out.getvalue().encode('utf-8-sig' if record['bom'] else 'utf-8')

def unpack(source,target):
    source=Path(source);target=Path(target);target.mkdir(parents=True,exist_ok=True)
    manifest=json.loads((source/'csv_manifest.json').read_text())
    solid={}
    if (source/'columns.tar.xz').exists():
        with tarfile.open(source/'columns.tar.xz','r:xz') as archive:
            for member in archive:
                if not member.isfile() or len(member.name)!=64:raise ValueError('Invalid column member')
                data=archive.extractfile(member).read()
                if sha(data)!=member.name:raise ValueError('Corrupt solid column')
                solid[member.name]=data
    for record in manifest['files']:
        cols=[]
        for key in record['columns']:
            data=solid[key] if solid else
lzma.decompress((source/'columns'/(key+'.xz')).read_bytes())
            if sha(data)!=key:raise ValueError('Corrupt column: '+key)
            vals=data.decode('utf-8').split('\n') if record['rows'] else []
            if len(vals)!=record['rows']:raise ValueError('Wrong row count')
            cols.append(vals)
        raw=reconstruct(record,cols)
        if sha(raw)!=record['sha256']:raise ValueError('Corrupt CSV: '+record['name'])
        (target/record['name']).write_bytes(raw)
    for p in source.iterdir():
        if p.is_file() and p.name not in ['csv_manifest.json','columns.tar.xz'] and p.parent!=target:shutil.copy2(p,target/p.name)
    return len(manifest['files'])

```

```

def solidify(source,target):
    source=Path(source)
    manifest=json.loads((source/'csv_manifest.json').read_text())
    labels={}
    for r in manifest['files']:
        for label,key in zip(r['header'],r['columns']):labels.setdefault(key,label)
    keys=sorted(labels,key=lambda key:(labels[key],key))
    with
lzma.open(target,'wb',format=lzma.FORMAT_XZ,filters=[{'id':lzma.FILTER_LZMA2,'prese
t':9,'dict_size':128*1024*1024}]) as dst:
        with tarfile.open(fileobj=dst,mode='w|') as archive:
            for key in keys:
                p=source/'columns'/(key+'.xz')

raw=lzma.decompress(p.read_bytes());info=tarfile.TarInfo(p.stem);info.size=len(raw);info.mt
ime=0;archive.addfile(info,io.BytesIO(raw))

if __name__=='__main__':
    import argparse

ap=argparse.ArgumentParser();ap.add_argument('operation',choices=['pack','unpack']);ap.add
_argument('source');ap.add_argument('target');a=ap.parse_args()
(pack if a.operation=='pack' else unpack)(a.source,a.target)

```

scripts/make_charts.py

```

SHA256 c2f3803d727c9d75ad253ae2b9bf5a7d954a529110bac539847f08e97c7f6879
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT/'work/python_deps'))
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np,pandas as pd
F=ROOT/'outputs/c_final_v1/frozen';C=F.parent/'charts';C.mkdir(exist_ok=True)
plt.rcParams.update({'font.family':'DejaVu
Sans','font.size':10,'axes.spines.top':False,'axes.spines.right':False,'figure.dpi':160})
s=pd.read_csv(F/'experiment_summary.csv');official=s[(s.model_arm=='joint_empirical')&((s
['mode'].isin(['q2','q4_2']))|(s.scenario=='S3'))]
fig,ax=plt.subplots(figsize=(8,3.6));x=np.arange(len(official));bottom=np.zeros(len(x))
for col,label,color in [('plan_cost_yuan','Scheduled','#35658a'),('adjustment_cost_yuan','Net
revisions','#dfa04f'),('emergency_cost_yuan','Emergency','#b85450')]:
    vals=official[col].to_numpy()/1e6;ax.bar(x,vals,bottom=bottom,label=label,color=color);bott
om+=vals
ax.set_xticks(x,official['mode']);ax.set_ylabel('Realized cost / million
yuan');ax.legend(ncol=3,frameon=False);fig.tight_layout();fig.savefig(C/'cost_components.pn
g');plt.close(fig)
fig,axs=plt.subplots(1,2,figsize=(8,3.5))

```

```

for ax,mode in zip(axes,['q3','q4_3']):

d=s[(s['mode']==mode)&(s.model_arm=='joint_empirical')].sort_values('scenario');ax.plot(d.scenario,d.final_relative_cost_yuan/1e6,'o-',color='#35658a');ax.set_title(mode.upper());ax.set_ylabel('Main cost / million yuan');ax.grid(axis='y',alpha=.2)
fig.tight_layout();fig.savefig(C/'schedule_comparison.png');plt.close(fig)
fig,axes=plt.subplots(2,1,figsize=(8,4.8),sharex=True);d=pd.read_csv(F/'q3_dispatch.csv');d=d[d.date=='2025-06-21'];h=(d.position.to_numpy()-1)/6
axes[0].plot(h,d.plan_00_grid_kwh,label='Common day-ahead');axes[0].plot(h,d.final_adjusted_grid_kwh,label='Final commitment');axes[0].fill_between(h,0,d.emergency_kwh,color='#b85450',alpha=.5,label='Emergency');axes[0].set_ylabel('Energy / kWh');axes[0].legend(ncol=3,frameon=False,fontsize=8)
axes[1].plot(h,d.soc_start_kwh,color='#35658a');axes[1].axhline(1200,ls='--',color='gray');axes[1].axhline(10800,ls='--',color='gray');axes[1].set_ylabel('SOC / kWh');axes[1].set_xlabel('Physical hour on 2025-06-21');fig.tight_layout();fig.savefig(C/'representative_dispatch.png');plt.close(fig)
fig,ax=plt.subplots(figsize=(8,3.6))
for mode,color in [('q3','#35658a'),('q4_3','#b85450')]:
a=pd.read_csv(F/f'{mode}_S0_daily.csv');b=pd.read_csv(F/f'{mode}_S3_daily.csv');delta=b.final_relative_cost_yuan-a.final_relative_cost_yuan;ax.plot(pd.to_datetime(a.date),delta.cumsum()/1000,label=mode,color=color)
ax.axhline(0,color='gray',lw=.8);ax.set_ylabel('Cumulative S3 - S0 / thousand yuan');ax.legend(frameon=False);fig.autofmt_xdate();fig.tight_layout();fig.savefig(C/'cumulative_difference.png');plt.close(fig)
print('Four data-derived figures written',flush=True)
monthly=pd.read_csv(F.parent/'research/monthly_mechanisms.csv')
fig,axes=plt.subplots(2,1,figsize=(8,5.0),sharex=True)
for mode,color in zip(['q2','q4_2','q3','q4_3'], ['#35658a','#b85450','#548764','#9b6a9a']):
z=monthly[monthly['mode']==mode];x=np.arange(len(z))
axes[0].plot(x,z.cost/1e6,'o-',color=color,label=mode)
axes[1].plot(x,z.emergency_cost/z.cost*100,'o-',color=color)
axes[0].set_ylabel('Monthly cost / million yuan');axes[0].legend(ncol=4,frameon=False)
axes[1].set_ylabel('Emergency cost share / %');axes[1].set_xticks(x,z.month.str[5:]);axes[1].set_xlabel('Month in 2025')
for ax in axes:ax.grid(axis='y',alpha=.2)
fig.tight_layout();fig.savefig(C/'monthly_risk.png');plt.close(fig)

```

scripts/native_equations.py

SHA256 178b05030cd6abeb1ca97603ddfad843c100387694b29c9a0a789e69cac7b2dd
 """Small native OMML builder for the paper's reviewed equations.

Supports grouped subscripts/superscripts and fractions, without raster formulas or Unicode presentation subscripts that some DOCX converters overlap.
 """

```

from docx.oxml import OxmlElement
from docx.oxml.ns import qn
import unicodedata

```

```

EQUATIONS = {
1: r'F_τ = σ {观测结束时刻 ≤ τ; 预报发布时间 ≤ τ; 既有采购承诺}',
2: r'A_t + V_t + D_t + Q_t = L_t + C_t + U_t; Q_t, U_t ≥ 0',
3: r'E_{t+1} = E_t + 0.9 C_t - \frac{D_t}{0.9}',
4: r'1200 ≤ E_t ≤ 10800; 0 ≤ C_t, D_t ≤ \frac{5000}{6}',
5: r'\tilde{X}_t^s = \max\{0, \hat{X}_{t|\tau} + (X_{h^s} - \hat{X}_{h^s|\tau_h})\}',
6: r'G_t + D_t - C_t + Q_t^s ≥ L_t^s - V_t^s; Q_t^s ≥ 0, \forall s, t \in H_\tau',
7: r'G_t^s = G_t, C_t^s = C_t, D_t^s = D_t, E_t^s = E_t, \forall s',
8: r'\min \frac{1}{N_\tau} \sum_s \sum_t [p_t^s G_t + 5 p_t^s Q_t^s] + \lambda |E_{\text{末}} - 6000| + \epsilon \sum_t (C_t + D_t)',
9: r'f_t(x) = p_t x + 0.5 p_t |x|',
10: r'J_{\text{净}} = \sum_t [p_t B_t + f_t(A_t^{\text{末}} - B_t) + 5 p_t Q_t]',
11: r'J_{\text{逐次}} = \sum_t [p_t B_t + \sum_k f_t(A_t^k - A_t^{k-1}) + 5 p_t Q_t]',
12: r'J_{\text{逐次}} - J_{\text{净}} = 0.5 \sum_t p_t [\sum_k |\Delta A_t^k| - |\sum_k \Delta A_t^k|] ≥ 0',
}

```

```

def node(tag, children=()):
    n=OxmlElement('m:'+tag)
    for c in children:n.append(c)
    return n

```

```

def run(text):
    r=node('r');pr=node('rPr');sty=node('sty');sty.set(qn('m:val'),'p');pr.append(sty);r.append(pr)
    wr=OxmlElement('w:rPr');font=OxmlElement('w:rFonts')
    for k in ['ascii','hAnsi','eastAsia']:font.set(qn('w:'+k),'Arial Unicode MS')

    wr.append(font);size=OxmlElement('w:sz');size.set(qn('w:val'),'20');wr.append(size);r.append(wr)
    t=node('t');t.set(qn('xml:space'),'preserve');t.text=text;r.append(t);return r

```

```

def parse(text):
    i=0
    def atom():
        nonlocal i
        if text[i]=='{':
            i+=1;out=sequence(True);return out
        if text.startswith(r'\frac',i):
            i+=5;a=atom();b=atom();return [node('f',[node('num',a),node('den',b)])]
        c=text[i];i+=1
        while i<len(text) and unicodedata.combining(text[i]):c+=text[i];i+=1
        return [run(c)]
    def sequence(group=False):
        nonlocal i
        out=[]
        while i<len(text):
            if group and text[i]=='}':i+=1;return out

```

```

base=atom();sub=sup=None
while i<len(text) and text[i] in '_^':
    marker=text[i];i+=1;value=atom()
    if marker=='_':sub=value
    else:sup=value
    if sub is not None and sup is not None:out.append(node('sSubSup',
[node('e',base),node('sub',sub),node('sup',sup)]))
    elif sub is not None:out.append(node('sSub',[node('e',base),node('sub',sub)]))
    elif sup is not None:out.append(node('sSup',[node('e',base),node('sup',sup)]))
    else:out.extend(base)
return out
return sequence()

```

```

def equation(number):
    # Literal grouping braces outside scripts are displayed as parentheses.
    source=EQUATIONS[number]
    if number in
(1,5):source=source.replace('σ','σ').replace('max','max');source=source[:-1]+'
    return node('oMath',parse(source)+[run(' '+str(number)+'')])

```

scripts/package_final.py

```

SHA256 a3318552522c7eda1848c923e59b87b3f4b5f33a25d97c6d7d33675026385ce6
from pathlib import Path
from datetime import datetime
import hashlib,json,lzma,shutil,zipfile,tempfile,os
import re
from freeze_codec import pack,solidify
ROOT=Path(__file__).resolve().parents[1];FINAL=ROOT/'outputs/c_final_v1'
def digest(p):return hashlib.sha256(p.read_bytes()).hexdigest()
def main():
    v=json.loads((FINAL/'final_validation_v3.json').read_text())
    if v['status']!='PASS':raise RuntimeError('Refusing to package failed current run')
    source=[]
    for folder in ['src','scripts','tests','paper','config']:
        for p in (ROOT/folder).rglob('*'):
            if p.is_file() and '__pycache__' not in
p.parts:source.append(dict(path=str(p.relative_to(ROOT)),sha256=digest(p)))

(FINAL/'evidence/active_source_hashes.json').write_text(json.dumps(source,ensure_ascii=False,indent=2))
provenance=json.loads((FINAL/'evidence/source_provenance.json').read_text())
original=[r for r in provenance if r.get('kind')=='original_input']
for r in original:
    source_root=os.environ.get('CUMCM_SOURCE_ROOT',json.loads((ROOT/'config/
final.yaml').read_text())['source_root'])
    if digest(Path(source_root)/r['path'])!=r['sha256']:raise RuntimeError('Original input
changed: '+r['path'])
    (FINAL/'evidence/source_provenance.json').write_text(json.dumps(original+
[dict(kind='current_source',**r) for r in source],ensure_ascii=False,indent=2))

```

```

cache=ROOT/'work/packed_frozen';manifest=cache/'csv_manifest.json';solid=ROOT/'work/
columns_solid.tar.xz';tag=ROOT/'work/columns_solid_manifest.sha256'

duplicates={'q3_decision_versions.csv','q4_3_decision_versions.csv','q3_S3_decisions.csv','q
4_3_S3_decisions.csv','q3_S3_dispatch.csv','q4_3_S3_dispatch.csv'}
expected={p.name:digest(p) for p in (FINAL/'frozen').glob('*.*csv') if p.name not in
duplicates}
valid=manifest.exists() and {r['name']:r['sha256']} for r in json.loads(manifest.read_text())
['files']}==expected
if not valid:pack(FINAL/'frozen',cache)
if not solid.exists() or not tag.exists() or tag.read_text()!=digest(manifest):
    solidify(cache,solid);tag.write_text(digest(manifest))
with tempfile.TemporaryDirectory(prefix='v3-package-') as td:
    stage=Path(td)
    for folder in ['src','scripts','tests','paper','config']:
        shutil.copytree(ROOT/folder,stage/folder,ignore=shutil.ignore_patterns('__pycache__'))
        for folder in ['audit','evidence','workbooks','charts','logs','research']:
            shutil.copytree(FINAL/folder,stage/folder)
        # Full research trajectories are in the common lossless column archive.
        for p in (stage/'research').glob('*_ipm.csv.xz'):p.unlink()
        for name in ['run_pipeline.sh','requirements.txt']:shutil.copy2(ROOT/name,stage/name)
        shutil.copy2(FINAL/'paper/AI 工具使用详情.pdf',stage/'AI 工具使用详
情.pdf');shutil.copy2(FINAL/'final_validation_v3.json',stage/'final_validation_v3.json')
        (stage/'frozen').mkdir()
        shutil.copy2(manifest,stage/'frozen/csv_manifest.json');shutil.copy2(solid,stage/'frozen/
columns.tar.xz')
        for p in (FINAL/'frozen').iterdir():
            if not p.is_file() or p.name in duplicates:continue
            if p.suffix!='.csv':shutil.copy2(p,stage/'frozen'/p.name)
        (stage/'README.md').write_text("# V3 闭环修订支撑材料")

```

原始输入为官方 C 题及附件，未修改观测。先在 config/final.yaml 配置 source_root（其下应含 C 题/附件），或设置 CUMCM_SOURCE_ROOT。运行 run_pipeline.sh 即可重建；运行环境需 Python 及 requirements.txt、Node 与 @oai/artifact-tool，DOCX 转 PDF 使用 Codex 文档运行时。对冻结结果只读检查可先执行 scripts/unpack_frozen.py。

frozen 保存全精度结果，CSV 按列去重后固实 XZ 压缩，未舍入任何数字。csv_manifest 保存每份 CSV 的原始 SHA256；unpack_frozen 将逐字节恢复并校验，另恢复重复 S3 轨迹及正式子账本，输出到 outputs/c_final_v1/frozen，随后可运行 src/audit/audit_final.py --workbooks。主结果 S3 与 S0/S1/S2 共享参考日前承诺；所有年度状态来自一月可核验启动账本。训练配置见 resolved_config，不能用未解析默认值替代。

论文费用按取消退原价保留 50%违约金、交付物理区间价解释；同路径逐次与不退款费用另列。不声称其他结算解释下最优。历史场景为预测分布，非伪造的实际数据。未宣称团队已经完成提交签核。

```

")
    for p in stage.rglob('*'):
        if p.is_file() and p.suffix in ['.log','.md','.json','.txt'] and not
p.is_relative_to(stage/'frozen'):
            content=p.read_text(encoding='utf-8')
            content=re.sub('/'+Users/['^\\s\\'\\'\\x27]+','<LOCAL_HOME>',content)
            p.write_text(content,encoding='utf-8')
            entries=[dict(path=str(p.relative_to(stage)),bytes=p.stat().st_size,sha256=digest(p)) for p in
stage.rglob('*') if p.is_file()]

(stage/'SHA256SUMS.json').write_text(json.dumps(dict(generated_at=datetime.now().astime
zone().isoformat(),files=entries),ensure_ascii=False,indent=2))
    with zipfile.ZipFile(FINAL/'支撑材料.zip','w',zipfile.ZIP_DEFLATED) as z:
        for p in stage.rglob('*'):
            if p.is_file():z.write(p,str(p.relative_to(stage)))
        size=(FINAL/'支撑材料.zip').stat().st_size
        if size>20_000_000:raise RuntimeError(f'Support archive exceeds 20MB: {size}')
        files=[dict(path=str(p.relative_to(FINAL)),bytes=p.stat().st_size,sha256=digest(p)) for p in
FINAL.rglob('*') if p.is_file() and p.name!='SHA256SUMS.json' and 'qa' not in
p.relative_to(FINAL).parts]

(FINAL/'SHA256SUMS.json').write_text(json.dumps(dict(generated_at=datetime.now().asti
mezone().isoformat(),files=files,archive_bytes=size),ensure_ascii=False,indent=2));print('Pac
kaged',size,'bytes',flush=True)
if __name__=='__main__':main()

```

scripts/paper_content.py

```

SHA256 a64bdd16d9e2f314521aa672578924eb4ec4381f2302c29d81eaff82547e7ec0
from pathlib import Path
from decimal import Decimal,ROUND_HALF_UP
import json,re
import pandas as pd,numpy as np
ROOT=Path(__file__).resolve().parents[1];F=ROOT/'outputs/c_final_v1/frozen'

def fmt(x,places=2):
    rounded=Decimal(str(float(x))).quantize(Decimal(1).scaleb(-
places),rounding=ROUND_HALF_UP)
    return format(abs(rounded) if rounded==0 else rounded,f',{places}f')

def table(headers,rows):
    return '\n'.join(['| '+'| '.join(headers)+'|','| '+'| '.join(['---']*len(headers))+'|']+[ ' '+'|
|.join(map(str,r))+| ' for r in rows])

def emergency(z):
    groups=[];active=[]

```

```

def add():
    a=(active[0].position-1)*10;b=active[-1].position*10;groups.append([f'{a//60}:{a%60:02d}-{b//60}:{b%60:02d}',fmt(sum(x.emergency_kwh for x in active),4)])
    for x in z.itertuples():
        if x.emergency_kwh>=.00005:active.append(x)
        elif active:active.add();active=[]
    if active:active.add()
    return groups

def specified():
    lines=[];checks=[]
    for mode in ['q1','q2','q3','q4_2','q4_3']:
        d=pd.read_csv(F/f'{mode}_dispatch.csv');days=['2025-01-01'] if mode=='q1' else ['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
        for day in days:
            z=d[d.date==day].sort_values('position');lines.append(f'##### {mode.upper()} {day}
指定结果')
            rows=[]
            for h in [10,12,14,16,18,20]:
                physical=f'{h:02d}:00-{h:02d}:10';selected=z[z.physical_interval==physical]
                if len(selected)!=1:raise ValueError('Nonunique physical interval')

x=selected.iloc[0];rows.append([physical,fmt(x.plan_00_grid_kwh,4),fmt(x.final_adjusted_grid_kwh,4)])

checks.append(dict(mode=mode,date=day,kind='ten_minute',key=physical,plan_grid=float(x.plan_00_grid_kwh),final_grid=float(x.final_adjusted_grid_kwh)))
    lines.append(table(['物理区间','0点计划量','最终承诺量'],rows))
    lines.append(table(['全天计划量','全天最终量','全天总购电费'],
[[fmt(z.plan_00_grid_kwh.sum(),4),fmt(z.final_adjusted_grid_kwh.sum(),4),fmt(z.final_relative_cost_yuan.sum())]]))
    rows=[]
    for b in range(6):
        zz=z[(z.position>b*24)&(z.position<=(b+1)*24)];span=f'{b*4}:00-{(b+1)*4}:00';rows.append([span,fmt(zz.charge_kwh.sum(),4),fmt(zz.discharge_kwh.sum(),4)])
        checks.append(dict(mode=mode,date=day,kind='four_hour',key=span,charge=float(zz.charge_kwh.sum()),discharge=float(zz.discharge_kwh.sum())))
        lines.append(table(['物理区间','充电量','放电量'],rows));lines.append(table(['0点SOC','24点SOC'],[[fmt(z.soc_start_kwh.iloc[0],4),fmt(z.soc_end_kwh.iloc[-1],4)]]))
        if mode!='q1':lines.append(table(['紧急购电物理区间','紧急购电量'],emergency(z) or ['无正缺口','0.0000'])))

(F/'paper_specified_values.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2))
    return '\n\n'.join(lines)

def generate():
    cfg=json.loads((F/'resolved_config.json').read_text());exp=pd.read_csv(F/'experiment_summa

```

```

ry.csv');official=pd.read_csv(F/'official_summary.csv');training=pd.read_csv(F.parent/'eviden
ce/
january_parameter_validation.csv');perfect=pd.read_csv(F/'perfect_information_summary.csv
');jan=pd.read_csv(F/'startup_summary.csv');ci=pd.read_csv(F/'comparison_stats.csv');settle=
pd.read_csv(F/'settlement_sensitivity.csv');solver=pd.read_csv(F/'solver_instance_sensitivity.
csv');audit=json.loads((F.parent/'audit/
independent_validation.json').read_text());q1=pd.read_csv(F/'q1_dispatch.csv')
def row(mode,scenario='S0',arm='joint_empirical'):
    z=exp[(exp['mode']==mode)&(exp.scenario==scenario)&(exp.model_arm==arm)]
    if len(z)!=1:raise ValueError((mode,scenario,arm,len(z)))
    return z.iloc[0]

values={'Q1_COST':fmt(q1.final_relative_cost_yuan.sum()),'Q2_COST':fmt(row('q2').final_r
relative_cost_yuan),'Q42_COST':fmt(row('q4_2').final_relative_cost_yuan),'Q3_DIFF':fmt(ro
w('q3','S3').final_relative_cost_yuan-
row('q3').final_relative_cost_yuan),'Q43_DIFF':fmt(row('q4_3','S3').final_relative_cost_yuan-
row('q4_3').final_relative_cost_yuan),'SELECTED_MODEL':{'raw_history':'原始历史联合
场景','causal_residual':'逐日前推因果残差场景'}[cfg['forecast']
['scenario_model']],SELECTED_MULT:str(cfg['rolling']
['selected_soft_penalty_multiplier'])}
baseline=float(np.sum(q1.settlement_price*np.maximum(q1.actual_load_kwh-
q1.actual_pv_kwh,0)))
values['Q1_TABLE']=table(['方案','全天购电量','全天费用'],[['储能
LP',fmt(q1.plan_00_grid_kwh.sum(),4),values['Q1_COST']],['无储
能',fmt(np.maximum(q1.actual_load_kwh-q1.actual_pv_kwh,0).sum(),4),fmt(baseline)]]))

tr=training.groupby(['scenario_model','penalty_multiplier']).final_relative_cost_yuan.sum();v
alues['TRAIN_TABLE']=table(['场景候选','惩罚倍数','四模式一月验证合计费用'],
[[sm,str(mult),fmt(cost)] for (sm,mult),cost in tr.items()])
rows=[]
for mode in ['q2','q4_2']:
    for arm,label in [('joint_empirical','联合场景风险采购'),('mean_forecast','均值预测'),
('no_storage','无储能')]:
r=row(mode,arm=arm);rows.append([mode,label,fmt(r.final_relative_cost_yuan),fmt(r.emerg
ency_kwh,4)])
p=perfect[perfect['mode']==mode].iloc[0];rows.append([mode,'完全信息事后下
界',fmt(p.purchase_cost_yuan),'0.0000'])
values['RISK_TABLE']=table(['模式','方案','正式期总费用','紧急购电量'],rows)
values['SETTLEMENT_TABLE']=table(['模式','最终净额','同链逐次','不退原价同路径'],
[[r['mode'],fmt(r.main_cost),fmt(r.same_chain_sequential_cost),fmt(r.no_refund_same_path_
cost)] for _,r in settle.iterrows()])
values['STARTUP_TABLE']=table(['模式','一月启动费用','一月末 SOC','二月初 SOC'],
[[r['mode'],fmt(r.cost_yuan),fmt(r.terminal_soc_kwh,4),fmt(pd.read_csv(F/f'{r["mode"]}
_dispatch.csv').soc_start_kwh.iloc[0],4)] for _,r in jan.iterrows()])

```

```

values['OFFICIAL_TABLE']=table(['模式','计划费','调整净费','紧急费','总费用'],
[[r['mode'],fmt(r.plan_cost_yuan),fmt(r.adjustment_cost_yuan),fmt(r.emergency_cost_yuan),f
mt(r.final_relative_cost_yuan)] for _,r in official.iterrows()])
values['SCHEDULE_TABLE']=table(['模式','时点策略','主费用','逐次费用','紧急购电
量'],
[[mode,sc,fmt(row(mode,sc).final_relative_cost_yuan),fmt(row(mode,sc).sequential_cost_yua
n),fmt(row(mode,sc).emergency_kwh,4)] for mode in ['q3','q4_3'] for sc in
['S0','S1','S2','S3']])
statements=[]
for mode in ['q3','q4_3']:
    costs={sc:float(row(mode,sc).final_relative_cost_yuan) for sc in
['S0','S1','S2','S3']};winner=min(costs,key=costs.get);diff=costs['S3']-
costs['S0'];statements.append(f'{mode.upper()}在本样本共同日前合同下，主费用最低为
{winner}，费用{fmt(costs[winner])}元；S3 相对 S0'+('增加' if diff>=0 else '减少')
+f{fmt(abs(diff))}元。该差额来自本算法在更新信息下选择的后续采购和储能，不能推
广成信息本身必然有害或必然有益。')

```

```

values['SCHEDULE_NARRATIVE']='\n\n'.join(statements);values['CI_TABLE']=table(['模
式','比较','配对差额','95%区间下端','95%区间上端'],
[[r['mode'],r.comparison,fmt(r.difference_yuan),fmt(r.ci_low),fmt(r.ci_high)] for _,r in
ci.iterrows()])

```

values['SOLVER_NARRATIVE']=f'共完成{len(solver)}次固定实例复求。相对 HiGHS 自动法的目标最大绝对差为{solver.objective_difference_vs_highs.abs().max():.6g}元，窗口采购量最大 L1 差为{solver.grid_l1_difference.max():.6g} kWh；最大 SOC 递推残差为{solver.soc_residual.max():.3e} kWh。目标接近而动作不一致时应视为并列解敏感性，全年策略排序仍需额外完整回放才能确认。'

failures=[c for c in audit['checks'] if c['status']!=
='PASS'];values['AUDIT_NARRATIVE']=f'本轮独立检查共{len(audit["checks"])}项，失败{len(failures)}项。检查覆盖逐时段原数据匹配、物理区间、设备边界、真实缺口及费用、正式承诺连续性、共同日前合同和一月衔接。测试还覆盖未来观测隔离、分段状态不重置、不同充放电效率和年末边界。该结果是所列检查范围内的证据，不是模型最优性或外推有效性的证明。'

if failures:raise RuntimeError('Refusing to publish paper over failed independent checks')

```

values['SPECIFIED_TABLES']=specified();values['REPRODUCTION_TABLE']=table(['证
据','位置'],[['正式明细','frozen/q*_dispatch.csv（解包后）'],['真实承诺链','frozen/
decision_versions.csv（解包后）'],['训练与解析配置','evidence/
january_parameter_validation.csv 及 frozen/resolved_config.json'],['独立复核','audit/
independent_validation.json'],['源码与输入校验','evidence/source_provenance.json'],['可执
行入口','run_pipeline.sh'],['数学约束测试','tests/test_closed_loop.py']])
text=(ROOT/'paper/main.md').read_text()
for key,val in values.items():text=text.replace('{{'+key+'}}',val)
if re.search(r'\{\{[^}]+\}\}',text):raise ValueError('Unresolved result macro')
ai=""# AI 工具使用详情

```

使用范围与责任边界

本轮在 OpenAI Codex 任务中使用基于 GPT-6 的智能体，辅助独立复核、程序修订、实验组织、数值审计及文档生成。此前会话的具体模型版本无法由现存材料完整验证，未猜填。用户于本轮明确授权修复，并要求使用真实数据、避免未来信息泄漏、遵守数学模型约束。

输入与操作

输入包括原题 PDF、附件 1 至 5、V3 源码与交付文件、两份历史审查报告。本轮保留修订前快照，停止旧模型后台重算后建立新的正式运行。核心改动包括因果联合场景、共享决策约束、真实承诺逐次计费、共同日前计划比较、可核验一月启动、物理时间抽取及独立交付校验。

数据与预测

原始观测不作伪造或未来回填。残差场景由已观测历史构造，属于明确标记的预测分布；移动块重采样用于统计区间，不当作新观测数据。没有历史时不使用当天实际曲线预测当天。一月训练选择完成后评价二月至十二月。

可核查记录

运行日志位于 logs；配置为 frozen/resolved_config.json；训练表为 evidence/january_parameter_validation.csv；一月和正式期轨迹、各策略承诺均位于 frozen；独立复算位于 audit；源码与原始输入哈希在 evidence/source_provenance.json；数学约束和反事实测试见 tests/test_closed_loop.py。

尚需人工审阅的内容

退费和交付时段价格是题意解释，不能冒充额外给定的市场规则。经验场景与有限窗口不保证覆盖所有极端情形。团队需人工审阅假设、策略选择、公式、程序、引文与最终提交要求。本文件只登记可验证的本轮操作，不声称此前全部记录已恢复，也不声称已完成团队签核。

'''

return text,ai

scripts/rebuild_control_details.py

```
SHA256 1c634a0c2b71c370ffbd2c7186dda4abea47039e8c612d7b6a0f7da8562afd90
""""Recover full control trajectories for a run initially saved with daily controls."""
from pathlib import Path
import sys,json
```

```

import pandas as pd
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
from src.model.config import load_config
from src.model.data import load_data
from src.model.experiments import write_csv
from src.model.rolling import simulate,RunSpec
cfg=load_config('config/
final.yaml');F=Path(cfg['_output_root'])/'frozen';cfg.update(json.loads((F/'resolved_config.json').read_text()));data=load_data(cfg)
for mode in ['q2','q4_2']:
    for arm,point,nostore in [('mean_forecast',True,False),('no_storage',False,True)]:
        initial=float(pd.read_csv(F/f'{mode}
_january_dispatch.csv').soc_end_kwh.iloc[-1]);r=simulate(data,cfg,RunSpec(mode,'S0',arm,collect_detail=False),initial_soc=initial,point=point,no_storage=nostore)
        old=pd.read_csv(F/f'{mode}_{arm}
_daily.csv');expected=float(old.final_relative_cost_yuan.sum());actual=r['summary']
['final_relative_cost_yuan']
        if abs(expected-actual)>1e-6:raise AssertionError((mode,arm,expected,actual))
        write_csv(F/f'{mode}_{arm}_dispatch.csv',r['dispatch']);write_csv(F/f'{mode}_{arm}
_solver.csv',r['solve_stats']);print('Recovered verified control',mode,arm,actual,flush=True)

```

scripts/research_content.py

```

SHA256 ab5ee95b15d1f1ee6d4b00eeced9c56b701d930476c6368dd885a9feb81c85d4
""""Inject measured diagnostics and a byte-verifiable complete source appendix.""""
from pathlib import Path
import json, hashlib
import numpy as np
import pandas as pd
from docx import Document
from docx.shared import Pt
from docx.oxml.ns import qn
from paper_content import table,fmt
ROOT=Path(__file__).resolve().parents[1]
FINAL=ROOT/'outputs/c_final_v1'
R=FINAL/'research'

def source_files():
    files=[]
    for folder in ['src','scripts','tests']:
        files.extend(p for p in (ROOT/folder).rglob('*') if p.is_file() and p.suffix in ['.py','.mjs']
and '__pycache__' not in p.parts)
    files.extend([ROOT/'run_pipeline.sh',ROOT/'requirements.txt',ROOT/'config/final.yaml'])
    return sorted(files,key=lambda p:str(p.relative_to(ROOT)))

def append_sources(path):
    doc=Document(path)
    h=doc.add_paragraph('附录 C 完整源程序与运行配置',style='Heading
2');h.paragraph_format.page_break_before=True

```

doc.add_paragraph('以下逐文件收录当前工程自编的全部 Python、JavaScript、Shell 程序和配置。第三方依赖按 requirements.txt 安装；工作簿导出还需 Node 与 @oai/artifact-tool，文档转换需配套 LibreOffice 运行环境。程序以支撑材料内的原始文件运行，版面换行不改变源文件。独立数值复验仅依赖 Python，不要求文档转换环境。')

```
records=[]
for p in source_files():
    name=str(p.relative_to(ROOT));raw=p.read_bytes();content=raw.decode('utf-8')
    doc.add_paragraph(name,style='Heading 3')
    note=doc.add_paragraph('SHA256 '+hashlib.sha256(raw).hexdigest())
    note.paragraph_format.space_after=Pt(2)
    for run in note.runs:run.font.size=Pt(7)
    for line in content.splitlines():

para=doc.add_paragraph();para.paragraph_format.space_after=Pt(0);para.paragraph_format.space_before=Pt(0)
    para.paragraph_format.line_spacing=1.0;para.paragraph_format.keep_together=False
    run=para.add_run(line if line else ' ');run.font.name='Consolas';run.font.size=Pt(7.5)
    run._element.get_or_add_rPr().rFonts.set(qn('w: eastAsia'),'Arial Unicode MS')

records.append({'path':name,'sha256':hashlib.sha256(raw).hexdigest(),'lines':len(content.splitlines())})
doc.save(path)

(FINAL/'evidence/source_appendix_manifest.json').write_text(json.dumps(records,ensure_ascii=False,indent=2))
```

```
def enrich(text,ai):
```

```
annual=pd.read_csv(R/'annual_solver_comparison.csv');tail=pd.read_csv(R/'tail_risk.csv');err=
pd.read_csv(R/'prediction_error.csv');blocks=pd.read_csv(R/'block_sensitivity.csv');months
=pd.read_csv(R/'monthly_mechanisms.csv')
```

```
start=text.index('针对负载');end=text.index("\n\n 关键词",start)
text=text[:start]+""针对微网负载、光伏与电价不确定性，本文以十分钟物理区间为统一尺度，将储能调度、非对称缺口损失与购电承诺建立为可实施的联合优化模型。数据仅来自题目附件，所有预测输入满足观测完成和预报发布时间约束。
```

问题一采用含效率、功率与储电量边界的线性规划，得到日购电费 35,126.95 元。问题二和问题四的日前模型把五倍价紧急采购直接纳入经验场景目标；历史负载、光伏和电价按同日联合取样，候选场景与终端惩罚只在一月选定。二月至十二月费用分别为 14,794,949.72 元和 15,623,134.83 元，相对均值预测控制分别减少 1,662,260.45 元和 1,668,740.06 元。

问题三及其波动电价扩展采用 24 小时滚动窗口。固定共同日前承诺后，0、6、12、18 点更新的 S3 方案相对仅 0 点更新的 S0 方案分别减少 123,506.61 元和 155,342.73 元；同

时报告 S2、不同求解器和计费解释的结果，不把单一路径排序推广为信息价值定理。对真实承诺链逐笔结算，逐次调整相对最终净额分别多 96,160.53 元和 98,826.62 元。

为避免无历史首日的信息泄漏，一月采用明示的保守启动合同，并从 4464 个实际执行时段衔接二月 SOC。以独立物理复算、共同基线、全年替代求解器回放、月度机制分析及块长度敏感性检验形成证据链。结果支持在当前数据与计费解释下显式处理尾部缺口损失和日内信息更新；单个有限窗口的最优解不等于全年策略最优，经验尾部统计也不能保证未知极端场景下的供电表现。"""text[end:]

text=text.replace('微网由小区负载、光伏电源、储能和外部电网构成。','题目[1]中的微网由小区负载、光伏电源、储能和外部电网构成。',1)

intro="""

1.1 方法依据与本题贡献

线性规划提供物理可行域和费用目标的求解基础[2]；样本平均近似以有限场景表示不确定量[3]；滚动控制在新观测到达后更新有限窗口决策[4]。这些是已有方法，本文不把算法名称本身列为创新。本题的针对性工作是：将五倍缺口代价、联合历史误差和储能跨时段耦合放入同一规划；以共同日前合同隔离日内更新频次；记录每次承诺变化，使净额与逐次结算可在同一执行路径上核验。

本文分别回答三个问题：给定信息下决策是否可实施，计费与物理结果是否可复算，以及相对可行基线是否改善经济结果。前两者不能替代第三者；对有限窗口的求解最优性也不作全年策略最优性解释。

1.2 技术路线

原始附件先按日期和物理区间对齐并统一量纲；每次决策截取已完成历史和已发布预报，生成联合场景；在 SOC 和承诺约束下求解，仅执行至下一允许更新时点；区间结束后记录实际缺口、费用和下一状态。训练期选择、正式回放、对照实验及文稿生成分别留存输出，避免把预测样本当成实测数据。本文使用历史数据回放，因此因果信息检查证明的是逐次决策的信息边界，不宣称真实部署过该系统。

"""

text=text.replace("\n## 2 假设与符号',intro+"\n## 2 假设与符号')

proof="""

上述互斥处理具有支配关系。若 $z \geq 0$ ，则 $C' = C - D/(\eta c \eta d)$ 、 $D' = 0$ ，减少的母线需求为 $D(1/(\eta c \eta d) - 1) \geq 0$ ；若 $z < 0$ ，则 $C' = 0$ 、 $D' = D - \eta c \eta d C$ ，减少的母线需求为 $C(1 - \eta c \eta d) \geq 0$ 。两种情形均有 $C' \leq C$ 、 $D' \leq D$ ，故额定功率上界保留，SOC 轨迹不变，采购需求不增。由于本题允许不用富余且不设最小充放功率，此变换给出不劣的可行互斥解。若另加禁止弃电、负电价收益或最小运行时间等条件，需要重新论证，不能直接沿用该结论。

"""

text=text.replace("\n 问题一以 $Q=0$ ',proof+"\n 问题一以 $Q=0$ ')

text=text.replace('每一历史误差也必须相对当时仅用更早日期生成的预测计算，不能用当前的拟合曲线回填过去。','每一历史误差也必须相对当时仅用更早日期生成的预测计算，不能用当前的拟合曲线回填过去。这遵循逐次向前验证的时间切分原则[5]。')

text=text.replace('本文使用完整 LP 进行正式调度。','本文使用完整 LP 进行正式调度。更具体地，单期目标对采购 q 的导数为 $p-5p \cdot \Pr(N>q)$ ，令其为零得到 0.8 分位点；离散经验分布取对应分位区间即可。若价格和缺口相关，期望应对联合分布计算，不能先把价格与需求独立平均。')

text=text.replace('\n## 6 连续启动与因果检验','

5.3 信息价值与实际收益的区别

如果扩大的信息集允许完全忽略新增预报，且优化覆盖相同状态合同下的全部可行策略，则较丰富信息的最优期望费用不会更高：旧策略仍属于新策略集合。这是可行策略集合包含关系的结果，而不是某次实现路径上的费用保证。本文采用有限窗口、有限经验场景和数值求解，因此既不保证每次更新降低实际费用，也不以 S2 略低于 S3 否定信息的一般价值。是否引入更新应依据受控费用、缺口和实现负担共同判断。

6 连续启动与因果检验")

text=text.replace('逐日配对差使用 7 天移动块','逐日配对差参考块自助法处理局部时间相关的思路[6]，使用 7 天移动块')

text=text.replace('分别用 HiGHS 自动、对偶单纯形和内点法复求。','分别用 HiGHS 自动、对偶单纯形和内点法复求。HiGHS 求解器及其对偶单纯形实现的来源见文献[7]。')

text=text.replace('全年策略排序仍需额外完整回放才能确认。','其全年影响进一步通过第 7.3 节的连续回放检验。')

rows=[]

for (mode,method),g in annual.groupby(['mode','method'],sort=False):

a=g.set_index('scenario').cost;rows.append([mode,method,fmt(a['S2']),fmt(a['S3']),fmt(a['S3']-a['S2'])])

diagnostics='\n\n### 7.1 缺口与尾部成本\n\n'

diagnostics+='紧急购电补足实际缺口，以下指标衡量额外采购暴露，不等同于停电概率。每日费用最高的 17 天定义为本样本上尾约 5%，其均值仅作描述性尾部指标；本文没有把目标函数改为 CVaR。'\n\n'

diagnostics+=table(['模式','紧急时段占比','最高日费/元','上尾 17 日均费/元','紧急费占比'],

[[r['mode'],fmt(r.emergency_interval_fraction*100)+'%',fmt(r.max_daily_cost),fmt(r.tail5_mean_cost),fmt(r.emergency_cost_share*100)+'%'] for _,r in tail.iterrows()])

diagnostics+='\n\n### 7.2 预测误差与月度机制\n\n预测误差针对实际执行时段使用的场景均值，Q3 包含多个发布时刻，不能将其与 Q2 的差异单独归因为预测模型。

MAE=mean(|预测-实测|)，RMSE=sqrt(mean((预测-实测)²))；夜间光伏和接近零的量不适合用 MAPE。原始观测未被改动。'\n\n'

```

diagnostics+=table(['模式','变量','MAE','RMSE','偏差'],[[r['mode']],{'load':'负载/
kWh','pv':'光伏/kWh','price':'电价/元每 kWh'}
[r.variable],fmt(r.mae,4),fmt(r.rmse,4),fmt(r.bias,4)] for _,r in err.iterrows() if r.variable!
='price' or r['mode'].startswith('q4'))
diagnostics+='\n\n 月度统计见图 5：成本波动同时取决于负荷、电价和充放电轨迹，
不能仅由一个误差指标解释。\\n\\n[[FIGURE:monthly_risk.png|正式期月度费用与紧急采
购占比]]\\n\\n'
for mode in ['q2','q4_2','q3','q4_3']:

z=months[months['mode']==mode];a=z.loc[z.cost.idxmax()];b=z.loc[(z.emergency_cost/
z.cost).idxmax()]
diagnostics+=f'{mode}月总费最高为{a.month}，{fmt(a.cost)}元；紧急费用占比最
高为{b.month}，{fmt(b.emergency_cost/b.cost*100)}%。\\n\\n'
diagnostics+= '### 7.3 全年替代求解器回放\\n\\n 在补充回放前固定协议，以相同一月
末 SOC、同一组 S0 参考日前承诺和一月已选定参数，分别对两种电价的 S2、S3 运行
HiGHS 内点法。每组连续回放 334 天，不在每日重置状态；保留每段动作并从原始附
件重算费用。四组结果全部报告，不据本实验更换正式方案。\\n\\n'+table(['模式','求解方
法','S2 费用/元','S3 费用/元','S3 减 S2/元'],rows)
for mode in ['q3','q4_3']:

g=annual[annual['mode']==mode].pivot(index='method',columns='scenario',values='cost');delt
a=g.S3-g.S2
diagnostics+=f'\\n\\n{mode} 中，默认方法的 S3 减 S2 为{fmt(delta["highs"])}元，内点
法为{fmt(delta["highs-ipm"])}元。'+('两种方法的排序方向一致，但差距大小随数值方法
变化。' if np.sign(delta['highs'])==np.sign(delta['highs-ipm']) else '两种方法出现排序反
转，不能保留 S2 严格优于 S3 的稳健性结论。')
diagnostics+= '\\n\\n### 7.4 时间相关与块长度敏感性\\n\\n 在同一 334 天费用差上，预先
列出 3、7、14、28 天四种移动块长度，每种重采样 2000 次，使用固定随机种子
20260913。表中列 S3 减 S2，正数表示 S3 较贵；完整 S3 减 S0 区间也随支撑材料交
付。年份存在季节变化，块重采样不保证严格平稳条件，区间用于判断样本内结论对
重采样尺度的敏感性，不解释为下一年收益保证。\\n\\n'
z=blocks[blocks.comparison=='S3-S2'];diagnostics+=table(['模式','块长/天','实际差
额/元','95%下端/元','95%上端/元'],
[[r['mode'],int(r.block_days),fmt(r.difference),fmt(r.ci_low),fmt(r.ci_high)] for _,r in
z.iterrows()])
diagnostics+= '\\n\\n 四种块长度下，S3 相对 S0 的费用差区间均低于零，S3 相对 S2 的
区间均跨零。日内更新具有本样本经济价值，但尚不足以认定增加 18 点更新严格改善
经济性。将更新计算或交易操作成本加入后，还需重新计算其盈亏阈值。\\n\\n'
text=text.replace('\\n## 8 证据链与结论',diagnostics+'\\n## 8 证据链与结论')
text=text.replace('文档构建从持久化 paper/main.md 读取内容，全部结果宏由当前运行
值替换；生成后再核对指定区间和四小时聚合。','文档构建从持久化正文与补充诊断生
成器读取内容，全部表格由对应冻结值生成；生成后再核对指定区间和四小时聚合。')
a=text.index('## 参考材料');b=text.index('## 附录',a)

```

```
text=text[:a]+""## 参考文献
```

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题: 微网与外部电网电力调控策略及附件[Z]. 2026.

[2] BOYD S, VANDENBERGHE L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004. <https://web.stanford.edu/~boyd/cvxbook/>.

[3] SHAPIRO A, DENTCHEVA D, RUSZCZYNSKI A. Lectures on Stochastic Programming: Modeling and Theory[M]. 3rd ed. Philadelphia: SIAM, 2021. DOI:10.1137/1.9781611976595.

[4] RAWLINGS J B, MAYNE D Q, DIEHL M M. Model Predictive Control: Theory, Computation, and Design[M]. 2nd ed. Santa Barbara: Nob Hill Publishing, 2017. <https://sites.engineering.ucsb.edu/~jbrow/mpc/>.

[5] HYNDMAN R J, ATHANASOPOULOS G. Forecasting: Principles and Practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021[2026-09-12]. <https://otexts.com/fpp3/tscv.html>.

[6] KUNSCH H R. The jackknife and the bootstrap for general stationary observations[J]. The Annals of Statistics, 1989, 17(3): 1217-1241. DOI:10.1214/aos/1176347265.

[7] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119-142. DOI:10.1007/s12532-017-0130-5.

```
""+text[b:]
```

```
files=source_files()
```

```
text+='\n\n 支撑材料的必要文件按功能列示如下：每个源码文件在附录 C 逐一给出，完整文件哈希清单见包内 SHA256SUMS.json.\n\n'+table(['目录或文件','内容'],[['src/model','全部模型与数据加载程序'],['src/audit','独立逐时段及工作簿审核程序'],['scripts','运行、补充实验、图表、文稿、打包程序'],['tests','数学约束和未来观测扰动测试'],['config/final.yaml 与 requirements.txt','配置及依赖'],['frozen 与 workbooks','原正式结果及五份指定工作簿'],['research','补充全年回放、尾部、月度、误差与统计诊断'],['evidence 与 audit','来源、源码附录清单及复验记录'],['charts 与 logs','图像及运行日志'],['AI 工具使用详情.pdf','真实使用范围及仍需人工核验事项']])
```

```
ai+=""
```

```
## 本轮继续修订的真实提示与采用情况
```

用户要求“我同意 我们要解决问题 不用假的数据 不用未来函数 建模不做成通俗化建模 注意各种数学建模的约束 开始做”，后续要求“能继续做吗 继续做呗 给过你链路 记得我要高分啊”。本轮据此恢复已交付快照，补做两种电价下 S2 和 S3 的全年

内点法回放，独立复算其物理与费用指标；补充月度、尾部、预测误差及块长度诊断，修订论文和完整源码附录。

提供的两份自查 DOCX 及速成 PDF 仅作为参考材料。未采用其中示例数值、固定评分权重和获奖保证；适用的修改方向包括有数据支持的检验、文献对应和完整代码。学术文献由作者或出版社公开页面核对，文稿不声称这些既有方法由本队原创。

人工核验状态

现有可核实人工行为为用户提出约束、审阅对话结论并授权修订和仓库同步。核心代码修订、实验组织和本轮正文补充由 AI 实际参与完成，未降格描述为仅改错别字。尚无证据表明参赛队员已经逐项独立复核模型推导、程序和结果，因此这些项登记为待核验；不能由 AI 自检代替人工主导，也不能以本说明冒充团队完成签核。此前会话完整提示及采用修改记录存在缺口，参赛队需据真实记录补齐。

'''

return text,ai

scripts/run_model.py

SHA256 d18883b0bce2bcd10a73e1126443f420fca7b1d31247a3cb9ccaaab0b741b30e

#!/usr/bin/env python3

from src.model.cli import main

if __name__ == "__main__":

raise SystemExit(main())

scripts/strengthen_evidence.py

SHA256 7b64dcd4a9fc3bd8b342883f2739275add73444bd1f00846f968811397885d33

"""Prespecified diagnostic replays; never select the submitted policy on test data."""

from pathlib import Path

import sys, json, hashlib, time

import numpy as np

import pandas as pd

ROOT=Path(__file__).resolve().parents[1]

sys.path.insert(0,str(ROOT))

from src.model.config import load_config

from src.model.data import load_data

from src.model.rolling import RunSpec, simulate

F=ROOT/'outputs/c_final_v1/frozen'

R=F.parent/'research'

R.mkdir(exist_ok=True)

def main():

cfg=load_config('config/final.yaml')

selected=json.loads((F/'resolved_config.json').read_text())

```

for k in ('study','storage','forecast','rolling','cost','experiments'):cfg[k]=selected[k]
data=load_data(cfg)
protocol={'role':'post_selection_diagnostic; no policy replacement',
'methods':['highs (existing frozen run)','highs-ipm'],
'modes':['q3','q4_3'],'schedules':['S2','S3'],
'period':['2025-02-01','2025-12-31'],'initial_soc':6000,
'common_plan':'existing S0 reference plan, fixed across all methods and schedules',
'selection':'January-selected configuration is frozen; all four IPM runs reported',
'recorded_at':time.strftime('%Y-%m-%dT%H:%M:%S%z'),
'source_hashes':{str(p.relative_to(ROOT)).hashlib.sha256(p.read_bytes()).hexdigest() for
p in (ROOT/'src/model').glob('*.py')}}
protocol_path=R/'protocol.json'
if not
protocol_path.exists():protocol_path.write_text(json.dumps(protocol,ensure_ascii=False,inde
nt=2))
summaries=[];checks=[]
for mode in ['q3','q4_3']:
b=pd.read_csv(F/f{mode}_S0_dispatch.csv')
plans={day:z.plan_00_grid_kwh.to_numpy() for day,z in b.groupby('date',sort=False)}
for sc in ['S2','S3']:
existing=pd.read_csv(F/f{mode}_{sc}_daily.csv')

summaries.append({'mode':mode,'scenario':sc,'method':'highs','cost':float(existing.final_relati
ve_cost_yuan.sum()),'emergency_kwh':float(existing.emergency_kwh.sum())})
path=R/f{mode}_{sc}_ipm.csv.xz'
if not path.exists():
print('START',mode,sc,'full-year IPM',flush=True)
result=simulate(data,cfg,RunSpec(mode,sc,method='highs-
ipm',collect_detail=False),initial_soc=6000,common_plans=plans)
d=pd.DataFrame(result['dispatch'])

cols=['date','position','final_adjusted_grid_kwh','charge_kwh','discharge_kwh','soc_start_kwh',
'soc_end_kwh','emergency_kwh','final_relative_cost_yuan']
d[cols].to_csv(path,index=False,compression={'method':'xz','preset':9})
pd.DataFrame(result['daily']).to_csv(R/f{mode}_{sc}_ipm_daily.csv',index=False)
pd.DataFrame(result['solve_stats']).to_csv(R/f{mode}_{sc}
_ipm_solvers.csv',index=False)
d=pd.read_csv(path)
# Independently reconstruct physical constraints and cost from raw arrays.
ids=np.array([data.date_index[___import__('datetime').date.fromisoformat(x)] for x in
d.date])
pos=d.position.to_numpy()-1
L=data.load_kw[ids,pos]/6;V=data.pv_kw[ids,pos]/6
price=data.realtime_price[ids,pos] if mode.startswith('q4') else data.fixed_price[pos]
B=np.concatenate(list(plans.values()));A=d.final_adjusted_grid_kwh.to_numpy()

c=d.charge_kwh.to_numpy();dd=d.discharge_kwh.to_numpy();s=d.soc_start_kwh.to_numpy(
);e=d.soc_end_kwh.to_numpy()
q=np.maximum(L+c-V-dd-A,0)
cost=price*(B+(A-B)+.5*np.abs(A-B)+5*q)

```

```

        metrics={'rows':len(d),'cost_max_error':float(abs(cost-
d.final_relative_cost_yuan).max()),'emergency_max_error':float(abs(q-
d.emergency_kwh).max()),
        'soc_residual':float(abs(e-s-.9*c+dd/.9).max()),'continuity_residual':float(abs(s[1:]-
e[:-1]).max()),
        'soc_min':float(min(s.min(),e.min())), 'soc_max':float(max(s.max(),e.max())),

'power_max':float(max(c.max(),dd.max())), 'simultaneous_max':float(np.minimum(c,dd).max(
)),
        'start_soc':float(s[0]), 'end_soc':float(e[-1])}
        ok=(len(d)==48096 and all(metrics[k]<1e-7 for k in
['cost_max_error','emergency_max_error','soc_residual','continuity_residual','simultaneous_m
ax']))
        and metrics['soc_min']>=1200-1e-7 and metrics['soc_max']<=10800+1e-7 and
metrics['power_max']<=5000/6+1e-7
        and abs(s[0]-6000)<1e-7 and abs(e[-1]-6000)<1e-7
        assert ok,metrics
        checks.append({'mode':mode,'scenario':sc,'status':'PASS',**metrics})
        summaries.append({'mode':mode,'scenario':sc,'method':'highs-
ipm','cost':float(cost.sum()),'emergency_kwh':float(q.sum())})
        print('DONE',mode,sc,'cost',cost.sum(),flush=True)
        pd.DataFrame(summaries).to_csv(R'/annual_solver_comparison.csv',index=False)
        (R'/annual_independent_checks.json').write_text(json.dumps(checks,indent=2))
        descriptive()

```

```
def descriptive():
```

```

    monthly=[];risks=[];calibration=[];bootstrap=[]
    for mode in ['q2','q4_2','q3','q4_3']:
        d=pd.read_csv(F/f'{mode}_dispatch.csv')
        for month,z in d.groupby(d.date.str[:7]):

```

```

        monthly.append({'mode':mode,'month':month,'cost':z.final_relative_cost_yuan.sum(),'emerge
ncy_cost':z.emergency_cost_yuan.sum(),'emergency_kwh':z.emergency_kwh.sum(),
        'unused_kwh':z.unused_surplus_kwh.sum(),'throughput_kwh':
(z.charge_kwh+z.discharge_kwh).sum(),'low_soc_fraction':float((z.soc_start_kwh<1200.001)
.mean())})

```

```

daily=d.groupby('date').agg(cost=('final_relative_cost_yuan','sum'),q=('emergency_kwh','sum'
),ecost=('emergency_cost_yuan','sum'))
costs=daily.cost.to_numpy();n_tail=max(1,int(np.ceil(.05*len(costs))))

```

```

risks.append({'mode':mode,'emergency_interval_fraction':float((d.emergency_kwh>1e-7).me
an()),'emergency_days':int((daily.q>1e-7).sum()),'max_daily_cost':float(costs.max()),'p95_dai
ly_cost':float(np.quantile(costs,.95)),
        'tail5_mean_cost':float(np.sort(costs)[-
n_tail:].mean()),'emergency_cost_share':float(d.emergency_cost_yuan.sum()/
d.final_relative_cost_yuan.sum())})
        for var,a,b in [('load','forecast_load_kwh','actual_load_kwh'),
('pv','forecast_pv_kwh','actual_pv_kwh'),('price','forecast_price','settlement_price')]:

```

```

err=d[a]-
d[b];calibration.append({'mode':mode,'variable':var,'mae':float(abs(err).mean()),'rmse':float(n
p.sqrt((err**2).mean())),'bias':float(err.mean()),'scope':'executed-interval scenario mean;
mixed available decision times'})
rng=np.random.default_rng(20260913)
for mode in ['q3','q4_3']:
    s0=pd.read_csv(F/f'{mode}_S0_daily.csv').final_relative_cost_yuan.to_numpy()
    s2=pd.read_csv(F/f'{mode}_S2_daily.csv').final_relative_cost_yuan.to_numpy()
    s3=pd.read_csv(F/f'{mode}_S3_daily.csv').final_relative_cost_yuan.to_numpy()
    for name,delta in [('S3-S0',s3-s0),('S3-S2',s3-s2)]:
        for block in [3,7,14,28]:
            starts=rng.integers(0,len(delta)-block+1,size=(2000,(len(delta)+block-1)//block))
            ix=(starts[:,None]+np.arange(block)).reshape(2000,-1)[:,:len(delta)]
            sampled=delta[ix].sum(axis=1)

bootstrap.append({'mode':mode,'comparison':name,'block_days':block,'replicates':2000,'differ
ence':float(delta.sum()),'ci_low':float(np.quantile(sampled,.025)), 'ci_high':float(np.quantile(sa
mpled,.975))})
    for name,rows in [('monthly_mechanisms',monthly),('tail_risk',risks),
('prediction_error',calibration),('block_sensitivity',bootstrap)]:pd.DataFrame(rows).to_csv(R/
f'{name}.csv',index=False)
import lzma,shutil
for p in R.glob('*_ipm.csv.xz'):
    with lzma.open(p,'rb') as src,(F/('research_'+p.name.removesuffix('.csv.xz')
+'_dispatch.csv')).open('wb') as dst:shutil.copyfileobj(src,dst)
print('All diagnostic outputs completed',flush=True)

if __name__=='__main__':main()

```

scripts/unpack_frozen.py

```

SHA256 5f709e3aa4097da14c354d5ef1d8f82226c52d41d2f7633ab7ff53b198d3ce29
from pathlib import Path
import lzma,shutil, csv,json
ROOT=Path(__file__).resolve().parents[1]
cfg=json.loads((ROOT/'config/final.yaml').read_text())
FINAL=ROOT/cfg['output_root'];F=FINAL/'frozen';F.mkdir(parents=True,exist_ok=True)
SOURCE=ROOT/'frozen'
if not SOURCE.exists():SOURCE=F
if (SOURCE/'csv_manifest.json').exists():
    from freeze_codec import unpack
    unpack(SOURCE,F)
for p in SOURCE.iterdir():
    if p.name.endswith('.csv.xz'):
        with lzma.open(p,'rb') as src,(F/p.with_suffix('').name).open('wb') as
dst:shutil.copyfileobj(src,dst)
    elif p.is_file() and p.parent!=F and p.name not in
['csv_manifest.json','columns.tar.xz']:shutil.copy2(p,F/p.name)
for folder in ['workbooks','evidence','charts','research']:
    if (ROOT/folder).exists():shutil.copytree(ROOT/folder,FINAL/folder,dirs_exist_ok=True)

```

```

for mode in ['q3','q4_3']:
    shutil.copy2(F/f {mode}_dispatch.csv',F/f {mode}_S3_dispatch.csv')
    with (F/'decision_versions.csv').open(encoding='utf-8-sig',newline='') as src,(F/f {mode}_S3_decisions.csv').open('w',encoding='utf-8-sig',newline='') as dst:

r=csv.DictReader(src);w=csv.DictWriter(dst,fieldnames=r.fieldnames);w.writeheader();w.wri
terows(row for row in r if row['mode']==mode)
    shutil.copy2(F/f {mode}_S3_decisions.csv',F/f {mode}_decision_versions.csv')
print('Frozen CSVs restored')
for p in F.glob('research_*_dispatch.csv'):
    name=p.name.removeprefix('research_').removesuffix('_dispatch.csv')+'.csv.xz'
    with p.open('rb') as src,lzma.open(FINAL/'research'/name,'wb') as
dst:shutil.copyfileobj(src,dst)

```

scripts/write_improvement_report.py

```

SHA256 18201ad61edd6c3d0d5064c58762b46e130c266d76e2618d694eab6a389ce618
from pathlib import Path
import json,hashlib
import pandas as pd
ROOT=Path(__file__).resolve().parents[1];F=ROOT/'outputs/c_final_v1';R=F/'research'

def main():
    v=json.loads((F/'final_validation_v3.json').read_text())
    a=pd.read_csv(R/'annual_solver_comparison.csv');b=pd.read_csv(R/'block_sensitivity.csv')
    text="## V3 研究论证与提交格式修订报告

```

本次保留原正式模型及五份工作簿，补充独立诊断并修订论文。新增实验是在已查看正式结果后明确登记的诊断，不作为新的盲测或调参集；不因替代求解器的费用更低而替换正式结果。

实际新增工作

1. 两种电价下 S2、S3 各完成一次连续 334 天内点法回放，合计 192384 个十分钟执行时段。共同日前合同、初始 SOC 及模型参数固定；新增每段动作从原始附件独立复核供需、费用、SOC、功率和端点。
2. 计算四种模式的月度分解、预测误差、紧急时段比例、最高日费和上尾 17 日均费；以 3/7/14/28 天块长度重复检验 S3 相对 S0 和 S2 的费用差。
3. 论文补充方法来源、储能互斥变换的证明、信息价值与实现费用的区别；加入真实出版物引文及结果解释。
4. 摘要与正文分页，图表连续编号，源码逐文件完整收入附录，并检查其每一行与实际程序一致。代码原文件仍以支撑材料内版本运行。

全年替代求解器费用

模式	方法	S2/元	S3/元	S3 减 S2/元
----	----	------	------	-----------

```
|---|---|---:|---:|---:|
'''
```

```
    for (mode,method),g in a.groupby(['mode','method'],sort=False):
        x=g.set_index('scenario').cost;text+=f'|{mode}|{method}|{x.S2:,.6f}|{x.S3:,.6f}|{x.S3-
x.S2:,.6f}|\n'
    text+=""
```

两种求解方法下 S2 均略低于 S3，但费用差的大小发生变化；原正式轨迹的 S3 减 S2 在全部四个块长度下区间均跨零。故保留日内更新相对 S0 的样本证据，不把 18 点更新或 S2/S3 的微小排序写成稳定的一般结论。

复核与状态

```
'''
```

```
    text+=f'文档及交付检查状态：{v["status"]}；共{len(v["checks"])}项。论文 PDF 共
{v["metrics"]["pdf_pages"]}页，含完整源码附录；正文页数与摘要分页另有专门检查。
数值复核记录见 audit/independent_validation.json，补充实验复核见 research/
annual_independent_checks.json。\\n'
    text+=""
```

新增参考书为 75 页扫描 PDF，本轮实际查看目录与优化、检验、评价和附录相关页；采用其要求给出实证检验和完整来源的方向，不采用示例数据、保证获奖和缺乏依据的固定格式阈值。原有参考书未复制到交付包。

仍需真实人工完成的事项

本轮不承诺分数或奖项。参赛队员须独立核对题意、推导、代码、数值与 AI 采用记录，按真实贡献确认人工主导及符合本年度参赛规则；当前没有证据证明该人工核验已完成。AI 使用详情如实说明 AI 参与核心修订。单年历史场景、保守一月启动和交易时刻电价解释仍是模型限制。格式检查通过不能覆盖参赛资格、队员主导、匿名及其他纪律要求。

本轮所有路径调整仅为可移植配置，未修改原始观测；正式结果与上次交付的差异核对见交付清单。支撑包中的完整源码与压缩轨迹可恢复复算。

```
'''
```

```
    (F/'evidence/本轮研究与交付修订报告.md').write_text(text)
    print('Improvement report written')
```

```
if __name__=='__main__':main()
```

scripts/write_repair_report.py

```
SHA256 c3be81844147eab5040e50396e82cc7ac151b960e444e4f578352e9aef0e72a6
""""Current-run repair report, with computed effects and explicit limits.""""
from pathlib import Path
from datetime import datetime
import json,pandas as pd
```

```

ROOT=Path(__file__).resolve().parents[1];O=ROOT/'outputs/c_final_v1';F=O/'frozen'

def main():

audit=json.loads((O/'audit/independent_validation.json').read_text());quality=json.loads((O/'fi
nal_validation_v3.json').read_text())

exp=pd.read_csv(F/'experiment_summary.csv');official=pd.read_csv(F/'official_summary.csv'
);settle=pd.read_csv(F/'settlement_sensitivity.csv');jan=pd.read_csv(F/'startup_summary.csv');
q1=pd.read_csv(F/'q1_dispatch.csv')
    def get(mode,sc='S0',arm='joint_empirical'):
        return exp[(exp['mode']==mode)&(exp.scenario==sc)&(exp.model_arm==arm)].iloc[0]
    def f(x,n=2):return f'{x:.{n}f}'

old={'q2':17646794.819761,'q3':16662138.664212,'q4_2':18660362.332063,'q4_3':17539062.
517961}
    rows='\n'.join(f' {r["mode"]} | {f(old[r["mode"]])} | {f(r.final_relative_cost_yuan)} |
{f(r.final_relative_cost_yuan-old[r["mode"]])} |' for _,r in official.iterrows())
    risk='\n'.join(f' {mode} | {f(get(mode,arm="mean_forecast").final_relative_cost_yuan)} |
{f(get(mode).final_relative_cost_yuan)} |
{f(get(mode,arm="mean_forecast").final_relative_cost_yuan-
get(mode).final_relative_cost_yuan)} | {f(get(mode).emergency_kwh,4)} |' for mode in
['q2','q4_2'])
    ledger='\n'.join(f' {r["mode"]} | {f(r.main_cost)} | {f(r.same_chain_sequential_cost)} |
{f(r.same_chain_sequential_cost-r.main_cost)} | {f(r.no_refund_same_path_cost)} |' for _,r in
settle.iterrows() if r['mode'] in ['q3','q4_3'])
    schedule='\n'.join(f' {mode.upper()} | {f(get(mode).final_relative_cost_yuan)} |
{f(get(mode,"S3").final_relative_cost_yuan)} | {f(get(mode,"S3").final_relative_cost_yuan-
get(mode).final_relative_cost_yuan)} | {f(get(mode,"S3").final_relative_cost_yuan-
get(mode,"S2").final_relative_cost_yuan)} |' for mode in ['q3','q4_3'])
    times='\n'.join(f' {h}:00-{h}:10 | {int(h*6+1)} |
{f(q1.loc[q1.position==h*6+1,"plan_00_grid_kwh"].iloc[0],6)} |' for h in
[10,12,14,16,18,20])
    text=f""# V3 闭环修复与独立复验报告

```

生成时间：{datetime.now().astimezone().isoformat()}。对象：桌面“最新版本V3”。用户本轮已明确授权实际修复，要求保留真实数据、禁止未来信息泄漏并遵守数学约束。此前只读审查报告描述修复前快照；本报告使用新的正式运行。

当前结论

本轮已实际重写风险规划、滚动承诺和启动逻辑，重跑一月选型及二月至十二月正式实验，重新导出论文和五份结果工作簿。原始附件未修改。独立数值及工作簿检查{len(audit['checks'])}项，状态{audit['status']}；文档与导出检查{len(quality['checks'])}项，状态{quality['status']}。这些检查证明下述实现、约束和证据一致性，不证明模型在所有解释和所有未来数据下最优。

没有用旧报告数值充当本轮结果，没有把历史预测场景伪装成实际观测，也没有仅修改 PASS 字段。旧流水线被停止，修订前完整备份保留在本次 Codex 任务的 work/before_repair；项目自身旧输出保留在 work/pre_repair_output。

1. 已修复事项与数值影响

五倍缺口成本已进入日前规划

`src/model/risk.py:cases/solve`使用完整观测的历史联合轨迹构造经验预测分布。所有采购、充放电和状态在场景之间共享；仅紧急采购随场景变化，目标直接含五倍价缺口损失。没有提前给优化器未来实际负载、光伏或电价。残差的基准预测也在对应历史日只使用更早日期生成。

`experiments.py`只用 1 月 8—31 日比较两种场景构造与 0.5/1/1.5 终端惩罚倍数，锁定因果残差场景、倍数 1.0。已解析配置在`frozen/resolved\_config.json`；候选默认值不能代替实际解析配置。正式期 334 天不参与参数选择。

相同日期与状态合同下，均值预测控制与新风险策略比较如下，差额是本数据实现结果：

模式	均值预测费/元	风险策略费/元	费用降低/元	风险策略紧急量/kWh
---	---	---	---	---
{risk}				

无储能控制费用分别为 $\{f(\text{get}('q2', \text{arm}='no_storage').\text{final_relative_cost_yuan})\}$ 、 $\{f(\text{get}('q4_2', \text{arm}='no_storage').\text{final_relative_cost_yuan})\}$ 元。紧急购电是题面允许的缺口补足机制，不能把该量称为最终停电量。有限历史分布仍可能漏掉极端风险。

逐次结算已改为同一真实承诺链

`rolling.py:simulate`记录每次正式更新的旧承诺、新承诺、交付位置和发布时间。后续预览不记为正式订单；已交付段不允许重写。现有同链逐次结果如下：

模式	净额费用/元	同链逐次费用/元	逐次减净额/元	不退原价同路径/元
---	---	---	---	---
{ledger}				

独立审计从版本记录重新累积每一笔调整，并单独从原始曲线重算同一执行轨迹的紧急费。旧版以另一条确定性路径计算调整费、再拼正式路径紧急费的做法已删除。相同非负交付价下，`逐次-净额 = 0.5Σp(Σ|ΔA|-|ΣΔA|) ≥ 0`；100→80→100、价格 1 元的例子，逐次较净额多 20 元。退费和交付价是明示解释；其他定价解释须重新定义模型。

十分钟错位和 Q2—Q4 缺表已修复

以右端点数据代表前十分钟区间为统一离散化口径。论文按`physical\_interval`抽取，官方模板按原列序写入；新增“时段映射”页保留原标签与物理键的区别。Q1 费用仍为 $\{f(q1.final_relative_cost_yuan.sum(),6)\}$ 元；不以目标相同掩盖时段错误。

题面物理区间	正确位置	当前计划量/kWh（六位展示）
{times}

例如 12:00 应取位置 73、原模板 B74，旧论文误取位置 72。六个时段已在 DOCX、PDF、工作簿中核对。Q2/Q3/Q4-2/Q4-3 四类×四指定日期均包括六个十分钟值、六组四小时充放电、0/24 点 SOC、全天量/费用和连续紧急区间。含 Q1 共 17 组；文档存在的占位符数量为 0。修复前本次审查也已查明“剩 80 个占位”的历史说法不适用于当时快照，因此不把这一旧数字算作本轮成果。

S0/S3 比较已固定共同日前承诺

先用因果的 S0 参考过程形成各日 B。四种频次使用同一 B、相同一月启动和年末状态；滚动差异来自后续允许信息更新。全部计划动作在场景中共享完整窗口，避免旧版午夜分支共享范围不一致。

模式	S0 费用/元	S3 费用/元	S3 减 S0/元	S3 减 S2/元
{schedule}

本次 S2 略低于 S3，S3 仍作为题设四时点机制的正式结果，而非被称为全局最优。32 个相同 LP 实例、3 种求解法共 96 次诊断中，目标最大差约 7.28×10^{-12} 元，但采购轨迹 L1 最大差 292.46 kWh、SOC 轨迹最大差 2577.19 kWh。存在并列解敏感性；没有据此宣称替代求解器的全年策略排序已经稳定。逐日配对 7 天块重采样区间已列论文，重采样不被当作新观测。

一月 SOC 衔接与无历史启动已重建

删除“首日无历史时用当天完整实际曲线”的未来信息兜底。采用明示、可行的一月启动合同：电池待机 $C=D=0$ ；首日计划为 0，实际缺口按紧急合同支付；以后只用已经完成的历史日作采购分位数决策。逐段记录 4464 个时段，因此一月末 6000 是状态递推的结果，二月初值从账本读取，不是空填 6000。Q2/Q3 一月启动费各 $\{f(jan[jan['mode']=='q2'].cost_yuan.iloc[0])\}$ 元，Q4-2/Q4-3 各 $\{f(jan[jan['mode']=='q4_2'].cost_yuan.iloc[0])\}$ 元，均披露在论文。

这是一项保守的启动策略，不声称一月最优。离线选型回放与真实启动合同区分记录。普通分段结束不强行回到 6000；仅明确评价末端（训练子实验或 12 月 31 日）施加终端条件。首日、跨日、分段恢复、不同充放电效率和年末边界已测试。

2. 与修复前已交付版本的差异

模式	修复前已交付费/元	当前正式费/元	当前减旧版/元
---	---	---	---

{rows}

旧版数值来自本轮修复前的独立复算，不是历史报告直接抄录的结论。修复前后台另有 Q3 从 1200 起步的未同步结果 16,664,426.073252 元；它与当时 6000 起步的交付稿不一致。本次所有分支已统一启动合同。表中总差额同时包含风险模型、预测、合同与边界改变，不能全部归因为一个修复点；模型改进效果优先看第 1 节受控比较。

3. 数学约束与信息隔离证据

- 真实负载、光伏、价格直接从附件 1、2、4 读取重算，逐时段匹配；源功率转电量除以 6。
- 独立重算供需、紧急量、未用富余、费用和 $E_{next}=E+0.9C-D/0.9$ ；检查 SOC 在 [1200,10800]、每段充放电 $\leq 5000/6$ 、不同时充放电、前后状态连续。
- Q1 另构建一套 LP 约束矩阵验证最优目标，未调用原模型费用函数。
- 未来观测扰动保持决策已知信息不变；改变未发生的负载、光伏、电价后，场景和当前决策不变。更晚发布的预报不能影响 0 点结果。测试范围见 `tests/test\_closed\_loop.py`，共 7 个测试。
- 对全部 8 条 S0—S3 承诺链独立回放，核对旧值连续、没有改写过去、最终值等于正式执行及同路径费用不等式。
- 五份工作簿检查原始列序、逐时段量、日期、日总量/费用、四小时汇总、端点 SOC 及全部紧急区间，不只查文件存在。
- DOCX/PDF 指定表逐值与全精度轨迹核对；PDF 中的表格数字文字与 DOCX 一致，公式、图及各页进行了渲染检查。

4. 可复算证据链与运行

证据环节	当前文件/入口
---	---
原题与原附件	原题 C 题.pdf、附件 1—5；evidence/source_provenance.json 保存原始 SHA256
信息与模型	src/model/risk.py、forecast.py、rolling.py、experiments.py
参数来源	evidence/january_parameter_validation.csv、frozen/resolved_config.json
执行与合同	frozen 中正式/控制/各频次逐时段轨迹、decision_versions、各 S0—S3 账本

| 独立复算 | src/audit/audit_final.py、audit/independent_validation.json |
| 表格及文稿 | 五份 result 工作簿、paper/main.md、scripts/paper_content.py、
native_equations.py |
| 最终检查 | final_validation_v3.json、SHA256SUMS.json；哈希清单排除自身 |

工程完整运行：在支撑材料目录执行`./run\_pipeline.sh`。可通过
CODEX_PYTHON_BIN、CODEX_NODE_BIN 指定解释器；原附件根路径配置在
config/final.yaml 或 CUMCM_SOURCE_ROOT。Python 依赖见 requirements.txt；表
格/文档构建还使用已安装的 Codex 工作区运行时。

压缩交付使用列去重与 XZ 无损归档，全精度 CSV 文本不舍入。运行`scripts/
unpack_frozen.py`后恢复到配置指定的 outputs/c_final_v1/frozen，并逐文件校验原始
SHA256；再运行`src/audit/audit\_final.py --workbooks`即可独立复验。压缩仅为满足支
撑材料体积约束，不丢弃不利结果或实验分支。

5. 仍需讨论的模型取舍

已确认的错位、混链、未来信息冷启动、对照不一致及文档缺表已在本轮修复。剩余
的是需要明确承认的模型边界：

1. 退费和价格时点存在题意解释空间；论文已列同路径敏感性，不能冒充官方唯一解
释。
2. 一月待机启动虽严格可行，经济上保守；若进一步优化一月，应使用可用信息重新
运行全年，不能仅把启动末 SOC 改成更有利数值。
3. 20 条历史经验场景、全窗口共同动作及有限期终端惩罚可能偏保守，不能保证未见
极端风险或全局最优。
4. S2 与 S3 的差额很小，若要声称更新频次排序稳健，修复顺序应是先增加有明确二级
经济含义的选解准则，再做完整年度替代求解器回放；目前只作名义样本比较。

下一轮如继续提升，优先讨论上述结算解释与一月启动目标，其次才是提高预测/风险
分布质量和全年排序稳定性。禁止通过挑正式期参数、选择性省略成本或改写实际数
据获得更好结果。

'''

```
(O/'evidence/闭环修复与独立复验报告.md').write_text(text)
print('Repair report written')
```

```
if __name__ == '__main__':main()
```

src/audit/_init_.py

```
SHA256 0e5daf34a6792487bce8e142d9f81487b6c2edb4b3ceba6019ce985674182432
""""Independent audit gate for the C-problem delivery."""
```

src/audit/audit_final.py

```
SHA256 6318fce7aa9336723fb22e8fc194df9b26e970a10c02f0b9e71e3c42b5cf4367
"""Independent validation from original workbooks and public dispatch/commitment data.
Does not import the model, scenario generator or model settlement functions.
"""

from pathlib import Path
from datetime import datetime
import json, csv, argparse, hashlib, os
import numpy as np
import pandas as pd
import openpyxl
from scipy.optimize import linprog
ROOT=Path(__file__).resolve().parents[2]

def run(workbooks=False):

    cfg=json.loads((ROOT/'config/final.yaml').read_text());F=ROOT/cfg['output_root']/'frozen';O
    =F.parent/'audit';O.mkdir(parents=True,exist_ok=True);checks=[];metrics={}
        def check(name,ok,actual):checks.append(dict(check=name,status='PASS' if bool(ok) else
'FAIL',actual=actual))
            P=Path(os.environ.get('CUMCM_SOURCE_ROOT',cfg['source_root']))/'C 题/附件'
            def matrix(path,sheet):

                w=openpyxl.load_workbook(path,read_only=True,data_only=True);rows=list(w[sheet].value
s);w.close();return np.array(rows[1:],object)[:,1:145].astype(float)
                    w=openpyxl.load_workbook(P/'附件
1.xlsx',read_only=True,data_only=True);a=np.array(list(w.active.values
[1:],object);w.close();price=a[:,1].astype(float);qL=a[:,2].astype(float)/
6;qV=a[:,3].astype(float)/6
                        L=matrix(P/'附件 2.xlsx','小区负载')/6;V=matrix(P/'附件 2.xlsx','光伏发电实际功率')/
6;price_rt=matrix(P/'附件 4.xlsx','Sheet1');models={};tol=1e-7
                            for path in sorted(F.glob('*dispatch.csv')):
                                if 'january' in path.name or path.name.startswith('research_'):continue
                                    d=pd.read_csv(path);name=path.stem;mode='q1' if name.startswith('q1') else ('q4_3' if
name.startswith('q4_3') else 'q4_2' if name.startswith('q4_2') else name[:2]);models[name]=d
                                        if mode=='q1':l,v,p=qL,qV,price
                                            else:
                                                i=(pd.to_datetime(d.date)-pd.Timestamp('2025-01-
01')).dt.days.to_numpy();j=d.position.to_numpy()-1;l=L[i,j];v=V[i,j];p=price_rt[i,j] if
mode.startswith('q4') else price[j]

                                c=d.charge_kwh.to_numpy();dis=d.discharge_kwh.to_numpy();g=d.final_adjusted_grid_kwh.
to_numpy();b=d.plan_00_grid_kwh.to_numpy();s0=d.soc_start_kwh.to_numpy();s1=d.soc_e
nd_kwh.to_numpy();q=np.maximum(l+c-v-dis-g,0);u=np.maximum(g+v+dis-l-c,0);delta=g-b
                                    cost=p*b+p*(delta+.5*np.abs(delta))+5*p*q
                                        maxerr=lambda x:float(np.max(np.abs(x)))
```

```

        check(name+'.count',len(d)==(144 if mode=='q1' else 48096) and not
d.duplicated(['date','position']).any() and all(len(z)==144 and
set(z.position)==set(range(1,145)) for _,z in d.groupby('date')),len(d))
        check(name+'.raw_load_pv_price',max(maxerr(l-d.actual_load_kwh),maxerr(v-
d.actual_pv_kwh),maxerr(p-d.settlement_price))<tol,dict(load=maxerr(l-
d.actual_load_kwh),pv=maxerr(v-d.actual_pv_kwh),price=maxerr(p-d.settlement_price)))
        labels=[f'{(int(x)-1)*10//60:02d}':{(int(x)-1)*10%60:02d}-{int(x)*10//60:02d}':
{int(x)*10%60:02d}' for x in d.position]
        check(name+'.physical_interval',labels==d.physical_interval.tolist(),sum(x!=y for x,y in
zip(labels,d.physical_interval)))
        sr=maxerr(s1-s0-.9*c+dis/.9);continuity=maxerr(s0[1:]-s1[:-1]);check(name+'.soc',sr<tol
and continuity<tol,dict(recursion=sr,continuity=continuity))
        check(name+'.device_constraints',min(s0.min(),s1.min())>=1200-tol and
max(s0.max(),s1.max())<=10800+tol and c.min()>=-tol and dis.min()>=-tol and
max(c.max(),dis.max())<=5000/6+tol and not
np.any((c>tol)&(dis>tol)),dict(min_soc=float(min(s0.min(),s1.min())),max_soc=float(max(s0.
max(),s1.max())),max_charge=float(c.max()),max_discharge=float(dis.max())))
        check(name+'.realized_balance',max(maxerr(q-d.emergency_kwh),maxerr(u-
d.unused_surplus_kwh))<tol,maxerr(q-d.emergency_kwh))
        check(name+'.main_cost',maxerr(cost-d.final_relative_cost_yuan)<tol,maxerr(cost-
d.final_relative_cost_yuan))
        check(name+'.same_path_inequality',float((d.sequential_cost_yuan-
d.final_relative_cost_yuan).min())>=-tol,float((d.sequential_cost_yuan-
d.final_relative_cost_yuan).min()))
        check(name+'.end_soc',abs(s1[-1]-6000)<tol,float(s1[-1]))
        if mode!='q1':
            jan=pd.read_csv(F/f'{mode}
_january_dispatch.csv');check(name+'.January_link',abs(s0[0]-
jan.soc_end_kwh.iloc[-1])<tol,dict(january_end=float(jan.soc_end_kwh.iloc[-1]),february_sta
rt=float(s0[0])))

metrics[name]=dict(total_cost=float(cost.sum()),plan_cost=float((p*b).sum()),adjustment_cos
t=float((p*(delta+.5*abs(delta))).sum()),emergency_cost=float((5*p*q).sum()),emergency_k
wh=float(q.sum()),emergency_intervals=int((q>=.00005).sum()),max_emergency_kwh=float(
q.max()))
        summary=pd.read_csv(F/'experiment_summary.csv')
        for r in summary.itertuples():
            key=(r.mode+'_dispatch') if r.mode in ['q2','q4_2'] and r.model_arm=='joint_empirical'
            else (r.mode+'_'+r.model_arm+'_dispatch' if r.mode in ['q2','q4_2'] else
            r.mode+'_'+r.scenario+'_dispatch')
            m=metrics[key];err=max(abs(m['total_cost']-
r.final_relative_cost_yuan),abs(m['emergency_kwh']-r.emergency_kwh),abs(m['plan_cost']-
r.plan_cost_yuan),abs(m['adjustment_cost']-
r.adjustment_cost_yuan),abs(m['emergency_cost']-r.emergency_cost_yuan))
            check(key+'.summary_reconciliation',err<1e-6,err)
            # Reconstruct every saved schedule's formal chain, including repeated revisions.
            ledgers=[]
            for mode in ['q3','q4_3']:
                base=models[f'{mode}_S0_dispatch'].plan_00_grid_kwh.to_numpy()
                for sc in ['S0','S1','S2','S3']:

```

```

d=models[f'{mode}_{sc}_dispatch'];check(f'{mode}
{sc}.common_plan',np.max(np.abs(base-
d.plan_00_grid_kwh))<tol,float(np.max(np.abs(base-d.plan_00_grid_kwh))))
dd=d.set_index(['date','position']);v=pd.read_csv(F/f'{mode}_{sc}
_decisions.csv',low_memory=False);chain={};fees={};bad=0;past=0
prices={(r.date,int(r.position)):float(r.settlement_price) for r in d.itertuples()}
for r in v.itertuples():
    if r.is_preview_next_day:continue
    k=(r.target_date,int(r.position));new=float(r.new_commitment_kwh)
    delivery=pd.Timestamp(r.target_date)
+pd.Timedelta(minutes=(int(r.position)-1)*10)
    past+=int(delivery<pd.Timestamp(r.decision_time))
    if k in chain:
        bad+=int(abs(chain[k]-r.old_commitment_kwh)>tol);delta=new-
chain[k];fees[k]=fees.get(k,0)+prices[k]*(delta+.5*abs(delta))
        chain[k]=new
    last=np.array([chain[k] for k in dd.index]);computed=dd.plan_cost_yuan.to_numpy()
+dd.emergency_cost_yuan.to_numpy()+np.array([fees.get(k,0) for k in
dd.index]);err=float(np.max(abs(computed-dd.sequential_cost_yuan.to_numpy()))
    check(f'{mode}_{sc}.formal_chain',bad==0 and past==0 and np.max(abs(last-
dd.final_adjusted_grid_kwh.to_numpy()))<tol and
err<tol,dict(old_commitment_errors=bad,past_rewrites=past,final_error=float(np.max(abs(las
t-dd.final_adjusted_grid_kwh.to_numpy()))),sequential_error=err))

ledgers.append(dict(mode=mode,scenario=sc,formal_chain_cost_yuan=float(computed.sum()
),main_cost_yuan=float(dd.final_relative_cost_yuan.sum()),difference_yuan=float(computed.
sum()-dd.final_relative_cost_yuan.sum()))
    for mode in ['q2','q4_2','q3','q4_3']:
        d=pd.read_csv(F/f'{mode}_january_dispatch.csv');i=(pd.to_datetime(d.date)-
pd.Timestamp('2025-01-01')).dt.days.to_numpy();j=d.position.to_numpy()-1;p=price_rt[i,j] if
mode.startswith('q4') else price[j];q=np.maximum(L[i,j]-V[i,j]-
d.plan_grid_kwh,0);cost=p*d.plan_grid_kwh+5*p*q
        history_ok=all(all(h<r.date for h in str(r.history_dates).split('|') if h and h!='nan') for r in
d.itertuples())
        check(mode+'.January_startup',len(d)==4464 and history_ok and
np.all(d.charge_kwh==0) and np.all(d.discharge_kwh==0) and
np.max(abs(d.soc_start_kwh-6000))<tol and np.max(abs(d.soc_end_kwh-6000))<tol and
np.max(abs(cost-
d.cost_yuan))<tol,dict(rows=len(d),history_before_decision=history_ok,cost_error=float(np.
max(abs(cost-d.cost_yuan))),cost=float(cost.sum()))

stats=pd.read_csv(F/'solver_decisions.csv');bad=int((pd.to_datetime(stats.latest_observation_t
ime)>pd.to_datetime(stats.decision_time)).sum());check('observation_availability',bad==0,bad)
    # Independent Q1 LP with raw input and physical units, separate construction.
    n=144;M=np.zeros((2*n,5*n));rhs=np.r_[qL-qV,np.zeros(n)];rhs[n]=6000
    for t in range(n):
        M[t,t]=1;M[t,n+t]=-1;M[t,2*n+t]=1;M[t,3*n+t]=-1;M[n+t,n+t]=-.9;M[n+t,2*n+t]=1/.9;M[n+t,
4*n+t]=1

```

```

if t:M[n+t,4*n+t-1]=-1
bounds=[(0,None)]*n+[(0,5000/6)]*(2*n)+[(0,x) for x in qV]+
[(1200,10800)]*n;bounds[-1]=(6000,6000)

opt=linprog(np.r_[price,np.zeros(4*n)],A_eq=M,b_eq=rhs,bounds=bounds,method='highs');c
heck('Q1.independent_optimum',opt.success and abs(opt.fun-metrics['q1_dispatch']
['total_cost'])<1e-6,float(opt.fun))
if workbooks:
    for mode,name in [('q1','result1'),('q2','result2'),('q3','result3'),('q4_2','result4-2'),
('q4_3','result4-3')]:
        d=models[mode+'_dispatch'];w=openpyxl.load_workbook(F.parent/f'workbooks/
{name}.xlsx',read_only=True,data_only=True)
        required={'计划购电量','充放电量','指定时段','时段映射'}
        if mode!='q1':required.add('紧急购电量')
        if mode in ['q3','q4_3']:required.add('调整购电量')
        check(name+'.required_sheets',required<=set(w.sheetnames),sorted(w.sheetnames))
        for sheet,col in [('计划购电量','plan_00_grid_kwh'),('调整购电
量','final_adjusted_grid_kwh')]:
            if sheet not in w.sheetnames:continue
            rows=list(w[sheet].values)
            vals=np.array([r[1] for r in rows[1:145]],float) if mode=='q1' else np.array([r[1:145]
for r in rows[1:335]],float).reshape(-1)
            e=float(np.max(abs(vals-d[col].to_numpy())));check(f'{name}.
{sheet}',e<=.000051,e)
            if mode!='q1':
                cost_col='plan_cost_yuan' if sheet=='计划购电量' and mode in ['q3','q4_3'] else
'final_relative_cost_yuan'
                expected=d.groupby('date',sort=False)[[col,cost_col]].sum().to_numpy()
                actual=np.array([r[145:147] for r in rows[1:335]],float)
                err=float(np.max(abs(expected-actual)))
                dates_ok=[str(r[0])[10] for r in rows[1:335]]==list(d.date.unique())
                check(f'{name}.{sheet}.daily_totals_and_dates',err<=.000051 and
dates_ok,dict(max_error=err,dates_match=dates_ok))
                # All storage blocks and SOC endpoints; all emergency interval keys.
                dates=d.date.unique();srows=list(w['充放电量'].values)[1:];errs=[]
                for di,day in enumerate(dates):
                    z=d[d.date==day]
                    for b in range(6):
                        rr=srows[di*6+b];off=1 if mode=='q1' else 2;zz=z.iloc[b*24:(b+1)*24]
                        errs.extend([abs(float(rr[off])-zz.charge_kwh.sum()),abs(float(rr[off+1])-
zz.discharge_kwh.sum())])
                        if b<2:errs.append(abs(float(rr[off+3])-(z.soc_start_kwh.iloc[0] if b==0 else
z.soc_end_kwh.iloc[-1])))
                    check(f'{name}.all_storage_blocks',max(errs)<=.000051,float(max(errs)))
                if mode!='q1':
                    expected=[]
                    for day,z in d.groupby('date'):
                        active=[]
                        def emit():

```

```

        a=(active[0].position-1)*10;b=active[-1].position*10
        expected.append((day,f {a//60}:{a%60:02d}-{b//60}:{
{b%60:02d}',sum(x.emergency_kwh for x in active)))
        for r in z.itertuples():
            if r.emergency_kwh>=.00005:active.append(r)
            elif active:emit();active=[]
            if active:emit()
        actual=[];day=None
        for r in list(w['紧急购电量'].values)[1:]:
            if r[0] is not None:day=str(r[0])[10]
            actual.append((day,r[1],float(r[2])))
        keys_ok=[x[:2] for x in actual]==[x[:2] for x in expected]
        err=max([abs(x[2]-y[2]) for x,y in zip(actual,expected)] or [0])
        check(name+'.emergency_intervals',keys_ok and
err<=.000051,dict(rows=len(actual),expected=len(expected),value_error=err))
        specified=list(w['指定时段'].values)[1:];specified_error=[]
        for rr in specified:
            day=str(rr[0])[10];zz=d[(d.date==day)&(d.physical_interval==rr[1])]
            if len(zz)!=1:specified_error.append('missing key')
            else:
                x=zz.iloc[0]
                if abs(float(rr[2])-x.plan_00_grid_kwh)>.000051 or abs(float(rr[3])-
x.final_adjusted_grid_kwh)>.000051:specified_error.append((day,rr[1]))
                check(name+'.specified_physical_values',not specified_error and len(specified)==(6 if
mode=='q1' else 24),specified_error)
        w.close()
        out=dict(status='PASS' if all(r['status']=='PASS' for r in checks) else
'FAIL',generated_at=datetime.now().astimezone().isoformat(),checks=checks,metrics=metrics
,formal_settlement=ledgers)

(O/'independent_validation.json').write_text(json.dumps(out,ensure_ascii=False,indent=2));pd
.DataFrame(ledgers).to_csv(O/'settlement_recalculation.csv',index=False)
    print(json.dumps(dict(status=out['status'],checks=len(checks),failures=[r for r in checks if
r['status']=='FAIL']),ensure_ascii=False),flush=True)
    return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--
workbooks',action='store_true');a=ap.parse_args();r=run(a.workbooks);raise SystemExit(0 if
r['status']=='PASS' else 1)

```

src/model/_init_.py

SHA256 5cb253a6bf42d59171e8d2a280901387c14f4942303ddd19a8b4b46f3c634cfe
 """"C-problem causal microgrid model and reproducible rolling experiments.""""

```

from .config import load_config
from .data import DataBundle, load_data
from .rolling import RunSpec, simulate

```

```
__all__ = ["DataBundle", "RunSpec", "load_config", "load_data", "simulate"]
```

src/model/cli.py

```
SHA256 44f9863d234037e56260c544ac3637b85f5274d451fb9afb65bb968820b6bebf
```

```
import argparse
from .config import load_config
from .data import load_data
from .experiments import run_all

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--config',default='config/
final.yaml');ap.add_argument('--
scope',choices=['all','train','official'],default='all');a=ap.parse_args();cfg=load_config(a.config)
;run_all(load_data(cfg),cfg,a.scope);return 0
```

src/model/config.py

```
SHA256 8f9800ac73bb699fa1d65cd5eea8ea4ac1d8949812fed19f930288b8ad171776
```

```
from __future__ import annotations
```

```
import json
import os
from pathlib import Path
from typing import Any
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
```

```
def load_config(path: str | Path) -> dict[str, Any]:
    """Load final.yaml.
```

```
    The file intentionally uses JSON syntax, which is a strict subset of YAML. This
    keeps the pipeline dependency-free while remaining readable by YAML tooling.
    """
```

```
    p = Path(path)
    if not p.is_absolute():
        p = PROJECT_ROOT / p
    cfg = json.loads(p.read_text(encoding="utf-8"))
    cfg["_config_path"] = str(p.resolve())
    source_value=os.environ.get("CUMCM_SOURCE_ROOT",cfg["source_root"])
    source=Path(source_value)
    if not source.is_absolute(): source=PROJECT_ROOT/source
    cfg["_source_root"]=str(source.resolve())
    out = Path(cfg["output_root"])
    if not out.is_absolute():
        out = PROJECT_ROOT / out
```

```

cfg["_output_root"] = str(out.resolve())
return cfg

```

src/model/data.py

```

SHA256 192c71b4505cf89305ddda219a898304c607445f32f0a559925ca4496ed24e6c

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from pathlib import Path
from typing import Any

```

```

import numpy as np
from openpyxl import load_workbook

```

```

def _date(value: Any) -> date:
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    return datetime.strptime(str(value).strip().replace(".", "-").replace("/", "-"), "%Y-%m-%d").date()

```

```

def _hm(value: Any) -> str:
    if isinstance(value, time):
        return f"{value.hour:02d}:{value.minute:02d}"
    return str(value).strip()

```

```

def _numeric(values: list[Any], label: str) -> np.ndarray:
    bad = [i for i, v in enumerate(values) if v is None or v == "" or isinstance(v, bool)]
    if bad:
        raise ValueError(f"{label}: blank/non-numeric cells at offsets {bad[:10]}")
    try:
        arr = np.asarray(values, dtype=float)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{label}: non-numeric cell present") from exc
    if not np.all(np.isfinite(arr)):
        raise ValueError(f"{label}: NaN/Inf present")
    return arr

```

```

@dataclass(frozen=True)
class DataBundle:
    dates: tuple[date, ...]

```

```

source_timestamps: tuple[str, ...]
official_labels: tuple[str, ...]
fixed_price: np.ndarray
q1_load_kw: np.ndarray
q1_pv_kw: np.ndarray
load_kw: np.ndarray
pv_kw: np.ndarray
realtime_price: np.ndarray
pv_forecast_kw: dict[tuple[date, int], np.ndarray]

@property
def date_index(self) -> dict[date, int]:
    return {d: i for i, d in enumerate(self.dates)}

def time_rows(self) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    for di, day in enumerate(self.dates):
        for pos in range(144):
            start = pos * 10
            end = (pos + 1) * 10
            rows.append({
                "date": day.isoformat(),
                "position": pos + 1,
                "global_position": di * 144 + pos,
                "source_timestamp": self.source_timestamps[pos],
                "physical_interval": f"{start//60:02d}:{start%60:02d}-{end//60:02d}:{end%60:02d}",
                "official_label": self.official_labels[pos],
            })
    return rows

def _read_matrix(path: Path, sheet: str) -> tuple[tuple[str, ...], tuple[date, ...], np.ndarray]:
    ws = load_workbook(path, data_only=True, read_only=True)[sheet]
    rows = ws.iter_rows(values_only=True)
    header = next(rows)
    if len(header) < 145:
        raise ValueError(f"{path.name}/{sheet}: expected 145 columns")
    headers = tuple(_hm(x) for x in header[1:145])
    dates: list[date] = []
    matrix: list[np.ndarray] = []
    for ri, row in enumerate(rows, start=2):
        if row[0] in (None, ""):
            raise ValueError(f"{path.name}/{sheet}!A{ri}: blank date")
        dates.append(_date(row[0]))
        matrix.append(_numeric(list(row[1:145]), f"{path.name}/{sheet} row {ri}"))
    out = np.vstack(matrix)
    if out.shape != (365, 144) or len(set(dates)) != 365:

```

```

        raise ValueError(f'{path.name}/{sheet}: expected 365 unique days x 144 intervals, got {out.shape}')
    return headers, tuple(dates), out

```

```

def load_data(cfg: dict[str, Any]) -> DataBundle:
    root = Path(cfg['_source_root']) / "C 题" / "附件"
    ws1 = load_workbook(root / "附件 1.xlsx", data_only=True, read_only=True).active
    rows1 = list(ws1.iter_rows(values_only=True))
    if len(rows1) != 145:
        raise ValueError("附件 1: expected header + 144 intervals")
    source_timestamps = tuple(_hm(r[0]) for r in rows1[1:])
    fixed_price = _numeric([r[1] for r in rows1[1:]], "附件 1/电价")
    q1_load = _numeric([r[2] for r in rows1[1:]], "附件 1/负载")
    q1_pv = _numeric([r[3] for r in rows1[1:]], "附件 1/光伏")

    h1, dates, load = _read_matrix(root / "附件 2.xlsx", "小区负载")
    h2, dates2, pv = _read_matrix(root / "附件 2.xlsx", "光伏发电实际功率")
    h3, dates3, rt = _read_matrix(root / "附件 4.xlsx", "Sheet1")
    if not (dates == dates2 == dates3 and h1 == h2 == h3):
        raise ValueError("附件 2/4 date or timestamp axes differ")
    expected = tuple(date(2025, 1, 1) + timedelta(days=i) for i in range(365))
    if dates != expected:
        raise ValueError("附件 2/4 dates are not exactly 2025-01-01..2025-12-31")

    template = load_workbook(root / "附件 5" / "result2.xlsx", data_only=True,
read_only=True)["计划购电量"]
    official = tuple(str(x.value) for x in next(template.iter_rows())[1:145])
    if len(official) != 144 or len(set(official)) != 144:
        raise ValueError("official result template does not contain 144 unique interval labels")

    ws3 = load_workbook(root / "附件 3.xlsx", data_only=True, read_only=True).active
    forecasts: dict[tuple[date, int], np.ndarray] = {}
    current: date | None = None
    for ri, row in enumerate(ws3.iter_rows(min_row=2, values_only=True), start=2):
        if row[0] not in (None, ""):
            current = _date(row[0])
        if current is None:
            raise ValueError(f'附件 3 row {ri}: missing carried date')
        issue_text = str(row[1]).strip()
        try:
            issue = int(issue_text.split(":", 1)[0])
        except Exception as exc:
            raise ValueError(f'附件 3 row {ri}: invalid issue time {row[1]!r}') from exc
        vals = _numeric(list(row[2:26]), f'附件 3 row {ri}')

```

```

    key = (current, issue)
    if key in forecasts:
        raise ValueError(f'附件 3 duplicate forecast {key}')
    forecasts[key] = vals
    expected_keys = {(d, h) for d in dates for h in (0, 6, 12, 18)}
    if set(forecasts) != expected_keys:
        raise ValueError(f'附件 3 forecast coverage mismatch: missing={len(expected_keys-
set(forecasts))}, extra={len(set(forecasts)-expected_keys)}')
    return DataBundle(dates, source_timestamps, official, fixed_price, q1_load, q1_pv, load,
pv, rt, forecasts)

```

src/model/experiments.py

SHA256 16629a5fbfac9794b8f5d6a742d636fb988efd59aa8f9229dde00e47e2bb5a05

```

from __future__ import annotations
import csv,json,hashlib,platform,sys,copy
from datetime import date,datetime,timedelta
from pathlib import Path
import numpy as np
from .rolling import simulate,RunSpec,summarize,interval
from .lp import solve_deterministic

def write_csv(path,rows):
    path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)
    if not rows:raise ValueError(f'No evidence rows: {path}')
    with path.open('w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)

def q1(data,cfg):

    sol=solve_deterministic(data.q1_load_kw/6,data.q1_pv_kw/6,data.fixed_price,6000,np.full(1
44,np.nan),'force_target',6000,0,cfg)
    rows=[];starts=np.r_[6000,sol.soc_end[:-1]]
    for j in range(144):

        g=float(sol.grid[j]);c=float(sol.charge[j]);d=float(sol.discharge[j]);L=float(data.q1_load_kw[j]
/6);V=float(data.q1_pv_kw[j]/6);p=float(data.fixed_price[j]);u=max(g+V+d-L-c,0)
        rows.append(dict(date='2025-01-
01',position=j+1,source_timestamp=data.source_timestamps[j],physical_interval=interval(j),o
fficial_label=data.official_labels[j],plan_00_grid_kwh=g,final_adjusted_grid_kwh=g,adjustm
ent_kwh=0,charge_kwh=c,discharge_kwh=d,soc_start_kwh=float(starts[j]),soc_end_kwh=flo
at(sol.soc_end[j]),curtail_pv_kwh=min(u,V),planned_unused_nonpv_kwh=max(u-
V,0),forecast_load_kwh=L,forecast_pv_kwh=V,actual_load_kwh=L,actual_pv_kwh=V,forec
ast_price=p,settlement_price=p,emergency_kwh=0,unused_surplus_kwh=u,plan_cost_yuan=
p*g,adjustment_cost_yuan=0,emergency_cost_yuan=0,final_relative_cost_yuan=p*g,sequentia
l_cost_yuan=p*g,decision_hour=0,history_cutoff_date='',model_arm='deterministic_q1',sce
nario='Q1'))
    return rows

```

```

def startup(data,cfg,mode):
    """Declared common January commissioning policy, independent of fitted model.
    Battery idle. With no completed day, scheduled purchase is zero and actual
    deficits are covered by the emergency contract. Thereafter use the empirical
    price-weighted 80% net-load quantile of completed days. No manufactured prior.
    """
    rows=[];soc=float(cfg['study']['initial_soc_kwh'])
    for di,day in enumerate(data.dates):
        if day>=date.fromisoformat(cfg['study']['official_start']):break
        hist=list(range(max(0,di-int(cfg['forecast']['scenario_count'])),di));B=np.zeros(144)
        if hist:
            nets=np.maximum((data.load_kw[hist]-data.pv_kw[hist])/6,0)
            P=data.realtime_price[hist] if mode.startswith('q4') else np.tile(data.fixed_price,
            (len(hist),1))
            for j in range(144):

order=np.argsort(nets[:,j]);cum=np.cumsum(P[order,j]);ix=np.searchsorted(cum,.8*cum[-1]);
B[j]=nets[order[ix],j]
            for j in range(144):

L=float(data.load_kw[di,j]/6);V=float(data.pv_kw[di,j]/6);p=float(data.realtime_price[di,j] if
mode.startswith('q4') else data.fixed_price[j]);Q=max(L-V-B[j],0);U=max(B[j]+V-L,0)

rows.append(dict(mode=mode,date=str(day),position=j+1,physical_interval=interval(j),plan_
grid_kwh=float(B[j]),charge_kwh=0.,discharge_kwh=0.,soc_start_kwh=soc,soc_end_kwh=soc,
c,actual_load_kwh=L,actual_pv_kwh=V,emergency_kwh=Q,unused_surplus_kwh=U,settleme
nt_price=p,cost_yuan=p*B[j]+5*p*Q,history_dates=''.join(str(data.dates[i]) for i in
hist),policy='zero_plan_no_history' if not hist else 'empirical_newsvendor_idle_battery'))
    return rows

def perfect(data,cfg):
    out=[];i=data.date_index[date.fromisoformat(cfg['study']
['official_start'])];n=(len(data.dates)-i)*144
    for mode in ['q2','q4_2']:
        p=data.realtime_price[i:].reshape(-1) if mode.startswith('q4') else
np.tile(data.fixed_price,len(data.dates)-i)
        sol=solve_deterministic(data.load_kw[i:].reshape(-1)/6,data.pv_kw[i:].reshape(-1)/
6,p,6000,np.full(n,np.nan),'force_target',6000,0,cfg)

out.append(dict(mode=mode,purchase_cost_yuan=float(p@sol.grid),initial_soc_kwh=6000,te
rminal_soc_kwh=6000,days=len(data.dates)-
i,information='ex_post_lower_bound_not_executable'))
    return out

def run_all(data,cfg,stage='all'):

```

```

root=Path(cfg['_output_root']);frozen=root/'frozen';ev=root/'evidence';frozen.mkdir(parents=True,exist_ok=True);ev.mkdir(parents=True,exist_ok=True)
# The input configuration never changes in place. Chosen January parameters
# live in a resolved copy and are reused by every later branch.
cfg=copy.deepcopy(cfg)
if stage in ('all','train'):
    training=[]
    for scenario_model in ['raw_history','causal_residual']:
        for mult in cfg['rolling']['soft_penalty_multipliers']:
            trial=copy.deepcopy(cfg);trial['rolling']=
['selected_soft_penalty_multiplier']=mult;trial['forecast']['scenario_model']=scenario_model
            for mode in ['q2','q3','q4_2','q4_3']:

r=simulate(data,trial,RunSpec(mode,'S0',collect_detail=False),initial_soc=6000,start_date=date(2025,1,8),end_date=date(2025,1,31),evaluation_terminal=True)

training.append(dict(scenario_model=scenario_model,penalty_multiplier=mult,**r['summary']
));print('TRAIN',scenario_model,mode,mult,r['summary']
['final_relative_cost_yuan'],flush=True)
    scenario_model,selected=min([(sm,m) for sm in ['raw_history','causal_residual'] for m in
cfg['rolling']['soft_penalty_multipliers']],key=lambda pair:sum(r['final_relative_cost_yuan']
for r in training if r['scenario_model']==pair[0] and r['penalty_multiplier']==pair[1]))
    cfg['forecast']['scenario_model']=scenario_model
    cfg['rolling']=
['selected_soft_penalty_multiplier']=selected;write_csv(ev/'january_parameter_validation.csv',training)
    (frozen/'resolved_config.json').write_text(json.dumps({k:v for k,v in cfg.items() if not
k.startswith('_')},ensure_ascii=False,indent=2))
    if stage=='train':return
    else:
        cfg.update(json.loads((frozen/'resolved_config.json').read_text()))
    if stage in ('all','official'):

write_csv(frozen/'time_index.csv',data.time_rows());write_csv(frozen/'q1_dispatch.csv',q1(data,
a,cfg));write_csv(frozen/'perfect_information_summary.csv',perfect(data,cfg))
    summary=[];versions=[];stats=[];startup_summary=[]
    for mode in ['q2','q4_2','q3','q4_3']:
        jan=startup(data,cfg,mode);write_csv(frozen/f'{mode}
_january_dispatch.csv',jan);initial=jan[-1]
['soc_end_kwh'];startup_summary.append(dict(mode=mode,initial_soc_kwh=jan[0]
['soc_start_kwh'],terminal_soc_kwh=initial,cost_yuan=sum(x['cost_yuan'] for x in
jan),policy='common_causal_commissioning'))
        if mode in ('q2','q4_2'):
            r=simulate(data,cfg,RunSpec(mode,'S0'),initial_soc=initial);write_csv(frozen/
f'{mode}
_dispatch.csv',r['dispatch']);summary.append(r['summary']);stats.extend(r['solve_stats'])
            for arm,point,nostore in [('mean_forecast',True,False),('no_storage',False,True)]:

control=simulate(data,cfg,RunSpec(mode,'S0',arm,collect_detail=False),initial_soc=initial,poi

```

```

nt=point,no_storage=nostore);write_csv(frozen/f {mode} _ {arm}
_daily.csv',control['daily']);write_csv(frozen/f {mode} _ {arm}
_dispatch.csv',control['dispatch']);write_csv(frozen/f {mode} _ {arm}
_solver.csv',control['solve_stats']);summary.append(control['summary'])
    print('OFFICIAL',mode,r['summary']['final_relative_cost_yuan'],flush=True)
    else:
        ref=simulate(data,cfg,RunSpec(mode,'S0'),initial_soc=initial);write_csv(frozen/
f {mode} _ S0_dispatch.csv',ref['dispatch']);write_csv(frozen/f {mode}
_S0_daily.csv',ref['daily']);write_csv(frozen/f {mode}
_S0_decisions.csv',ref['decision_versions']);summary.append(ref['summary']);stats.extend(ref
['solve_stats'])
        for sc in ['S1','S2','S3']:

r=simulate(data,cfg,RunSpec(mode,sc),initial_soc=initial,common_plans=ref['plans']);write_c
sv(frozen/f {mode} _ {sc} _ dispatch.csv',r['dispatch']);write_csv(frozen/f {mode} _ {sc}
_daily.csv',r['daily']);write_csv(frozen/f {mode} _ {sc}
_decisions.csv',r['decision_versions']);summary.append(r['summary']);stats.extend(r['solve_st
ats'])
        if sc=='S3':write_csv(frozen/f {mode}
_dispatch.csv',r['dispatch']);write_csv(frozen/f {mode}
_decision_versions.csv',r['decision_versions']);versions.extend(r['decision_versions'])
        print('OFFICIAL',mode,sc,r['summary']['final_relative_cost_yuan'],flush=True)

write_csv(frozen/'decision_versions.csv',versions);write_csv(frozen/'experiment_summary.cs
v',summary);write_csv(frozen/'official_summary.csv',[r for r in summary if
r['model_arm']=='joint_empirical' and (r['mode'] in ('q2','q4_2') or
r['scenario']=='S3')]);write_csv(frozen/'startup_summary.csv',startup_summary);write_csv(fro
zen/'solver_decisions.csv',stats)

(frozen/'model_run_summary.json').write_text(json.dumps(dict(generated_at=datetime.now().
astimezone().isoformat(),environment=dict(python=sys.version,platform=platform.platform()
),results=summary,training_selected_multiplier=cfg['rolling']
['selected_soft_penalty_multiplier'],official_scenario='S3',comparison_contract='identical_ref
erence_midnight_grid_plans; common_January_idle_battery_startup;
same_terminal_SOC'),ensure_ascii=False,indent=2))
    return cfg

```

src/model/forecast.py

SHA256 c148e88935f278fea55cd8aceb86b86781bc37bb2c6e770ae28a01d80cf764bb

"""Strict causal forecast interface. See risk.py for joint scenario provenance."""

from datetime import date

import numpy as np

def history_days(data,target_day:date,cutoff_day:date,count=4):

ids=[i for i,d in enumerate(data.dates) if d<cutoff_day and
d.weekday()==target_day.weekday()]

if not ids:ids=[i for i,d in enumerate(data.dates) if d<cutoff_day]

if not ids:raise ValueError('No causal history: startup policy is required')

return ids[-count:]

```

def profile_horizon(data,start_global,length,cutoff_day,kind,count):
    matrix={'load':data.load_kw,'pv':data.pv_kw,'price':data.realtime_price}
    [kind];out=[];used=set()
    for k in range(length):

di,pos=divmod(start_global+k,144);hist=history_days(data,data.dates[di],cutoff_day,count);u
sed.update(hist);out.append(matrix[hist,pos].mean())
    return np.array(out),[data.dates[i] for i in sorted(used)]

def published_pv_horizon(data,day,issue_hour,length):
    return np.repeat(data.pv_forecast_kw[(day,issue_hour)],6)[:length].copy()

```

src/model/lp.py

```

SHA256 9d84289c658d300124ce3c862359880a407f7a3154db01a65c146581d8d4c6e7
from __future__ import annotations

```

```

from dataclasses import dataclass
import time
from typing import Any

```

```

import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

```

```

@dataclass
class DispatchSolution:
    grid: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    curtail: np.ndarray
    emergency: np.ndarray
    soc_end: np.ndarray
    objective: float
    elapsed_seconds: float
    iterations: int
    max_balance_residual: float
    max_soc_residual: float

```

```

def _one_way(charge:np.ndarray,discharge:np.ndarray,unused:np.ndarray,eta:float) ->
tuple[np.ndarray,np.ndarray,np.ndarray]:
    """Remove simultaneous cycling while preserving SOC change and energy balance."""
    delta=eta*charge-discharge/eta
    c=np.maximum(delta,0)/eta
    d=np.maximum(-delta,0)*eta
    u=unused+(d-discharge)-(c-charge)

```

```

u[np.abs(u)<1e-10]=0.0
if np.min(u)<-1e-7: raise AssertionError(f"one-way normalization produced negative
unused energy {np.min(u)}")
return c,d,np.maximum(u,0.0)

def _method_options(method: str, tolerance: float) -> dict[str, Any]:
    options: dict[str, Any] = {"presolve": True}
    if method in {"highs", "highs-ds"}:
        options.update({"primal_feasibility_tolerance": tolerance, "dual_feasibility_tolerance":
tolerance})
    if method in {"highs", "highs-ipm"}:
        options["ipm_optimality_tolerance"] = max(tolerance, 1e-10)
    return options

```

```

def solve_deterministic(
    load_kwh: np.ndarray,
    pv_kwh: np.ndarray,
    price: np.ndarray,
    initial_soc: float,
    baseline: np.ndarray,
    terminal_rule: str,
    terminal_target: float,
    terminal_penalty: float,
    cfg: dict[str, Any],
    method: str = "highs",
    tiebreak: float | None = None,
) -> DispatchSolution:
    load = np.asarray(load_kwh, float); pv = np.asarray(pv_kwh, float); price =
np.asarray(price, float)
    base = np.asarray(baseline, float)
    T = len(load)
    if not (len(pv) == len(price) == len(base) == T):
        raise ValueError("LP input lengths differ")
    G=slice(0,T); C=slice(T,2*T); D=slice(2*T,3*T); U=slice(3*T,4*T)
    E=slice(4*T,5*T); DN=slice(5*T,6*T); UP=slice(6*T,7*T)
    ZP=7*T; ZN=7*T+1; n=7*T+2
    committed=np.isfinite(base)
    c=np.zeros(n)
    c[G]=np.where(committed,0.0,price)
    c[DN]=np.where(committed,-cfg["cost"]["down_adjustment_multiplier"]*price,0.0)
    c[UP]=np.where(committed,cfg["cost"]["up_adjustment_multiplier"]*price,0.0)
    tb=float(cfg["cost"]["throughput_tiebreak_yuan_per_kwh"]) if tiebreak is None else
tiebreak)
    c[C]=tb; c[D]=tb
    if terminal_rule == "soft_target": c[ZP]=terminal_penalty; c[ZN]=terminal_penalty
    rr:list[int]=[]; cc:list[int]=[]; vv:list[float]=[]; rhs:list[float]=[]; row=0
    for t in range(T):

```

```

    rr += [row]*4; cc += [G.start+t,C.start+t,D.start+t,U.start+t]; vv += [1,-1,1,-1]
    rhs.append(float(load[t]-pv[t])); row+=1
eta=float(cfg["storage"]["charge_efficiency"])
for t in range(T):
    rr += [row]*3; cc += [C.start+t,D.start+t,E.start+t]; vv += [-eta,1/eta,1]
    if t: rr.append(row); cc.append(E.start+t-1); vv.append(-1); rhs.append(0.0)
    else: rhs.append(float(initial_soc))
    row+=1
for t in np.flatnonzero(committed):
    rr += [row]*3; cc += [G.start+t,DN.start+t,UP.start+t]; vv += [1,1,-1]
    rhs.append(float(base[t])); row+=1
if terminal_rule == "soft_target":
    rr += [row]*3; cc += [E.stop-1,ZP,ZN]; vv += [1,-1,1];
rhs.append(float(terminal_target)); row+=1
Aeq=coo_matrix((vv,(rr,cc)),shape=(row,n)).tocsr()
st=cfg["storage"]; step=float(st["power_kw"])*float(st["interval_minutes"])/60
bounds=(
    [(0,None)]*T+[(0,step)]*T+[(0,step)]*T+[(0,float(x)) for x in pv]
    +[(float(st["soc_min_kwh"]),float(st["soc_max_kwh"]))]*T
    +[(0,float(base[t])) if committed[t] else (0,0) for t in range(T)]
    +[(0,None) if committed[t] else (0,0) for t in range(T)]
    +([(0,None),(0,None)] if terminal_rule == "soft_target" else [(0,0),(0,0)])
)
if terminal_rule in {"daily_closed","state_cycle","force_target"}:
    target = initial_soc if terminal_rule == "state_cycle" else terminal_target
    bounds[E.stop-1]=(float(target),float(target))
started=time.perf_counter()
res=linprog(c,A_eq=Aeq,b_eq=np.asarray(rhs),bounds=bounds,method=method,
            options=_method_options(method,float(cfg["audit"]["feasibility_tolerance"])))
elapsed=time.perf_counter()-started
if not res.success: raise RuntimeError(f"LP infeasible ({method}): {res.message}")
x=res.x; charge,discharge,unused=_one_way(x[C],x[D],x[U],eta);
e0=np.r_[initial_soc,x[E][-1]]
bal=x[G]+pv+discharge-load-charge-unused
soc=x[E]-e0-eta*charge+discharge/eta
return
DispatchSolution(x[G],charge,discharge,unused,np.zeros(T),x[E],float(res.fun),elapsed,int(res
.nit),float(np.max(np.abs(bal))),float(np.max(np.abs(soc))))

def solve_stochastic(
    load_kwh: np.ndarray,
    pv_scenarios_kwh: np.ndarray,
    price: np.ndarray,
    initial_soc: float,
    baseline: np.ndarray,
    execute_intervals: int,
    terminal_rule: str,
    terminal_target: float,
    terminal_penalty: float,

```

```

    cfg: dict[str, Any],
    method: str = "highs",
    tiebreak: float | None = None,
) -> DispatchSolution:
    """Two-stage extensive-form LP; current executable block is nonanticipative."""
    load=np.asarray(load_kwh,float); pvs=np.asarray(pv_scenarios_kwh,float);
    price=np.asarray(price,float); base=np.asarray(baseline,float)
    S,T=pvs.shape
    if len(load)!=T or len(price)!=T or len(base)!=T: raise ValueError("stochastic LP input
lengths differ")
    # Per scenario: grid, charge, discharge, curtail/unused, emergency, SOC, down, up.
    per=8*T; soft=terminal_rule=="soft_target"; n=S*per+(2*S if soft else 0)
    committed=np.isfinite(base); c=np.zeros(n); prob=1.0/S
    st=cfg["storage"]; eta=float(st["charge_efficiency"]);
    step=float(st["power_kw"])*float(st["interval_minutes"])/60
    down_mult=float(cfg["cost"]["down_adjustment_multiplier"]); up_mult=float(cfg["cost"]
["up_adjustment_multiplier"]); em_mult=float(cfg["cost"]["emergency_multiplier"])
    tb=float(cfg["cost"]["throughput_tiebreak_yuan_per_kwh"]) if tiebreak is None else
    tiebreak)
    bounds:list[tuple[float,float|None]]=[]
    def slices(s:int):
        o=s*per
        return tuple(slice(o+i*T,o+(i+1)*T) for i in range(8))
    for s in range(S):
        G,C,D,U,H,E,DN,UP=slices(s)
        c[G]=prob*np.where(committed,0.0,price); c[C]=prob*tb; c[D]=prob*tb
        c[H]=prob*em_mult*price
        c[DN]=prob*np.where(committed,-down_mult*price,0.0);
    c[UP]=prob*np.where(committed,up_mult*price,0.0)
    bounds += [(0,None)]*T+[(0,step)]*T+[(0,step)]*T+[(0,None)]*T+[(0,None)]*T
    bounds += [(float(st["soc_min_kwh"]),float(st["soc_max_kwh"]))]*T
    bounds += [(0,float(base[t])) if committed[t] else (0,0) for t in range(T)]
    bounds += [(0,None) if committed[t] else (0,0) for t in range(T)]
    if soft:
        for s in range(S):
            c[S*per+2*s:S*per+2*s+2]=prob*terminal_penalty
            bounds += [(0,None),(0,None)]
    rr:list[int]=[]; cc:list[int]=[]; vv:list[float]=[]; rhs:list[float]=[]; row=0
    for s in range(S):
        G,C,D,U,H,E,DN,UP=slices(s)
        for t in range(T):
            rr += [row]*5; cc += [G.start+t,C.start+t,D.start+t,U.start+t,H.start+t]; vv += [1,-1,1,-
1,1]
            rhs.append(float(load[t]-pvs[s,t])); row+=1
        for t in range(T):
            rr += [row]*3; cc += [C.start+t,D.start+t,E.start+t]; vv += [-eta,1/eta,1]
            if t: rr.append(row); cc.append(E.start+t-1); vv.append(-1); rhs.append(0.0)
            else: rhs.append(float(initial_soc))
            row+=1
        for t in np.flatnonzero(committed):

```

```

rr += [row]*3; cc += [G.start+t,DN.start+t,UP.start+t]; vv += [1,1,-1]
rhs.append(float(base[t])); row+=1
if soft:
    rr += [row]*3; cc += [E.stop-1,S*per+2*s,S*per+2*s+1]; vv += [1,-1,1]
    rhs.append(float(terminal_target)); row+=1
elif terminal_rule in {"daily_closed","state_cycle","force_target":
    target=initial_soc if terminal_rule=="state_cycle" else terminal_target
    bounds[E.stop-1]=(float(target),float(target))
k=min(int(execute_intervals),T)
for s in range(1,S):
    a=slices(0); b=slices(s)
    for block in (0,1,2):
        for t in range(k):
            rr += [row,row]; cc += [a[block].start+t,b[block].start+t]; vv += [1,-1];
rhs.append(0.0); row+=1
Aeq=coo_matrix((vv,(rr,cc)),shape=(row,n)).tocsr()
started=time.perf_counter()
res=linprog(c,A_eq=Aeq,b_eq=np.asarray(rhs),bounds=bounds,method=method,
            options=_method_options(method,float(cfg["audit"])[["feasibility_tolerance"]]))
elapsed=time.perf_counter()-started
if not res.success: raise RuntimeError(f"stochastic LP infeasible ({method}):
{res.message}")
x=res.x; G,C,D,U,H,E,_,_=slices(0);
charge,discharge,unused=_one_way(x[C],x[D],x[U],eta); e0=np.r_[initial_soc,x[E][:1]]
bal=x[G]+pvs[0]+discharge+x[H]-load-charge-unused
soc=x[E]-e0-eta*charge+discharge/eta
return
DispatchSolution(x[G],charge,discharge,unused,x[H],x[E],float(res.fun),elapsed,int(res.nit),fl
oat(np.max(np.abs(bal))),float(np.max(np.abs(soc))))

```

src/model/risk.py

SHA256 93a7bf5c1a8572ee99e4098d67ba03fac164fdea33f9ea8198eab654fff0591a
 """Causal empirical scenarios and a nonanticipative stochastic storage LP.

Only emergency procurement is scenario recourse. All scheduled grid, charge, discharge and SOC variables are shared across scenarios over the full horizon.

```

from __future__ import annotations
from dataclasses import dataclass
from datetime import date,datetime,time,timedelta
import time as timer
import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix
from .data import DataBundle

```

```

@dataclass
class Cases:
    load: np.ndarray

```

```

pv: np.ndarray
price: np.ndarray
histories: list[str]
observed_until: list[str]
issue_time: str
kind: str

def causal_profile(data, matrix, target_i, cutoff_i):
    ids=[i for i in range(cutoff_i) if data.dates[i].weekday()==data.dates[target_i].weekday()]
    [-4:]
    if not ids:ids=list(range(max(0,cutoff_i-4),cutoff_i))
    if not ids:raise ValueError("No prequential baseline before first observation")
    return matrix[ids].mean(axis=0)

def point_horizon(data, matrix, day_i, issue, length):
    first=causal_profile(data, matrix, day_i, day_i)[issue*6:]
    if length>len(first):first=np.r_[first,causal_profile(data, matrix, day_i+1, day_i)]
    return first[:length]

def cases(data:DataBundle, cfg:dict, day:date, issue:int, length:int, mode:str, point:bool=False)-
>Cases:
    di=data.date_index[day];now=di*144+issue*6
    histories=[];ends=[];loads=[];pvs=[];prices=[]
    n=int(cfg['forecast']['scenario_count']);
    calibrated=cfg['forecast'].get('scenario_model')=='causal_residual'
    if calibrated:
        current_load=point_horizon(data, data.load_kw, di, issue, length)
        current_pv=point_horizon(data, data.pv_kw, di, issue, length) if mode in ('q2', 'q4_2') else
None
        current_price=point_horizon(data, data.realtime_price, di, issue, length) if
mode.startswith('q4') else None
    # Each donor is an actually observed 24h trajectory ending no later than now.
    for old_i in range(max(0, di-n), di):
        old=data.dates[old_i];g=old_i*144+issue*6
        if calibrated and old_i==0:continue
        if g+144>now:continue
        sl=slice(g, g+length)
        l=data.load_kw.reshape(-1)[sl]/6
        if calibrated:l=np.maximum(current_load+6*l-
point_horizon(data, data.load_kw, old_i, issue, length), 0)/6
        if mode in ('q3', 'q4_3'):
            old_actual=data.pv_kw.reshape(-1)[g:g+144].reshape(24,6).mean(axis=1)
            residual=old_actual-data.pv_forecast_kw[(old, issue)]
            # Forecast lead 1 represents the hour after publication (right endpoint
            # zero-order hold). No interpolation using future actual observations.
            pp=np.maximum(np.repeat(data.pv_forecast_kw[(day, issue)]+residual, 6)[:length], 0)/6

```

```

else:
    pp=data.pv_kw.reshape(-1)[sl]/6
    if calibrated:pp=np.maximum(current_pv+6*pp-
point_horizon(data,data.pv_kw,old_i,issue,length),0)/6
    p=data.realtime_price.reshape(-1)[sl] if mode.startswith('q4') else
np.asarray([data.fixed_price[(issue*6+k)%144] for k in range(length)])
    if calibrated and mode.startswith('q4'):p=np.maximum(current_price+p-
point_horizon(data,data.realtime_price,old_i,issue,length),0)
    loads.append(l);pvs.append(pp);prices.append(p);histories.append(old.isoformat())
    ends.append((datetime.combine(old,time())+timedelta(hours=issue+24)).isoformat(sep='
',timespec='minutes'))
    if not loads:raise ValueError('No observed history: execute the declared startup policy;
never substitute future actuals')
    L=np.array(loads);V=np.array(pvs);P=np.array(prices)
    if
point:L=L.mean(axis=0,keepdims=True);V=V.mean(axis=0,keepdims=True);P=P.mean(axis
=0,keepdims=True)
    return Cases(L,V,P,histories,ends,f'{day} {issue:02d}:00','historical_joint' if mode in
('q2','q4_2') else 'published_pv_plus_observed_residual')

```

```

@dataclass
class Solution:
    grid:np.ndarray
    charge:np.ndarray
    discharge:np.ndarray
    soc_end:np.ndarray
    objective:float
    seconds:float
    balance_residual:float
    soc_residual:float
    model_scenarios:int

```

```

def
solve(cs:Cases,soc0:float,baseline:np.ndarray,cfg:dict,terminal:bool=False,lock:np.ndarray|
None=None,method:str='highs',no_storage:bool=False)->Solution:
    L,V,P=cs.load,cs.pv,cs.price;S,T=L.shape;meanp=P.mean(axis=0)
    # Shared G,C,D,E,down,up, then scenario emergency variables and terminal slacks.
    n=(6+S)*T+2;G=0;C=T;D=2*T;E=3*T;DN=4*T;UP=5*T;Q=6*T;Z=n-2
    committed=np.isfinite(baseline);obj=np.zeros(n)
    obj[G:G+T]=np.where(committed,0,meanp)
    obj[DN:DN+T]=np.where(committed,-.5*meanp,0)
    obj[UP:UP+T]=np.where(committed,1.5*meanp,0)
    obj[Q:Q+S*T]=(5*P/S).reshape(-1)
    tb=float(cfg['cost'] ['throughput_tiebreak_yuan_per_kwh']);obj[C:C+T]=tb;obj[D:D+T]=tb
    penalty=float(np.median(meanp))*float(cfg['rolling'] ['selected_soft_penalty_multiplier'])
    obj[Z:Z+2]=penalty
    eta_c=float(cfg['storage'] ['charge_efficiency']);eta_d=float(cfg['storage']
['discharge_efficiency'])

```

```

cap=float(cfg['storage']['power_kw']/6;low=cfg['storage']
['soc_min_kwh'];high=cfg['storage']['soc_max_kwh']
bounds=[(0,None)]*T+[(0,0 if no_storage else cap)]*(2*T)+[(low,high)]*T
bounds += [(0,float(baseline[t])) if committed[t] else (0,0) for t in range(T)]
bounds += [(0,None) if committed[t] else (0,0) for t in range(T)]
bounds += [(0,None)]*(S*T+2)
if lock is not None:
    for t,v in enumerate(lock):
        if np.isfinite(v):bounds[t]=(float(v),float(v))
    if terminal:bounds[E+T-1]=(cfg['study']['terminal_soc_kwh'],cfg['study']
['terminal_soc_kwh'])
    er=[];ec=[];ev=[];b=[]
    def eq(pairs,rhs):
        r=len(b);b.append(rhs)
        for col,val in pairs:er.append(r);ec.append(col);ev.append(val)
    for t in range(T):
        pairs=[(E+t,1),(C+t,-eta_c),(D+t,1/eta_d)]
        if t:pairs.append((E+t-1,-1))
        eq(pairs,soc0 if t==0 else 0)
    for t in np.flatnonzero(committed):eq([(G+t,1),(DN+t,1),(UP+t,-1)],float(baseline[t]))
    eq([(E+T-1,1),(Z,-1),(Z+1,1)],float(cfg['study']['terminal_soc_kwh']))
    # G + D - C + Q_s >= L_s - PV_s. Surplus is allowed, never sold.
    rr=[];cc=[];vv=[]
    for s in range(S):
        for t in range(T):
            r=s*T+t;rr.extend([r]*4);cc.extend([G+t,D+t,C+t,Q+s*T+t]);vv.extend([-1,-1,1,-1])
        A=coo_matrix((vv,(rr,cc)),shape=(S*T,n)).tocsr();Aeq=coo_matrix((ev,
(er,ec)),shape=(len(b),n)).tocsr()
        start=timer.perf_counter()
        res=linprog(obj,A_ub=A,b_ub=(V-
L).reshape(-1),A_eq=Aeq,b_eq=np.array(b),bounds=bounds,method=method,options={'prima
l_feasibility_tolerance':1e-8,'dual_feasibility_tolerance':1e-8,'ipm_optimality_tolerance':1e-9}
)
        seconds=timer.perf_counter()-start
        if not res.success:raise RuntimeError(res.message)
        x=res.x;c=x[C:C+T].copy();d=x[D:D+T].copy();e=x[E:E+T].copy()
        # Exact dominance transformation: preserve SOC, reduce purchases needed by
        # removing simultaneous cycling; free disposal and no minimum flow assumed.
        delta=eta_c*c-d/eta_d;c=np.maximum(delta,0)/eta_c;d=np.maximum(-delta,0)*eta_d
        q=x[Q:Q+S*T].reshape(S,T)
        violation=float(np.maximum(L+c[None,:]-V-d[None,:]-x[:T][None,:]-q,0).max())
        sr=float(np.abs(e-np.r_[soc0,e[:-1]]-eta_c*c+d/eta_d).max())
        if violation>1e-7 or sr>1e-7:raise AssertionError('Post-solve physical residual')
        return Solution(x[:T].copy(),c,d,e,float(res.fun),seconds,violation,sr,S)

```

src/model/rolling.py

```

SHA256 eab5712bae13320e2c960f9c9c07e55fd6814911ba2899992d56bf06cec3668e
from __future__ import annotations
from dataclasses import dataclass
from datetime import date,timedelta

```

```
import numpy as np
from .risk import cases,solve
```

```
@dataclass(frozen=True)
class RunSpec:
    mode:str
    scenario:str='S3'
    arm:str='joint_empirical'
    method:str='highs'
    start_date:date|None=None
    end_date:date|None=None
    collect_detail:bool=True
```

```
def interval(pos):
    a=pos*10;b=a+10
    return f'{a//60:02d}:{a%60:02d}-{b//60:02d}:{b%60:02d}'
```

```
def make_row(data,day,j,B,A,c,d,s0,s1,fl,fp,pricehat,owner,mode,scenario,seq,history,arm):
    di=data.date_index[day];L=float(data.load_kw[di,j]/6);V=float(data.pv_kw[di,j]/6)
    p=float(data.realtime_price[di,j] if mode.startswith('q4') else data.fixed_price[j])
    deficit=L+c-V-d-A;Q=max(deficit,0);U=max(-deficit,0);delta=A-B
    pc=p*B;ac=p*(delta+.5*abs(delta));em=5*p*Q
    forecast_unused=max(A+fp+d-fl-c,0)
    return
dict(date=day.isoformat(),position=j+1,source_timestamp=data.source_timestamps[j],physical_interval=interval(j),official_label=data.official_labels[j],

plan_00_grid_kwh=float(B),final_adjusted_grid_kwh=float(A),adjustment_kwh=float(delta),
charge_kwh=float(c),discharge_kwh=float(d),soc_start_kwh=float(s0),soc_end_kwh=float(s1
),

curtail_pv_kwh=min(forecast_unused,fp),planned_unused_nonpv_kwh=max(forecast_unused
-
fp,0),forecast_load_kwh=float(fl),forecast_pv_kwh=float(fp),actual_load_kwh=L,actual_pv_
kwh=V,

forecast_price=float(pricehat),settlement_price=p,emergency_kwh=Q,unused_surplus_kwh=
U,plan_cost_yuan=pc,adjustment_cost_yuan=ac,emergency_cost_yuan=em,

final_relative_cost_yuan=pc+ac+em,sequential_cost_yuan=pc+seq+em,decision_hour=owner
,history_cutoff_date=(day-
timedelta(days=1)).isoformat(),model_arm=arm,scenario=scenario)
```

```
def summarize(rows,mode,scenario,arm):
```

```

    sums={k:float(sum(r[k] for r in rows)) for k in
['plan_cost_yuan','adjustment_cost_yuan','emergency_cost_yuan','final_relative_cost_yuan','s
equential_cost_yuan','emergency_kwh','unused_surplus_kwh']}
    return dict(mode=mode,scenario=scenario,model_arm=arm,days=len({r['date'] for r in
rows})),initial_soc_kwh=rows[0]['soc_start_kwh'],terminal_soc_kwh=rows[-1]
['soc_end_kwh'],**sums)

```

```

def
simulate(data,cfg,spec:RunSpec,*,initial_soc:float,start_date=None,end_date=None,common
_plans=None,point=False,no_storage=False,evaluation_terminal=False):
    start=start_date or spec.start_date or date.fromisoformat(cfg['study']
['official_start']);end=end_date or spec.end_date or date.fromisoformat(cfg['study']
['official_end'])
    if initial_soc is None:raise ValueError('Explicit state from startup ledger required')
    soc=float(initial_soc);rows=[];versions=[];daily=[];stats=[];plans={}
    schedule=cfg['experiments']['schedules'][spec.scenario] if spec.mode in ('q3','q4_3') else [0]
    total=(data.date_index[end]+1)*144 if evaluation_terminal else len(data.dates)*144
    for day in data.dates:
        if not start<=day<=end:continue
        di=data.date_index[day];B=None;committed=None;seq=np.zeros(144);dayrows=[]
        for ix,issue in enumerate(schedule):
            pos=issue*6;remaining=144-pos;length=min(144,total-(di*144+pos)) if spec.mode in
('q3','q4_3') else 144
            cs=cases(data,cfg,day,issue,length,spec.mode,point)
            base=np.full(length,np.nan);lock=np.full(length,np.nan)
            if issue==0 and common_plans is not None:
                B=np.asarray(common_plans[day.isoformat()],float).copy();base[:144]=B;lock[:144]=B
                elif issue>0:base[:remaining]=B[pos:]
                terminal=di*144+pos+length==total

sol=solve(cs,soc,base,cfg,terminal=terminal,lock=lock,method=spec.method,no_storage=no_
storage)
    if issue==0:
        if B is None:B=sol.grid[:144].copy()
        committed=B.copy();plans[day.isoformat()]=B.copy()
    else:
        old=committed[pos:].copy();delta=sol.grid[:remaining]-old
        p=data.realtime_price[di,pos:] if spec.mode.startswith('q4') else
data.fixed_price[pos:]
        seq[pos:]+=p*(delta+.5*np.abs(delta))
        # Every shared current-day quantity is a genuine commitment; tomorrow
        # is a nonbinding preview. No deterministic alternate settlement path.
        for k in range(length):
            tdi,tj=divmod(di*144+pos+k,144);target=data.dates[tdi];preview=target!=day
            value=(B[k] if issue==0 else sol.grid[k]) if not preview else "
            previous=" if preview or issue==0 else float(committed[pos+k])
            if spec.collect_detail:

```

```

versions.append(dict(date=str(day),target_date=str(target),mode=spec.mode,scenario=spec.scenario,model_arm=spec.arm,decision_time=cs.issue_time,forecast_issue_time=cs.issue_time,
                    effective_start=cs.issue_time,effective_end=f'{day+timedelta(days=1)}
00:00',history_cutoff_date=str(day-
timedelta(days=1)),position=tj+1,physical_interval=interval(tj),official_label=data.official_labels[tj],
                    old_commitment_kwh=previous,new_commitment_kwh=float(value) if value!
=" else
",sensitivity_old_commitment_kwh=previous,sensitivity_new_commitment_kwh=float(value
) if value!=" else ",preview_grid_kwh=float(sol.grid[k]) if preview else ",
                    decision_basis='joint_nonanticipative_lp' if not preview else
'next_day_preview',soc_at_decision_kwh=soc,
                    load_history_dates='|'.join(cs.histories) if k==0 else
",price_history_dates='|'.join(cs.histories) if k==0 and spec.mode.startswith('q4') else
",pv_bias_history_dates='|'.join(cs.histories) if k==0 else
",residual_scenario_dates='|'.join(cs.histories) if k==0 else ",

latest_observation_time=max(cs.observed_until),is_preview_next_day=int(preview),is_previe
w_recourse=int(preview)))
    if issue>0:committed[pos:]=sol.grid[:remaining]
    nxt=schedule[ix+1] if ix+1<len(schedule) else 24;execute=(nxt-issue)*6
    sstarts=np.r_[soc,sol.soc_end[:-1]]
    for k in range(execute):
        j=pos+k

dayrows.append(make_row(data,day,j,B[j],sol.grid[k],sol.charge[k],sol.discharge[k],sstarts[k]
,sol.soc_end[k],cs.load[:,k].mean(),cs.pv[:,k].mean(),cs.price[:,k].mean(),issue,spec.mode,spe
c.scenario,seq[j],cs.histories,spec.arm))
    soc=float(sol.soc_end[execute-1])

stats.append(dict(mode=spec.mode,scenario=spec.scenario,date=str(day),issue_hour=issue,sol
ver_method=spec.method,seconds=sol.seconds,objective=sol.objective,max_balance_residual
_kwh=sol.balance_residual,max_soc_residual_kwh=sol.soc_residual,scenario_count=sol.mod
el_scenarios,latest_observation_time=max(cs.observed_until),decision_time=cs.issue_time,hi
story_dates='|'.join(cs.histories),common_action_intervals=length))

rows.extend(dayrows);daily.append(dict(date=str(day),**summarize(dayrows,spec.mode,spec
.scenario,spec.arm)))

summary=summarize(rows,spec.mode,spec.scenario,spec.arm);summary.update(solver_secon
ds=sum(s['seconds'] for s in
stats),solver_runs=len(stats),constraint_violation_count=sum(s['max_balance_residual_kwh']
>1e-7 or s['max_soc_residual_kwh']>1e-7 for s in stats))
    return
dict(dispatch=rows,decision_versions=versions,daily=daily,summary=summary,solve_stats=s
tats,plans=plans)

```

tests/test_closed_loop.py

SHA256 1f61a806e5df4a92ca5d11ef707618216086f85e54fd890dc008f11ee5583017

```

import unittest,copy,json
from dataclasses import replace
from datetime import date
from pathlib import Path
import numpy as np
from src.model.config import load_config
from src.model.data import load_data
from src.model.forecast import history_days
from src.model.risk import cases,solve
from src.model.rolling import simulate,RunSpec
from src.model.experiments import startup

class ClosedLoopTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.cfg=load_config('config/final.yaml');
resolved=Path(cls.cfg['_output_root'])/'frozen/resolved_config.json'
        if resolved.exists():cls.cfg.update(json.loads(resolved.read_text()))
        cls.data=load_data(cls.cfg)
    def test_no_future_cold_start(self):
        with self.assertRaises(ValueError):history_days(self.data,date(2025,1,1),date(2025,1,1))
        data=self.data;mut=data.load_kw.copy();mut[0]+=2000
        a=startup(data,self.cfg,'q2')[:144];b=startup(replace(data,load_kw=mut),self.cfg,'q2')
[:144]
        self.assertEqual([r['plan_grid_kwh'] for r in a],[r['plan_grid_kwh'] for r in
b]);self.assertTrue(all(r['soc_end_kwh']==6000 for r in a))
    def test_future_observations_cannot_change_decision(self):
        for mode,issue in [('q2',0),('q4_2',0),('q3',12),('q4_3',18)]:
            day=date(2025,2,1);di=self.data.date_index[day];cut=di*144+issue*6;kw={}
            for field in ['load_kw','pv_kw','realtime_price']:
                x=getattr(self.data,field).copy();x.reshape(-1)[cut:]+=321;kw[field]=x

alt=replace(self.data,**kw);a=cases(self.data,self.cfg,day,issue,144,mode);b=cases(alt,self.cfg
,day,issue,144,mode)
        for field in
['load','pv','price']:np.testing.assert_array_equal(getattr(a,field),getattr(b,field))

baseline=np.full(144,np.nan);x=solve(a,6000,baseline,self.cfg);y=solve(b,6000,baseline,self.c
fg);np.testing.assert_array_equal(x.grid,y.grid)
    def test_later_forecasts_not_visible(self):
        day=date(2025,2,1); forecasts={k:v.copy() for k,v in self.data.pv_forecast_kw.items()}
        for key in forecasts:
            if key[0]>day or (key[0]==day and key[1]>0):forecasts[key]+=9876
        alt=replace(self.data,pv_forecast_kw=forecasts)

a=cases(self.data,self.cfg,day,0,144,'q3');b=cases(alt,self.cfg,day,0,144,'q3');np.testing.assert_
array_equal(a.pv,b.pv)

    def test_nonanticipativity_and_formal_ledger(self):
        data=self.data;cfg=self.cfg;st=date(2025,2,1);en=date(2025,2,2)

```

```

ref=simulate(data,cfg,RunSpec('q3','S0'),initial_soc=6000,start_date=st,end_date=en)

r=simulate(data,cfg,RunSpec('q3','S3'),initial_soc=6000,start_date=st,end_date=en,common_
plans=ref['plans'])
    np.testing.assert_allclose([x['plan_00_grid_kwh'] for x in ref['dispatch']],
[x['plan_00_grid_kwh'] for x in r['dispatch']],atol=1e-8)
    commitments={},fees={}
    for v in r['decision_versions']:
        if v['is_preview_next_day']:continue
        key=(v['target_date'],v['position']);new=v['new_commitment_kwh']
        if key in commitments:
            self.assertAlmostEqual(commitments[key],v['old_commitment_kwh']);delta=new-
commitments[key];p=data.fixed_price[key[1]-
1];fees[key]=fees.get(key,0)+p*(delta+.5*abs(delta))
            commitments[key]=new
        for x in r['dispatch']:

key=(x['date'],x['position']);self.assertAlmostEqual(commitments[key],x['final_adjusted_grid
_kwh']);self.assertAlmostEqual(x['sequential_cost_yuan'],x['plan_cost_yuan']
+fees.get(key,0)+x['emergency_cost_yuan']);self.assertGreaterEqual(x['sequential_cost_yuan']
-x['final_relative_cost_yuan'],-1e-7)
        self.assertAlmostEqual(x['soc_end_kwh'],x['soc_start_kwh']+.9*x['charge_kwh']-
x['discharge_kwh']/.9);self.assertLess(min(x['charge_kwh'],x['discharge_kwh']),1e-7)
    def test_partition_does_not_impose_terminal_reset(self):
        st=date(2025,1,30);en=date(2025,1,31);cfg=self.cfg;d=self.data
        full=simulate(d,cfg,RunSpec('q2','S0'),initial_soc=6000,start_date=st,end_date=en)
        first=simulate(d,cfg,RunSpec('q2','S0'),initial_soc=6000,start_date=st,end_date=st)
        second=simulate(d,cfg,RunSpec('q2','S0'),initial_soc=first['summary']
['terminal_soc_kwh'],start_date=en,end_date=en)
        np.testing.assert_allclose([r['final_adjusted_grid_kwh'] for r in full['dispatch']],
[r['final_adjusted_grid_kwh'] for r in first['dispatch']+second['dispatch']],atol=1e-8)
    def test_separate_charge_discharge_efficiencies(self):
        cfg=copy.deepcopy(self.cfg);cfg['storage']['discharge_efficiency']=.85

cs=cases(self.data,cfg,date(2025,2,1),0,144,'q2');x=solve(cs,6000,np.full(144,np.nan),cfg)
    np.testing.assert_allclose(x.soc_end-np.r_[6000,x.soc_end[:-1]],.9*x.charge-
x.discharge/.85,atol=1e-8)
    def test_charge_limit_and_year_end(self):

r=simulate(self.data,self.cfg,RunSpec('q4_3','S3'),initial_soc=1200,start_date=date(2025,12,3
1),end_date=date(2025,12,31))
    self.assertAlmostEqual(r['summary']['terminal_soc_kwh'],6000)
    for x in r['dispatch']:

self.assertLessEqual(x['charge_kwh'],5000/6+1e-7);self.assertLessEqual(x['discharge_kwh'],5
000/6+1e-7);self.assertGreaterEqual(x['soc_end_kwh'],1200-
1e-7);self.assertLessEqual(x['soc_end_kwh'],10800+1e-7)

if __name__=='__main__':unittest.main(verbosity=2)

```

2.2 参数扰动补充程序

scripts/parameter_perturbation.py

SHA256 7d7c18ec4d6cb8703931d2e78ad2b00406d1d8800c2a31ae9d9b39df4cba3f13
"""固定窗口参数扰动；只新增诊断文件，不改变正式模型及结果。

运行前按 README 恢复冻结 CSV，设置 CUMCM_SOURCE_ROOT 指向原题父目录。

两类参数分别施加-20%、-10%、0、+10%、+20%扰动。

每次固定同一状态、日前合同、场景和窗口；不解释为全年策略回放。

```
"""
from pathlib import Path
from datetime import date
import sys, json, copy, hashlib
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.model.config import load_config
from src.model.data import load_data
from src.model.risk import cases, solve

def main():
    cfg = load_config('config/final.yaml')
    frozen = Path(cfg['_output_root']) / 'frozen'
    cfg.update(json.loads((frozen / 'resolved_config.json').read_text()))
    data = load_data(cfg)
    output = ROOT / 'research' / 'parameter_perturbation'
    output.mkdir(parents=True, exist_ok=True)
    # 输入质量检查采用实际读取矩阵，不删除真实高峰或填造观测。
    arrays = {'负载功率': data.load_kw, '光伏实测功率': data.pv_kw,
              '实时电价': data.realtime_price,
              '光伏预报功率': np.stack(list(data.pv_forecast_kw.values()))}
    quality = []
    for name, a in arrays.items():
        quality.append(dict(variable=name, values=int(a.size),
                             missing=int((~np.isfinite(a)).sum()),
                             negative=int((a < 0).sum()),
                             minimum=float(a.min()), maximum=float(a.max())))
    (output / 'input_quality.json').write_text(
        json.dumps(quality, ensure_ascii=False, indent=2))
    rows = []
    for mode in ['q3', 'q4_3']:
        dispatch = pd.read_csv(frozen / f'{mode}_dispatch.csv')
        for daystr in cfg['experiments']['representative_dates']:
            day = date.fromisoformat(daystr)
```

```

dd = dispatch[dispatch.date == daystr]
for issue in [0, 6, 12, 18]:
    pos = issue * 6
    length = min(144, len(data.dates)*144
                - data.date_index[day]*144 - pos)
    baseline = np.full(length, np.nan)
    baseline[:144-pos] = dd.plan_00_grid_kwh.to_numpy()[pos:]
    lock = np.full(length, np.nan)
    if issue == 0:
        lock[:144] = baseline[:144]
    cs = cases(data, cfg, day, issue, length, mode)
    initial = float(dd.soc_start_kwh.iloc[pos])
    reference = solve(cs, initial, baseline, cfg, lock=lock)
    # 求解器省略已支付日前合同的常数项；比较费用幅度时补回。
    committed = np.isfinite(baseline)
    constant = float(np.dot(cs.price.mean(axis=0)[committed],
                           baseline[committed]))
    # 两参数逐一变化，其他输入均保持相同；0档直接复用基线。
    for parameter in ['terminal_multiplier', 'power_kw']:
        for ratio in [0.8, 0.9, 1.0, 1.1, 1.2]:
            perturbed = copy.deepcopy(cfg)
            if parameter == 'terminal_multiplier':
                original = cfg['rolling']['selected_soft_penalty_multiplier']
                perturbed['rolling']['selected_soft_penalty_multiplier'] = original*ratio
            else:
                original = cfg['storage']['power_kw']
                perturbed['storage']['power_kw'] = original*ratio
            solution = (reference if ratio == 1.0 else
                       solve(cs, initial, baseline, perturbed, lock=lock))
            # 窗口目标可能为负或接近零，统一用绝对基值作分母。
            denominator = max(abs(reference.objective + constant), 1.0)
            rows.append(dict(
                mode=mode, date=daystr, issue=issue,
                parameter=parameter, ratio=ratio,
                parameter_value=original*ratio,
                base_objective=reference.objective + constant,
                objective=solution.objective + constant,
                objective_change_pct=100*(solution.objective-reference.objective)/
denominator,
                grid_11_kwh=float(abs(solution.grid-reference.grid).sum()),
                soc_max_change_kwh=float(abs(solution.soc_end-
reference.soc_end).max()),
                balance_residual=solution.balance_residual,
                soc_residual=solution.soc_residual))
    frame = pd.DataFrame(rows)
    frame.to_csv(output / 'instances.csv', index=False)
    # 汇总保留最大动作变化，避免只用小目标变化掩盖并列解敏感性。
    grouped = frame.groupby(['parameter', 'ratio'], sort=False)
    summary = grouped.agg(

```

```

instances=('objective', 'size'),
mean_objective_change_pct=('objective_change_pct', 'mean'),
max_abs_objective_change_pct=('objective_change_pct', lambda x: abs(x).max()),
max_grid_11_kwh=('grid_11_kwh', 'max'),
max_soc_change_kwh=('soc_max_change_kwh', 'max'),
max_balance_residual=('balance_residual', 'max'),
max_soc_residual=('soc_residual', 'max')).reset_index()
summary.to_csv(output / 'summary.csv', index=False)
protocol = dict(
    scope='32 fixed LP windows; not annual policy reoptimization',
    ratios=[0.8, 0.9, 1.0, 1.1, 1.2],
    solves=288, recorded_rows=len(frame),
    historical_states='formal S3 dispatch at the same decision time',
    baseline='same S0 reference day-ahead contract',
    parameters_selected_again=False,
    config_sha256=hashlib.sha256((frozen/'resolved_config.json').read_bytes()).hexdigest())
(output/'protocol.json').write_text(json.dumps(protocol, indent=2))
print(summary.to_string(index=False), flush=True)

```

```

if __name__ == '__main__':
    main()

```

附录3 公式推导与补充实验记录

3.1 充放电互斥变换的可行性

令 $z = \eta_c C - D / \eta_d$ ，其中 $C, D \geq 0$ ， $0 < \eta_c, \eta_d \leq 1$ 。取 $C' = \max(z, 0) / \eta_c$ ， $D' = \eta_d \max(-z, 0)$ ，则 $\eta_c C' - D' / \eta_d = z$ ，故变换前后储电量增量相同，且 $C'D' = 0$ 。

当 $z \geq 0$ 时， $C' = C - D / (\eta_c \eta_d)$ ， $D' = 0$ ，因此 $0 \leq C' \leq C$ ；母线净需求减少 $D(1/(\eta_c \eta_d) - 1) \geq 0$ 。当 $z < 0$ 时， $C' = 0$ ， $D' = D - \eta_c \eta_d C$ ，因此 $0 \leq D' \leq D$ ；母线净需求减少 $C(1 - \eta_c \eta_d) \geq 0$ 。原功率上界与储电量边界均保留。允许免费处置富余电量时，可通过增加未用富余或减少采购恢复供需平衡，费用不会增加。该结论依赖本文关于弃电、价格与运行约束的假设。

3.2 单期采购分位点与结算差额

设无储能单期净需求为 N ，采购 q ，固定价格 $p > 0$ ，目标 $g(q) = pq + 5pE[(N - q)^+]$ 。连续分布下 $g'(q) = p - 5p\Pr(N > q)$ ，驻点满足 $\Pr(N \leq q) = 0.8$ ；目标凸，故该分位点为最优解。离散分布下对应条件为 $F(q-) \leq 0.8 \leq F(q)$ 。该结论仅解释单期非对称损失，正式模型仍求解耦合储能的线性规划。

对每个交付时段 t ，令 $\Delta A_t^k = A_t^k - A_t^{k-1}$ ，则各次变化之和为最终承诺与初始承诺之差。主结算和逐次结算中的初始计划费、线性变化费与紧急采购费相消，只剩 $0.5 \sum_t p_t (\sum_k |\Delta A_t^k| - |\sum_k \Delta A_t^k|)$ 。当同一时段各次采用相同非负价格时，由三角不等式，该差额非负，即式 (13)。

3.3 参数调试与额外实验索引

参数候选、选择规则与费用见正文表 4；参数验证明细、解析后的锁定配置和原始输入校验记录随支撑材料交付。指定日期求解器复求、全年替代回放、月度费用、尾部风险、预测误差及块长度敏感性保存于 `research` 目录，完整文件路径与复算命令见配套阅读说明。新增参数扰动子目录保存输入检查、实验协议、320 条实例及汇总，复算程序全文见附录 2.2；这项诊断未改动正式模型参数和结果工作簿。